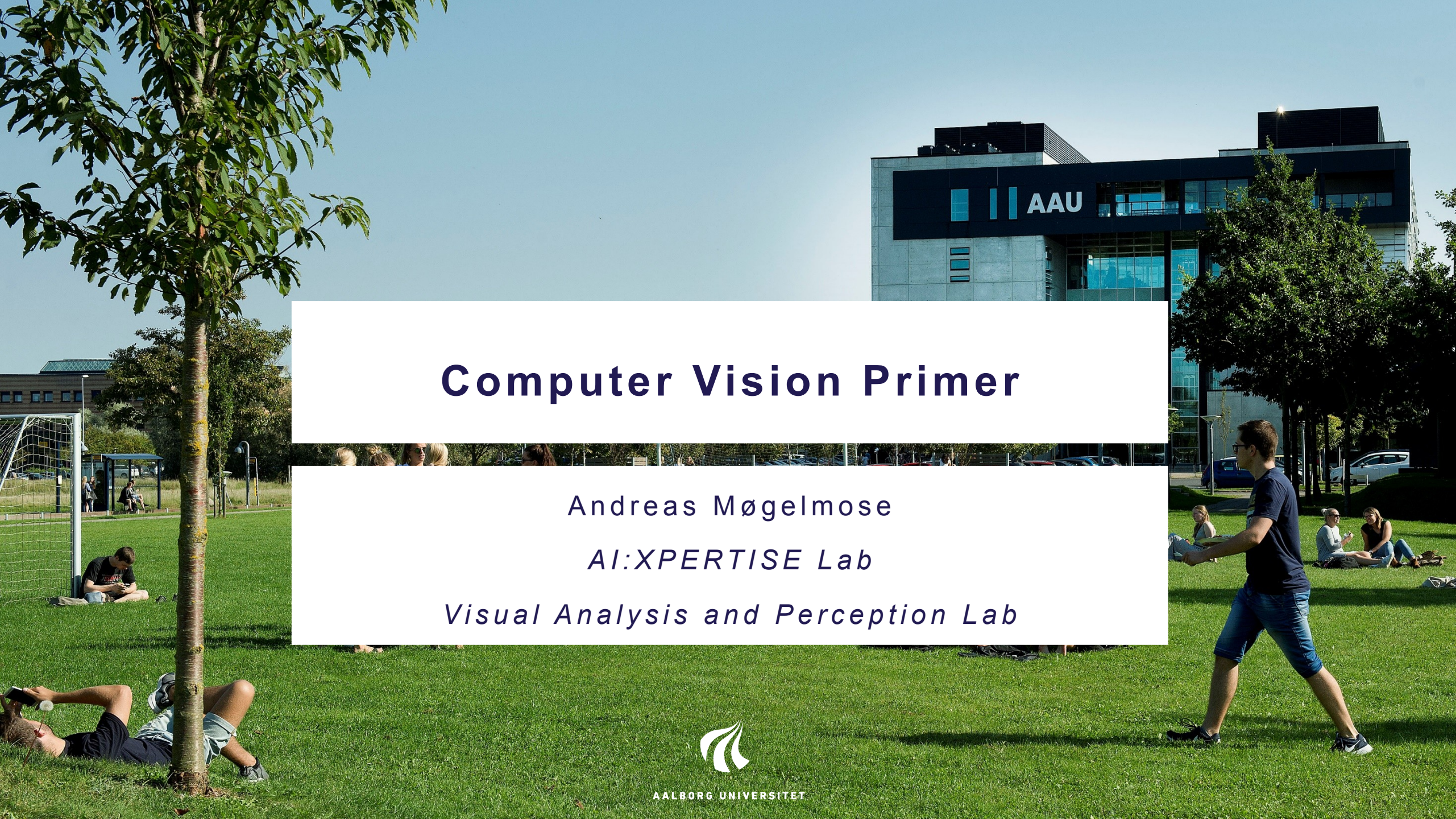


Computer Vision Primer

Andreas Møgelmose

AI:XPERTISE Lab

Visual Analysis and Perception Lab

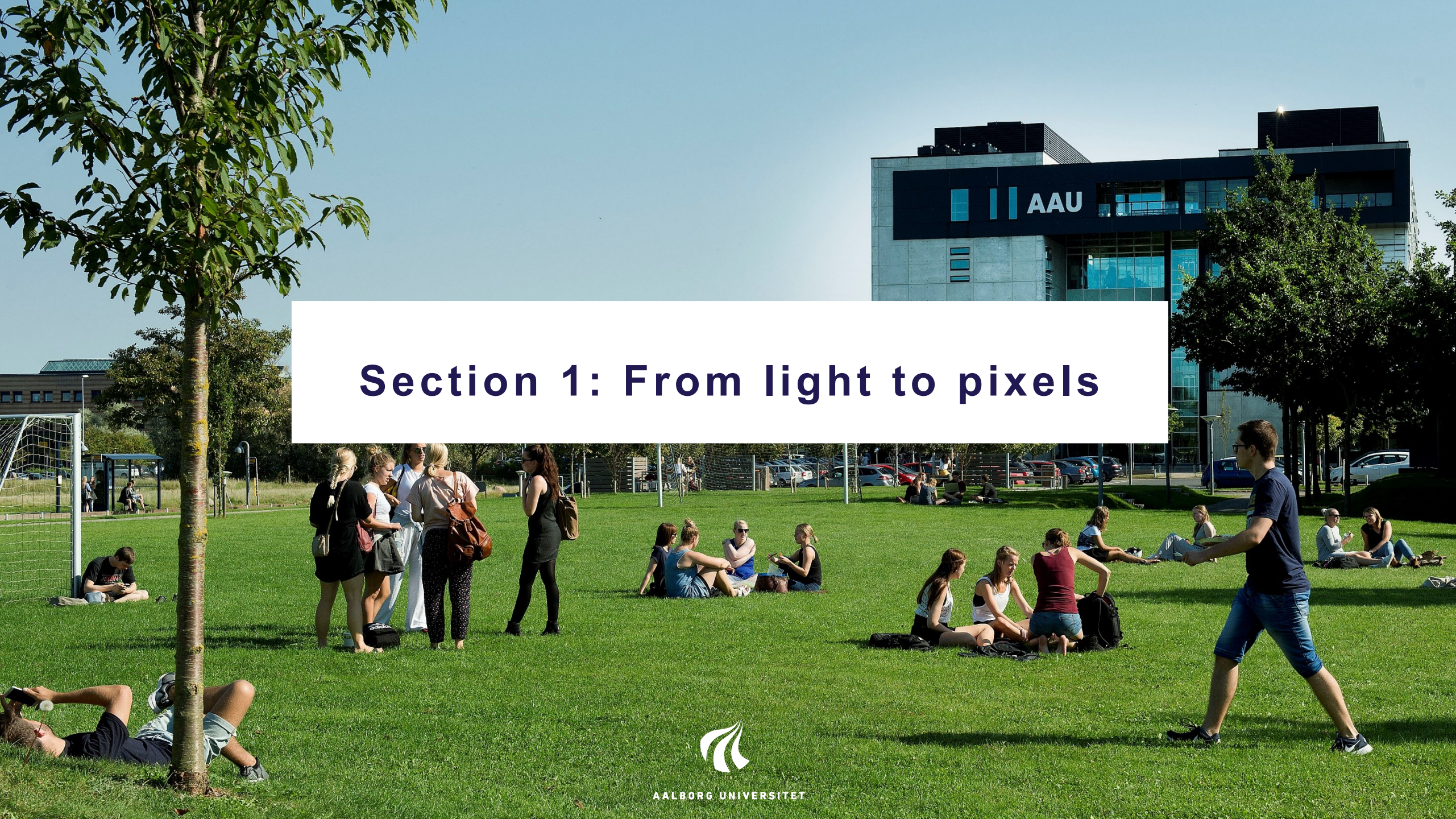
AALBORG UNIVERSITET

Agenda

- Section 1: From light to pixels, and basic image processing
  - Exercise 1 (10 min): Segment and count objects
- Section 2: Features
  - Exercise 2 (15 min): Build a Harris corner detector
- Section 3: Deep learning

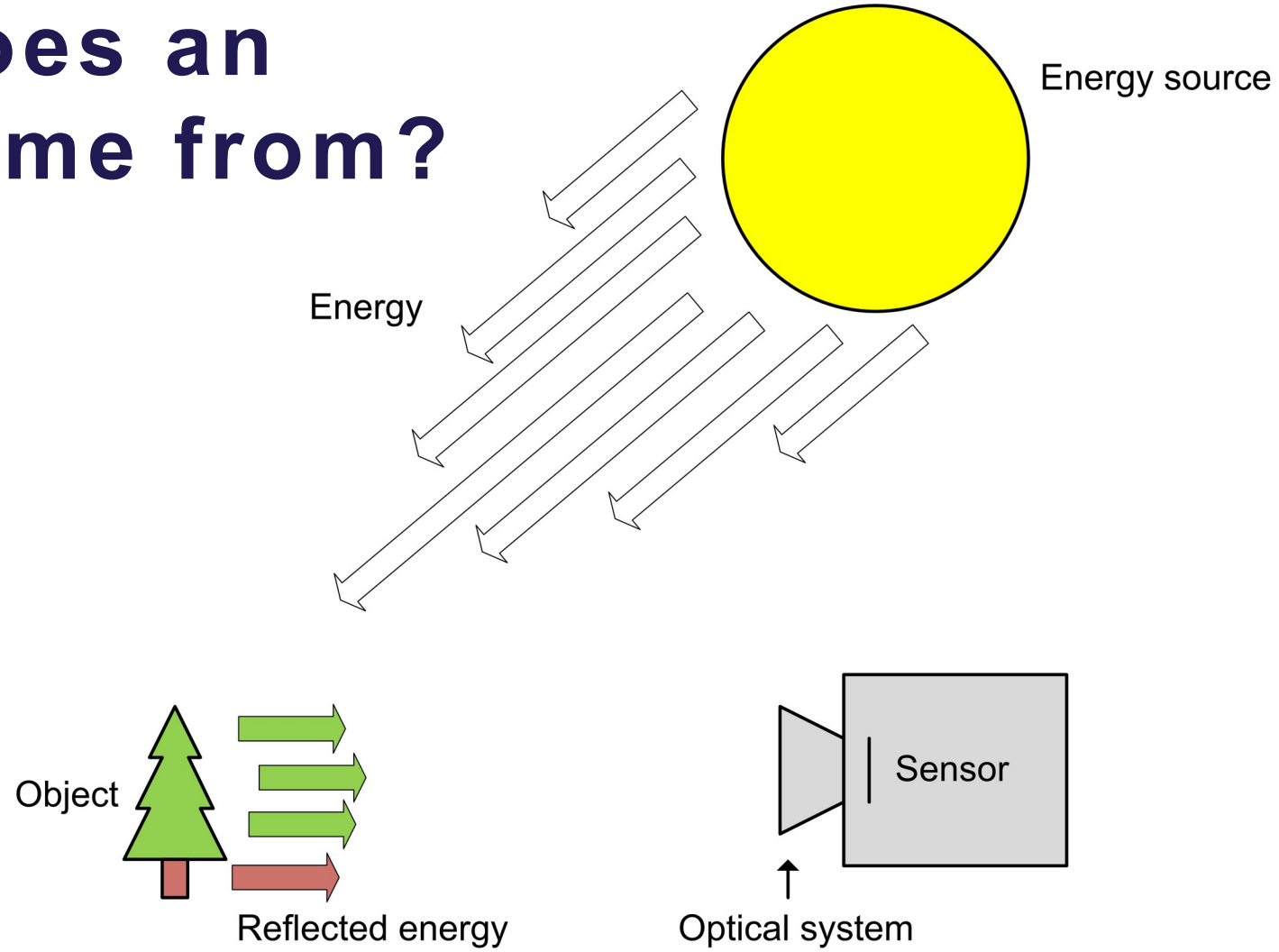
- An image is a measurement of light: numbers with physics and noise behind them.
- Classical methods exploit structure in those numbers by hand while deep learning learns it from data.



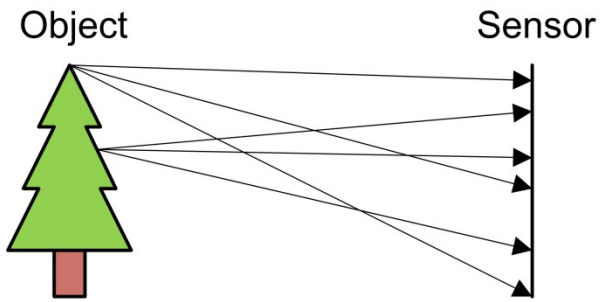


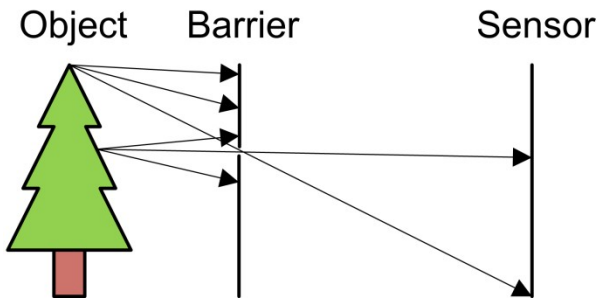
Section 1: From light to pixels

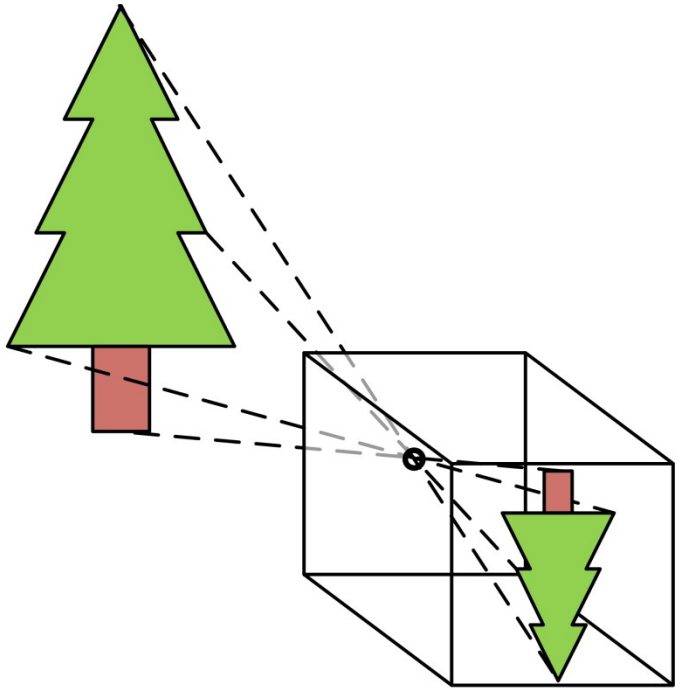
Where does an image come from?



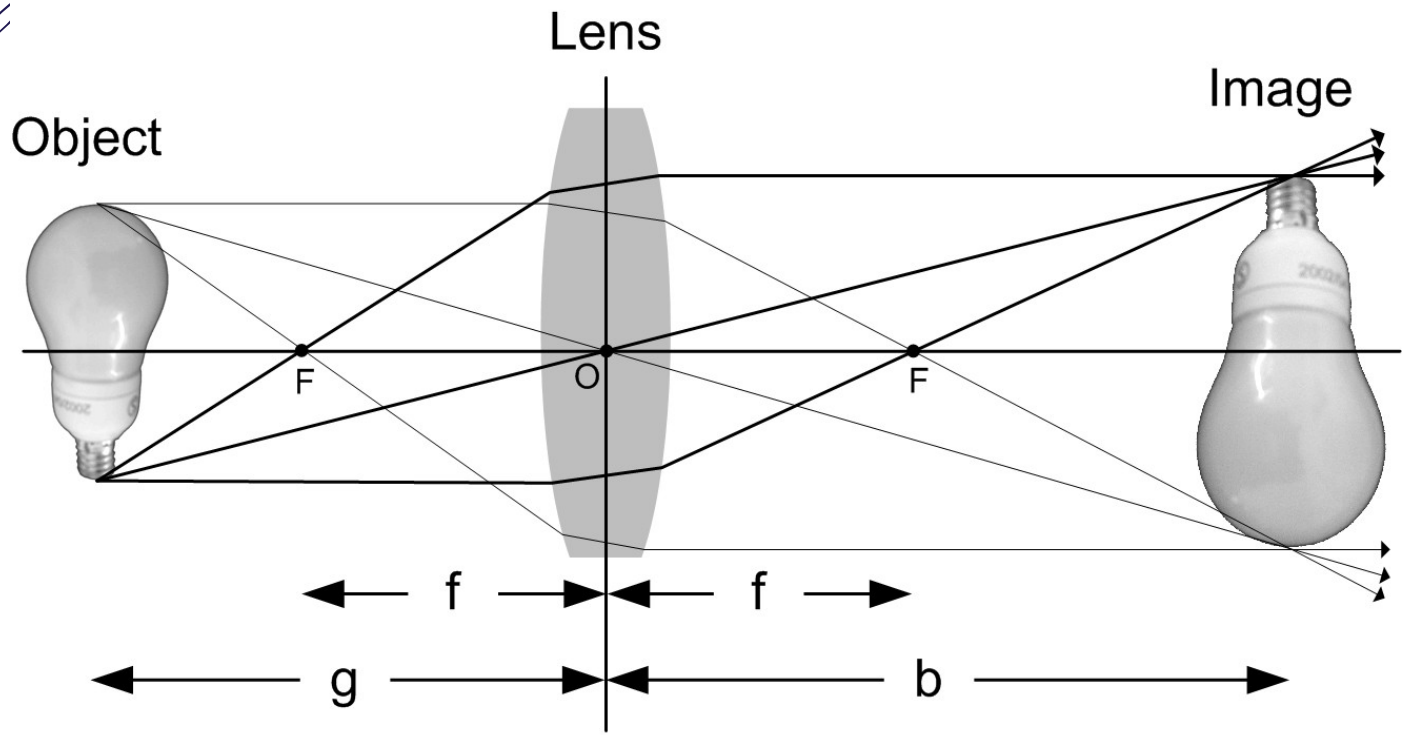
The lens







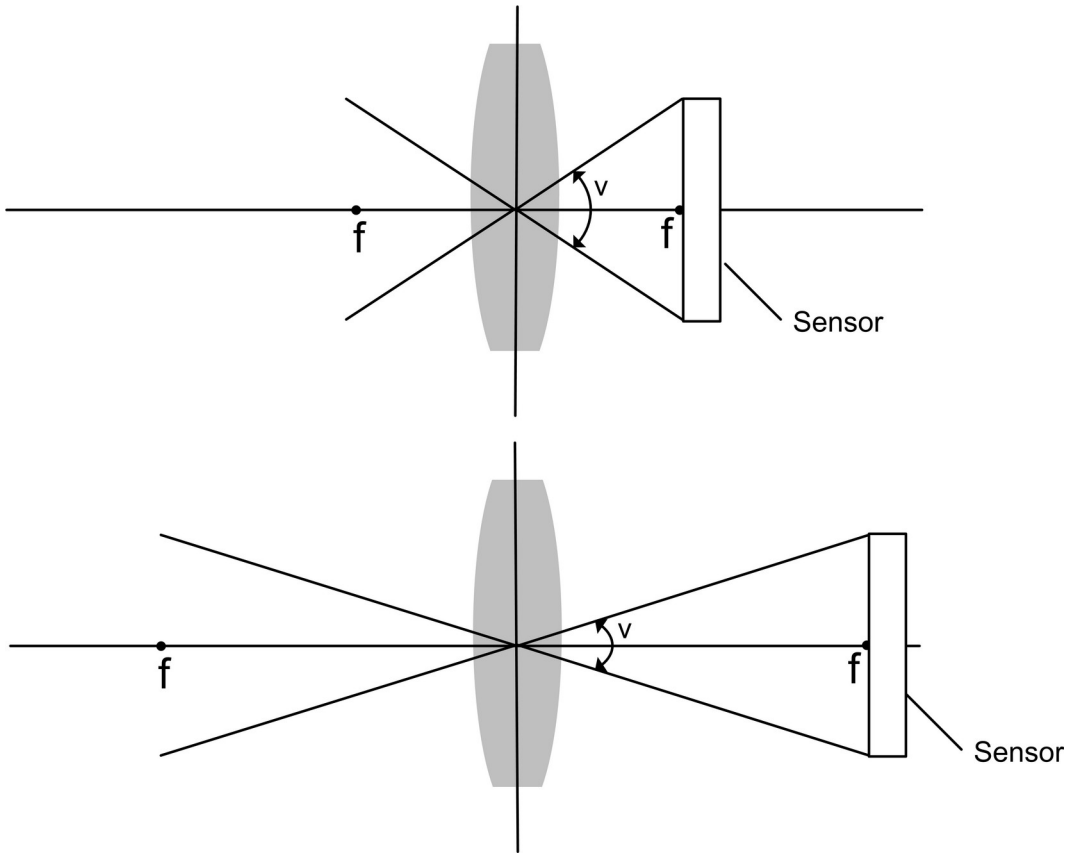
The lens

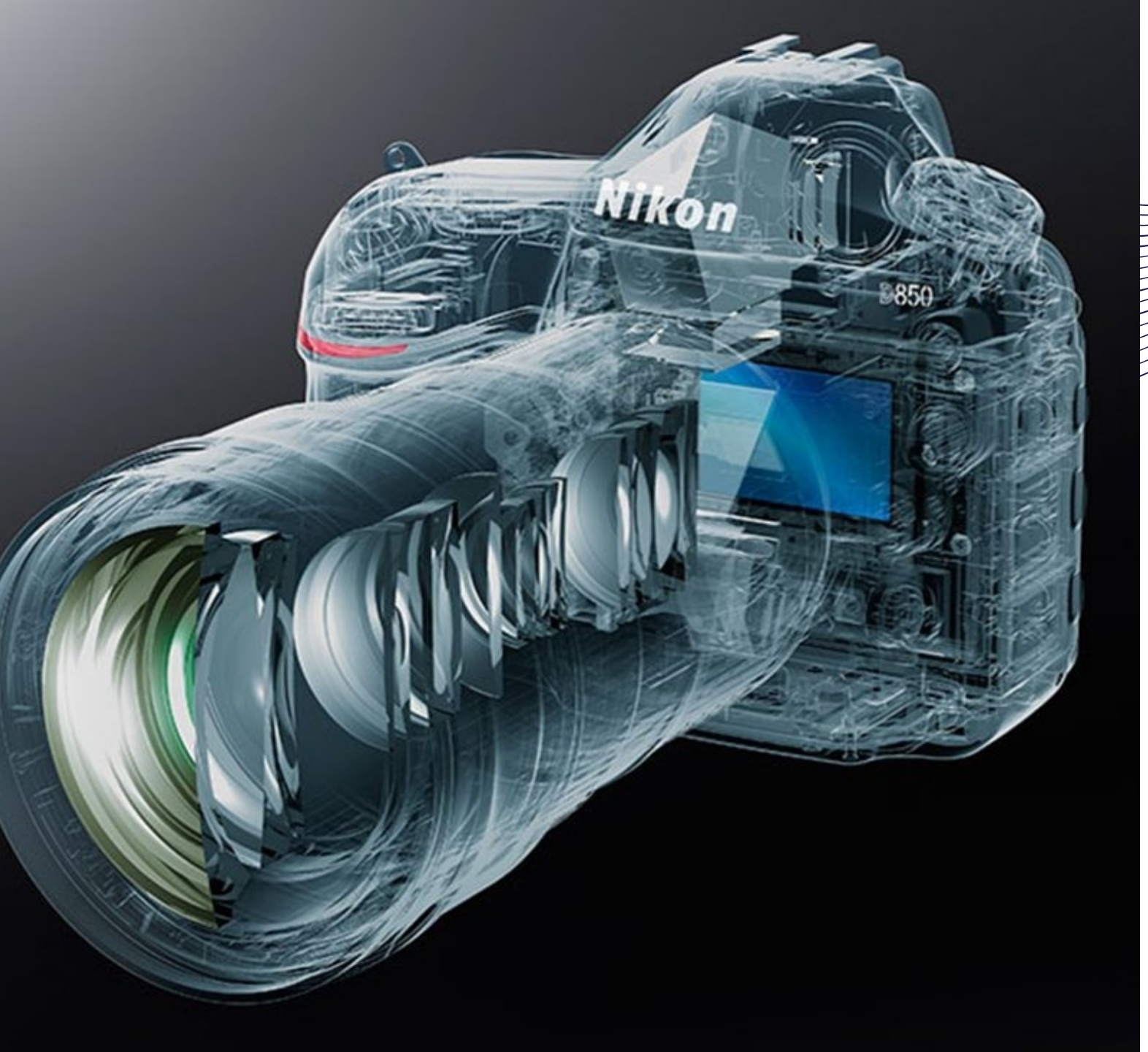


- A lens focuses a bundle of rays to one point
- Parallel rays pass through a focal point ***F*** at a distance ***f*** beyond the plane of the lens. ***f*** is the focal length
- ***O*** is the optical center
- ***F*** and ***O*** span the optical axis

Field of view

- Field-of-view, v, depends on size of the sensor and focal length
- ”Fisheye” lens → small focal length and large field-of-view
- Horizontal FOV and Vertical FOV

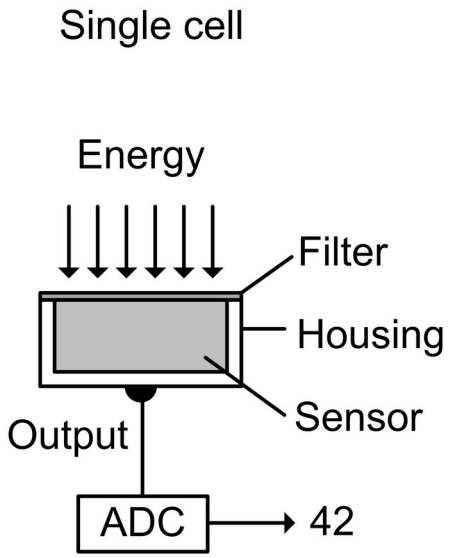


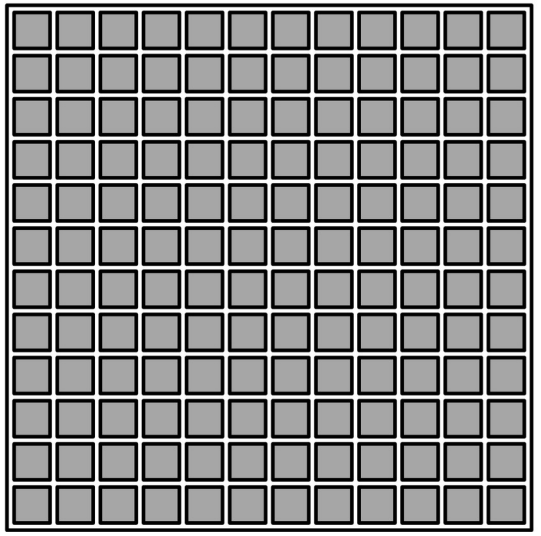


Objective

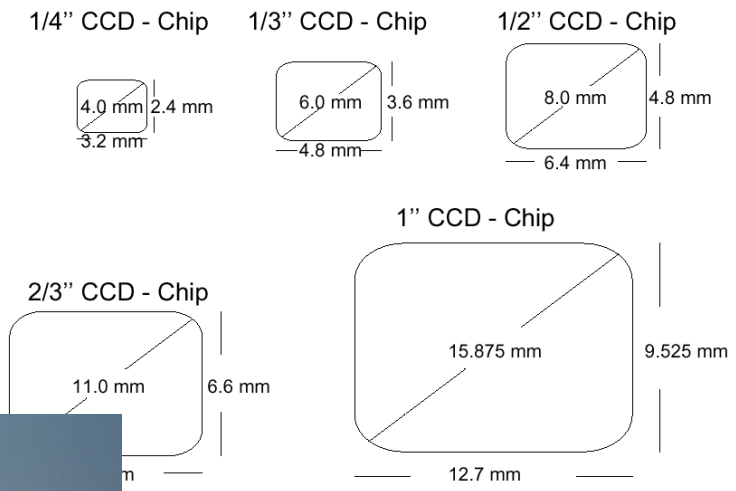
- Contains many lenses
- May allow you to change focal length = zoom = adjust field of view

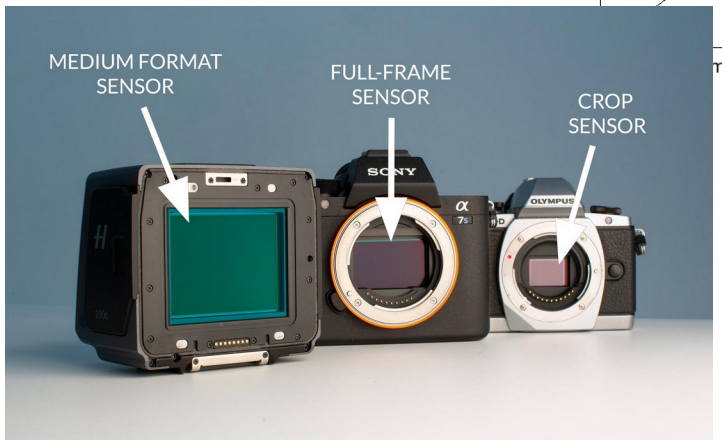
Sensing light





Sensor chip formats





- Number of pixels
  - from 500x500 (0.25 MP)
  - to 6000x6000 (36 MP)
  - … and more – even gigapixel-resolution
- Pixel size
  - from 4 µm x 4 µm
  - to 16 µm x 16 µm

Exposure parameters

- Shutter speed
- Aperture
- ISO

- Balance these against each other, so the correct amount of light hits the sensor.









Sensing light



Correctly exposed



Over exposed



Under exposed



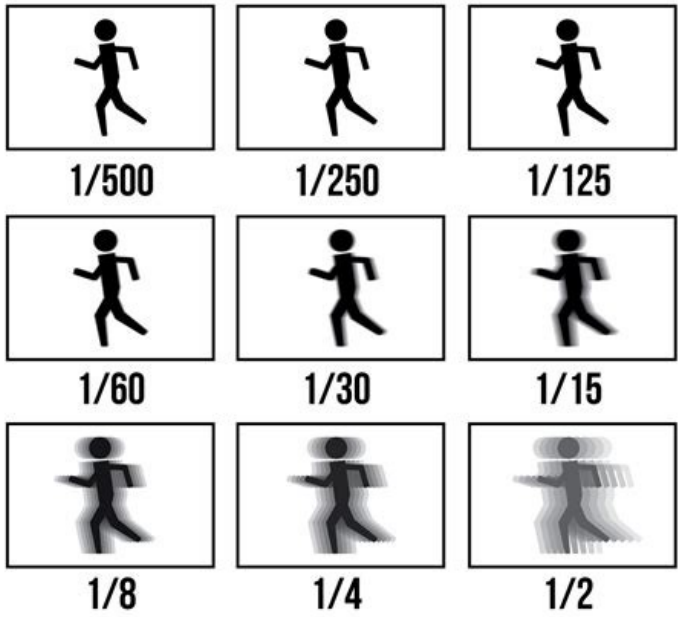
Motion blur

- Integration over time
  - Exposure time
    - Shutter time (DK: lukkertid)
  - Motion blur
  - Maximum charge
    - Saturation
    - Blooming (vertical streaks on CCD-chips)

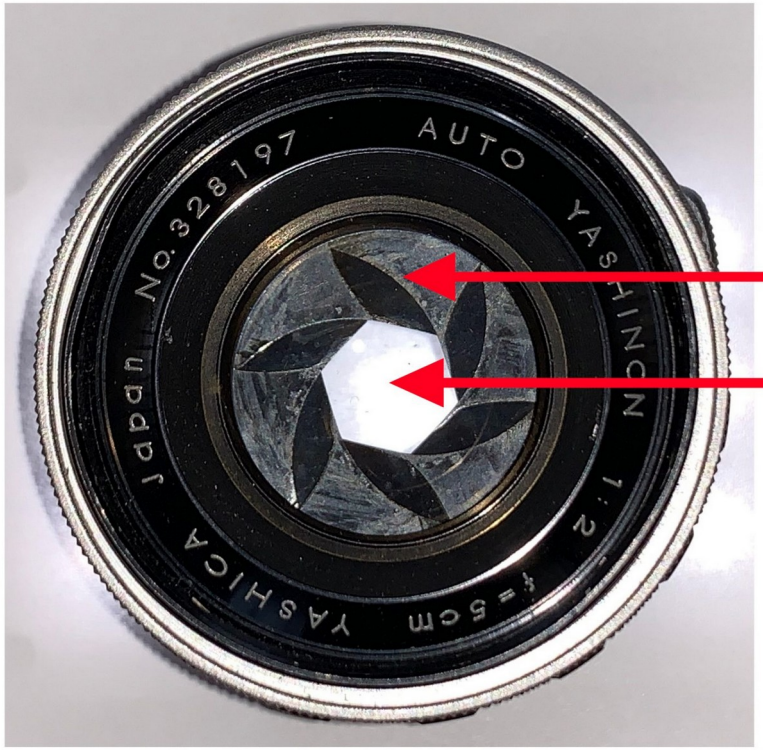
Shutter speed







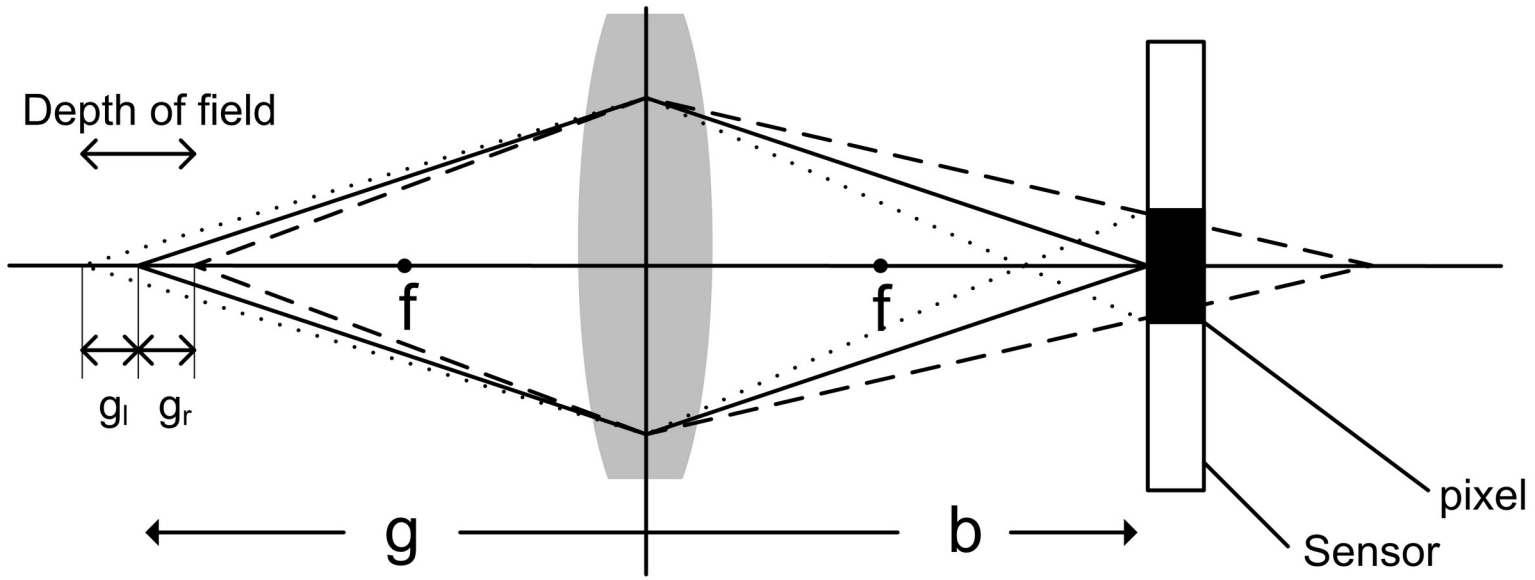
Aperture





Measured in fractions: f/1.2, f/4, f/32

Focus and depth of field



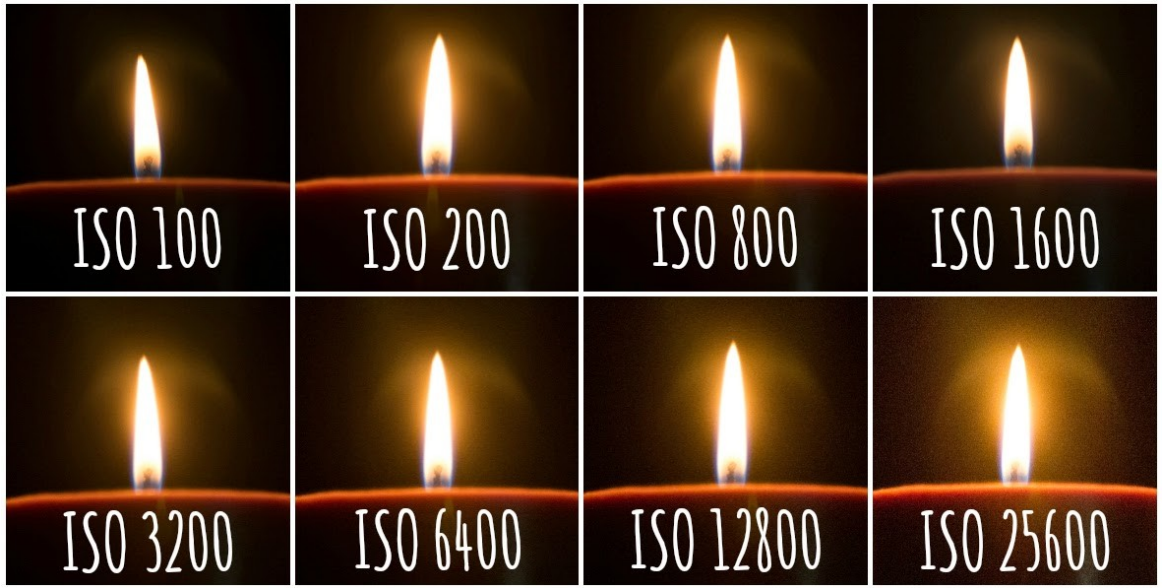
- Distance range in which the blur does not exceed a certain value
- Also known as DoF







ISO


ISO 100
ISO 200
ISO 800
ISO 1600
ISO 3200
ISO 6400
ISO 12800
ISO 25600

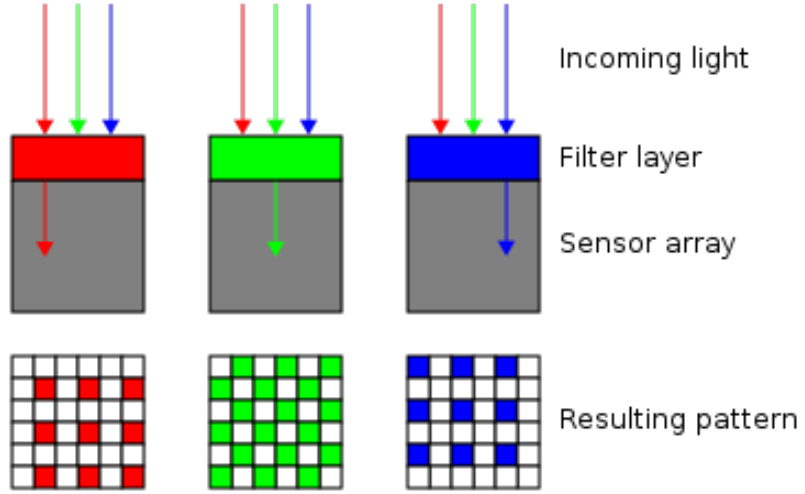

ISO 200
ISO 1000
ISO 1600
ISO 3200
ISO 6400

Single sensor color (most cameras)

Use achromatic sensors but filter the light in a certain pattern: The Bayer pattern.

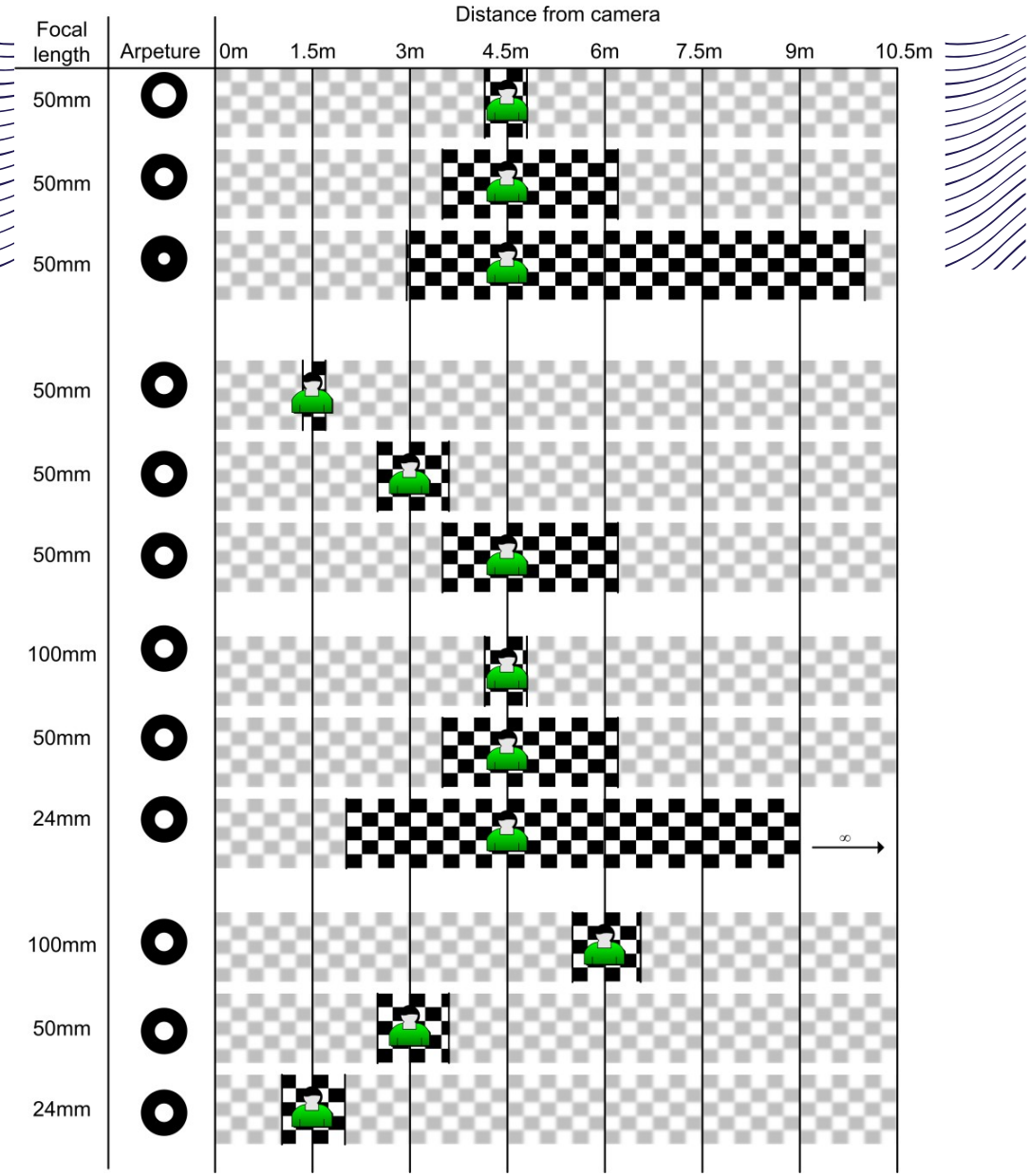
Demosaicing: Finding the RGB-value for a particular pixel.

- Set the color component for the filter color directly.
- Set the two other values by averaging the neighbors with that filter.
- Examples: For a G pixel, set G = value, R = mean($R_{neighbors}$) and B = mean($B_{neighbors}$)



B	G	B	G	B	G	B
G	R	G	R	G	R	G
B	G	B	G	B	G	B
G	R	G	R	G	R	G
B	G	B	G	B	G	B
G	R	G	R	G	R	G
B	G	B	G	B	G	B

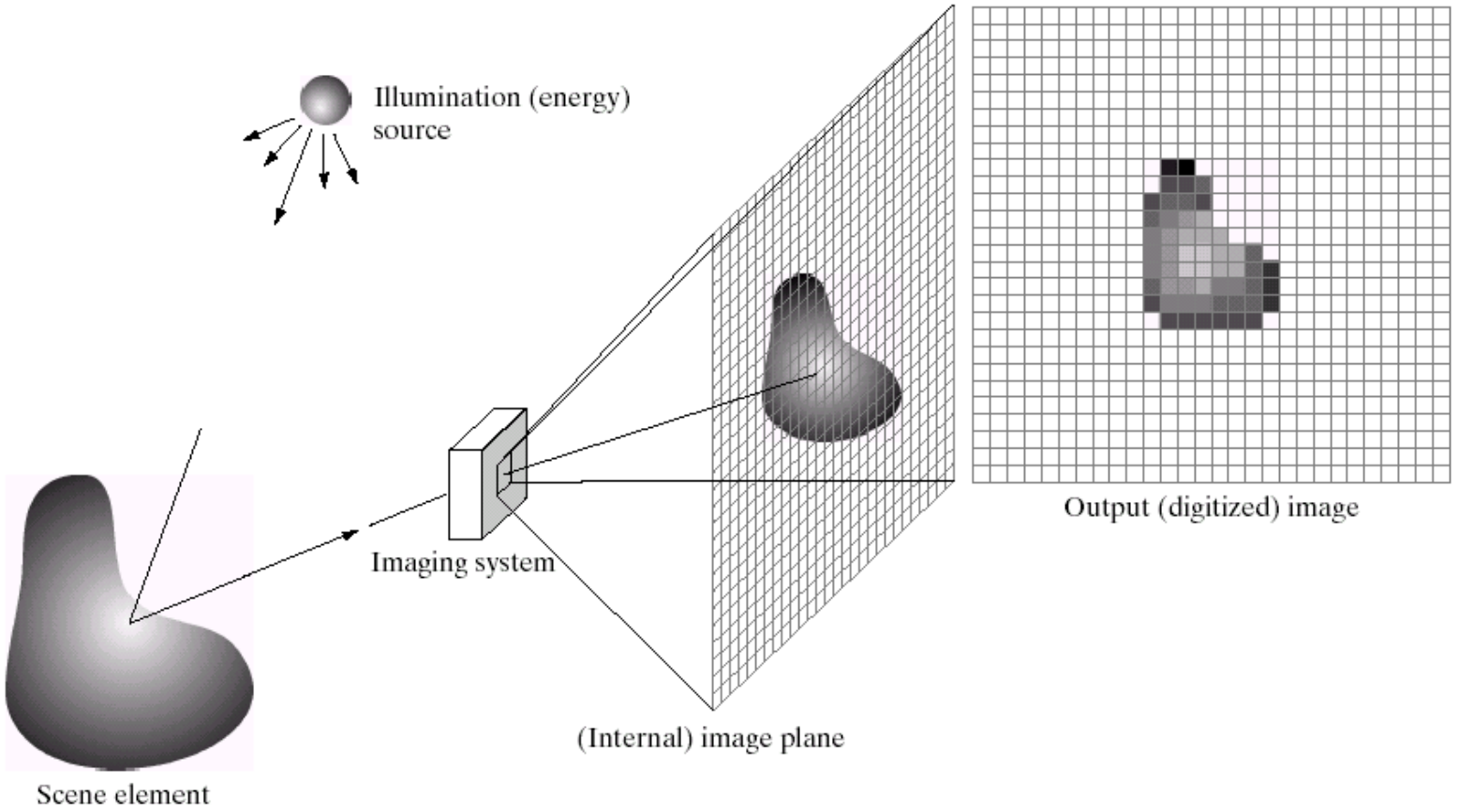
Bayer pattern



A good picture

- Issues to consider:
  - Dist. to object
  - Motion of object
  - Focus
  - Depth-of-field
  - Focal length/Field-of-view
  - Shutter
  - Aperture
  - Sensor (size and type)
  - ISO

Pixels



a
b c d e

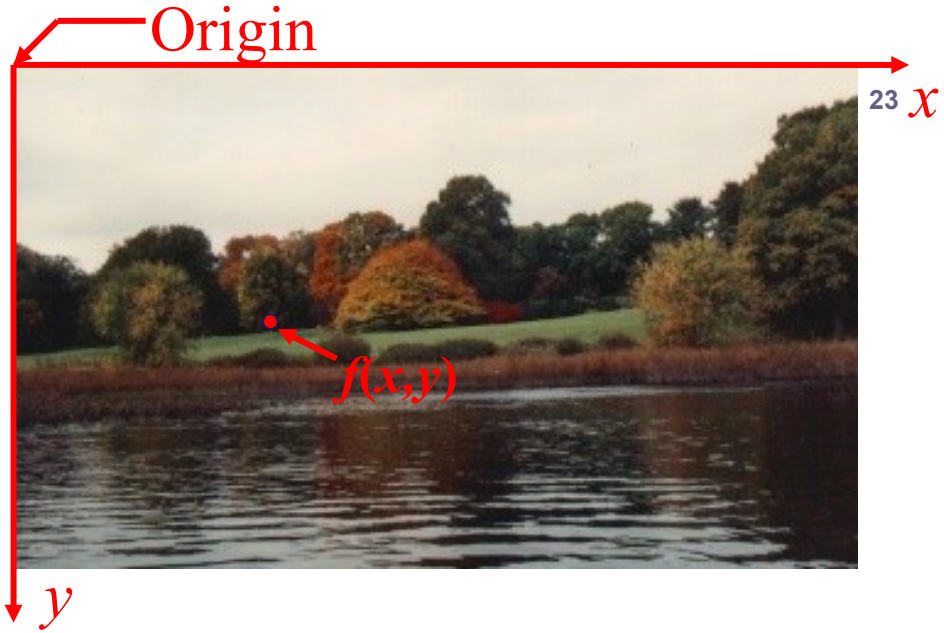
FIGURE 2.15 An example of the digital image acquisition process. (a) Energy ("illumination") source. (b) An element of a scene. (c) Imaging system. (d) Projection of the scene onto the image plane. (e) Digitized image.

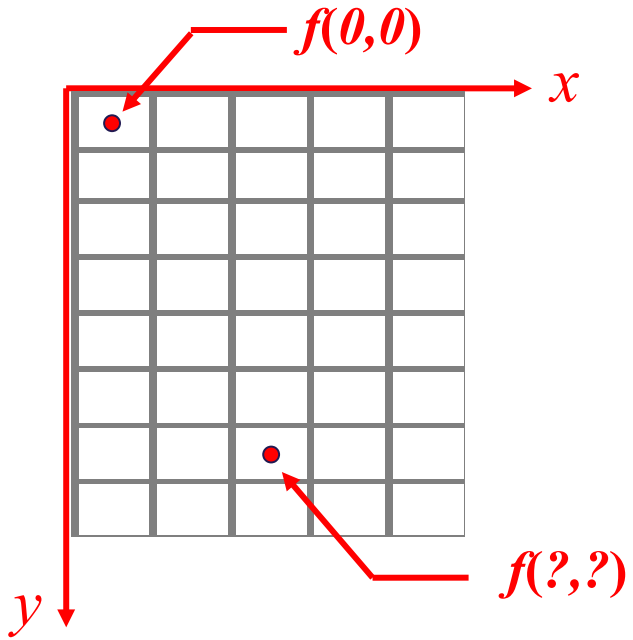
Pixel coordinates

- An image ***f*(*x*,*y*)** is represented as a 2D ***Array, matrix***
- An image is seen as a **discrete function *f*(*x*,*y*)** as opposed to a continuous function

- x and y **cannot** take on any value! They must be within the bounds of the image.
- Width = number of pixels in x-direction
- Height = number of pixels in y-direction
- Size = (width x height)

- ROI = region of interest
  - To reduce the amount of data



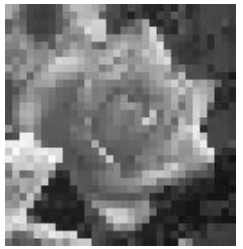
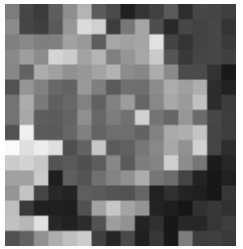


Resolution

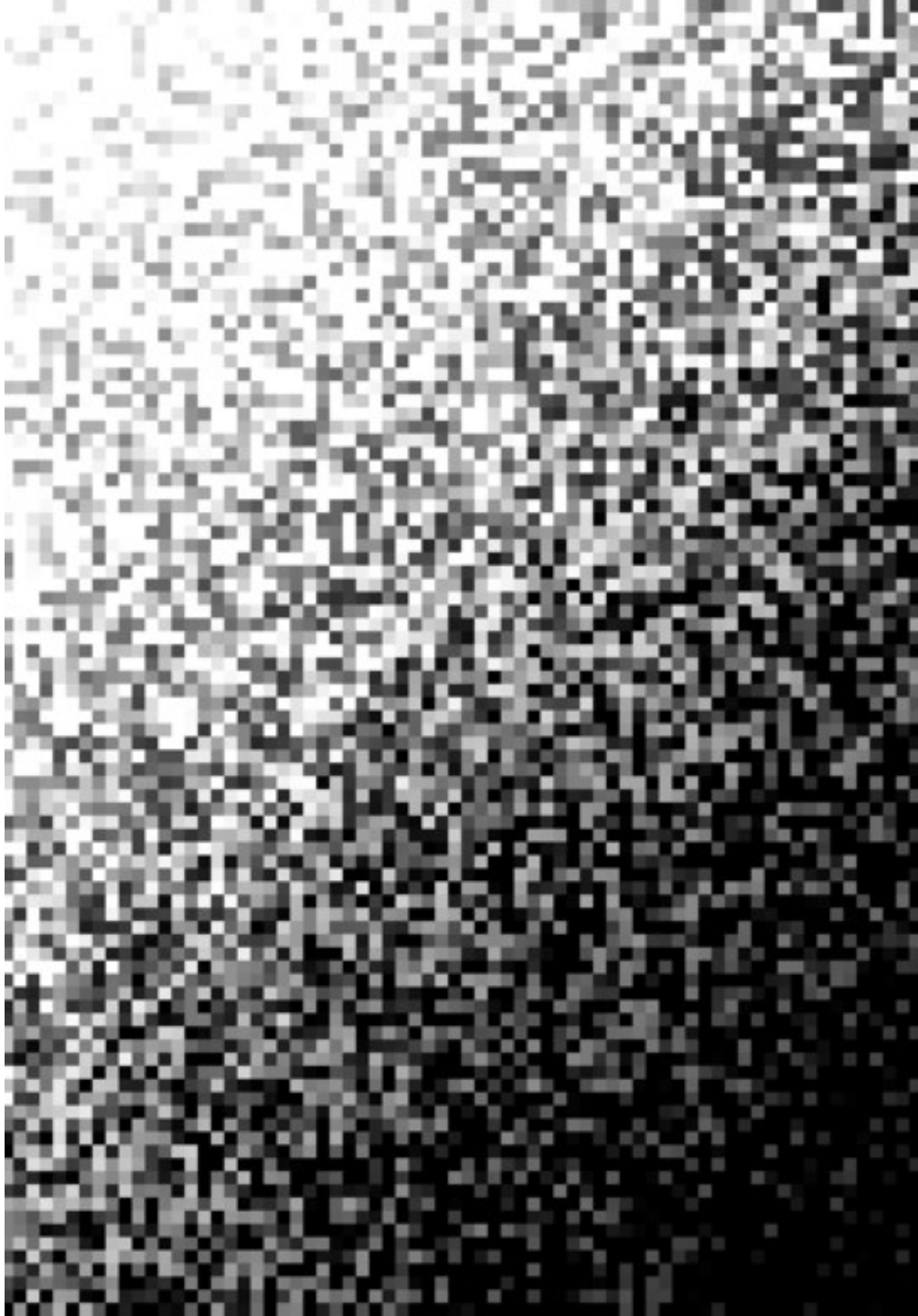
- The size of an area in a scene that is represented by one pixel in the image

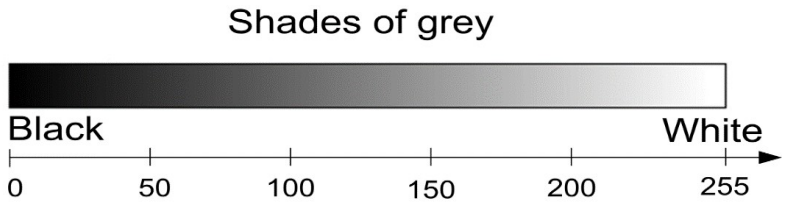
- Different Resolutions are possible (4000x3000 … 256x256 ... 16x16)



- Lower resolution leads to data reduction!









Pixel representation

- A few words on bits and bytes: One bit: {0,1}
- One pixel: one byte = eight bits = one number: [0,255]

- Grey-scale, intensity, black/white: 8 bits = [0,255]
- Binary image: 1 bit {0,1}. Black and white: visualized as: 8 bit {0,255}

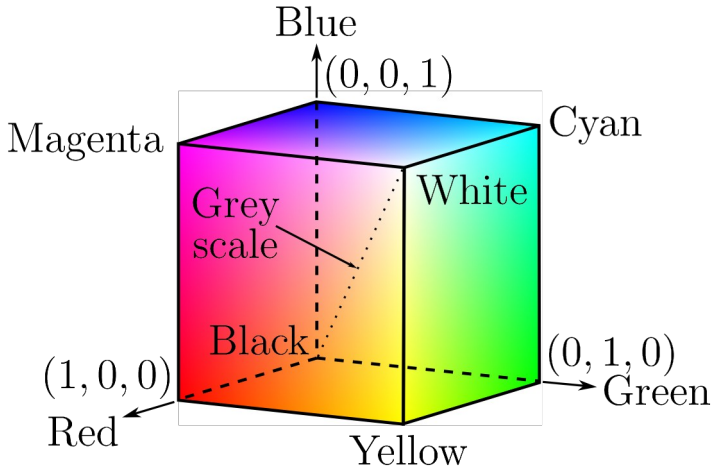
Grey-level resolution: Quantization

- Different gray-level resolutions: 256, 128, …, 2
- Fewer gray-levels leads to data reduction.
- For 256, 128, 64 gray-levels: Difference hardly visible



RGB color space

- The “classical” Computer Color space
- 3 different colors: Red, Green, Blue
- Similar to the human visual system!
- If R,G,B have the same energy, we perceive a shade of grey



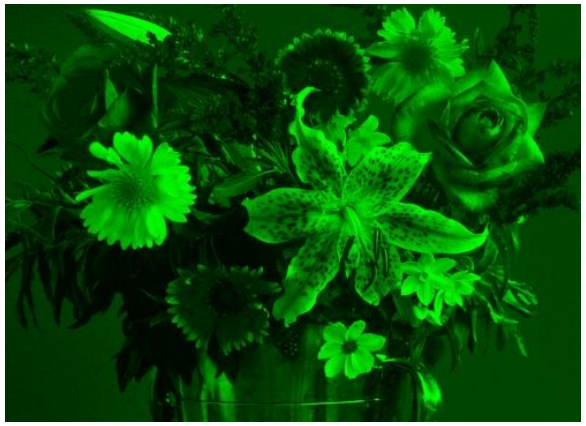
RGB example



Original Image



R-Component



G-Component



B-Component

Converting color to grayscale



?



- $I = (R+G+B) / 3$

$$\left(\quad + \quad + \quad \right) / 3 =$$

- $I = W_R \times R + W_G \times G + W_B \times B, \qquad W_R+W_G+W_B=1$

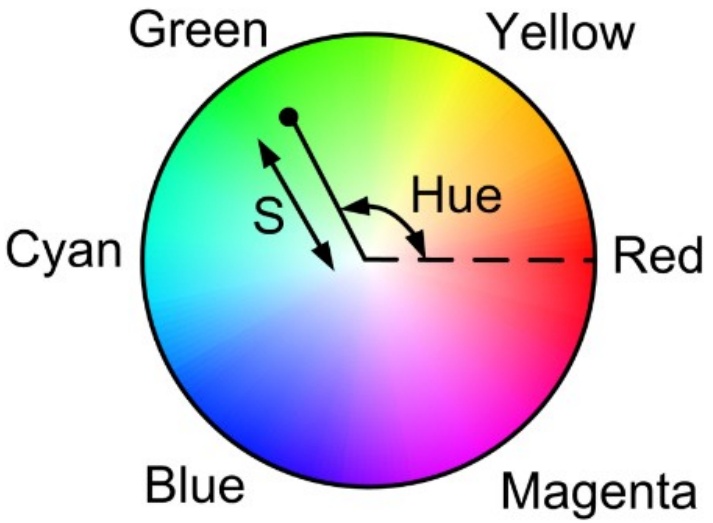
HSI color space to the rescue!

- Hue: The dominant wavelength in the mixture of light waves, as perceived by an observer. The “pure” color.
- Saturation: Relative purity; inverse of the amount of white light mixed with hue
- Intensity: The average of R, G, B









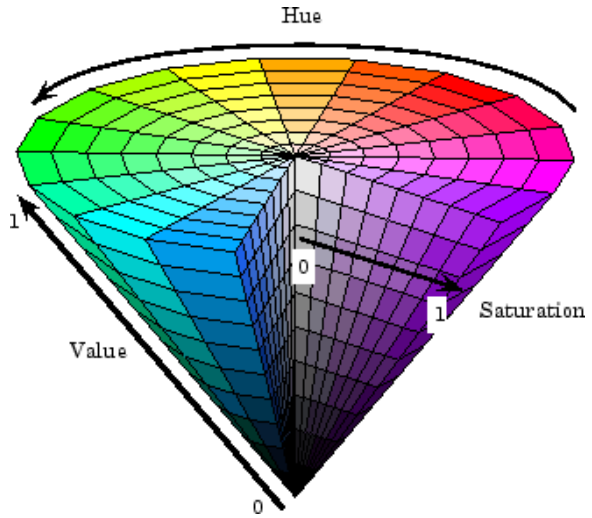
- **RGB → HSI**

$$\theta = \cos^{-1}\left\{\frac{[(R-G)+(R-B)]/2}{\sqrt{(R-G)^2+(R-B)(G-B)}}\right\}$$

$$H = \begin{cases} \theta & B \le G \\ 360-\theta & B > G \end{cases}$$

$$S = 1 - \frac{3\cdot\min(R,G,B)}{R+G+B}$$

$$I = (R+G+B)/3$$



HSI example



Original Image



Hue



Saturation



Intensity

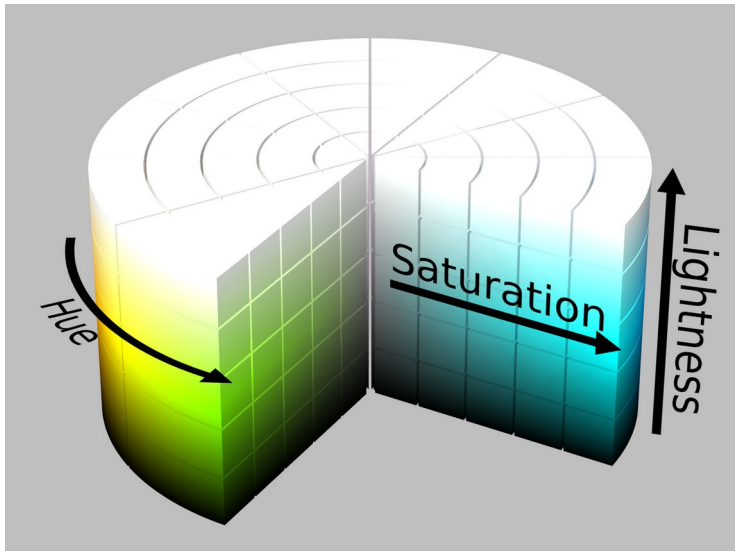
HSI, HSV, HSL, HSB

- Note: HSI is closely related to HSV, HSL, or HSB (for Value, Lightness, and Brightness).
- There are minor differences, but they do pretty much the same thing.
- Look them up if you care.

- OpenCV has a built in converter:

```
cvtColor(in, out, COLOR_BGR2HSV)
```

- Know that quite a few other color spaces/color models exist.



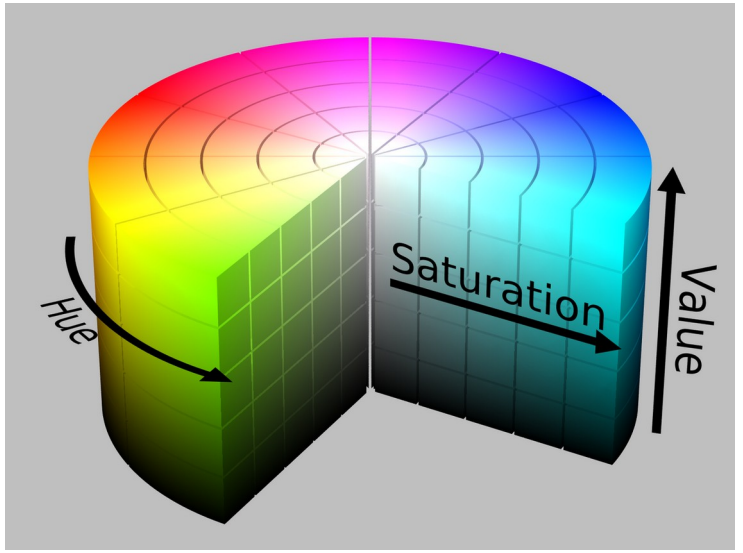
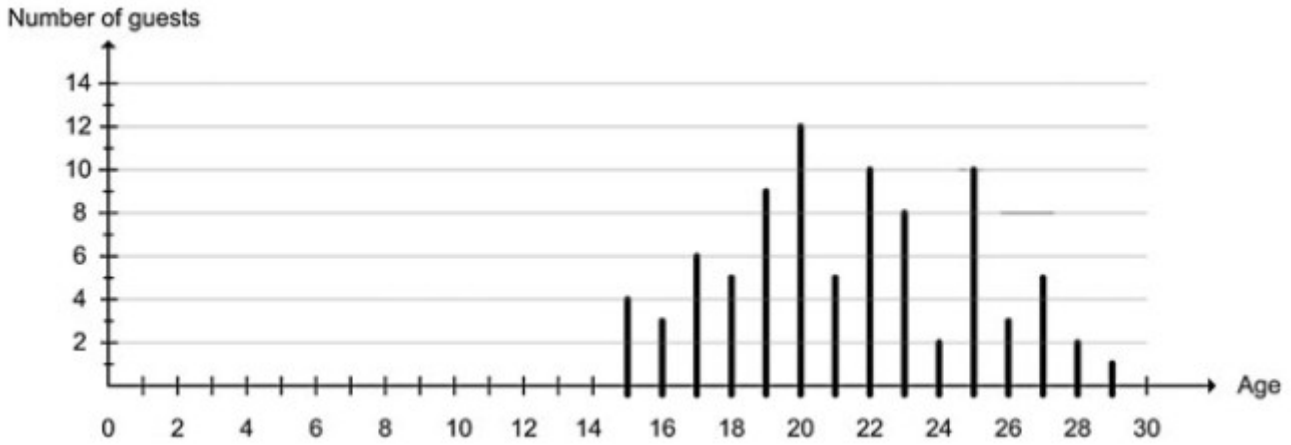
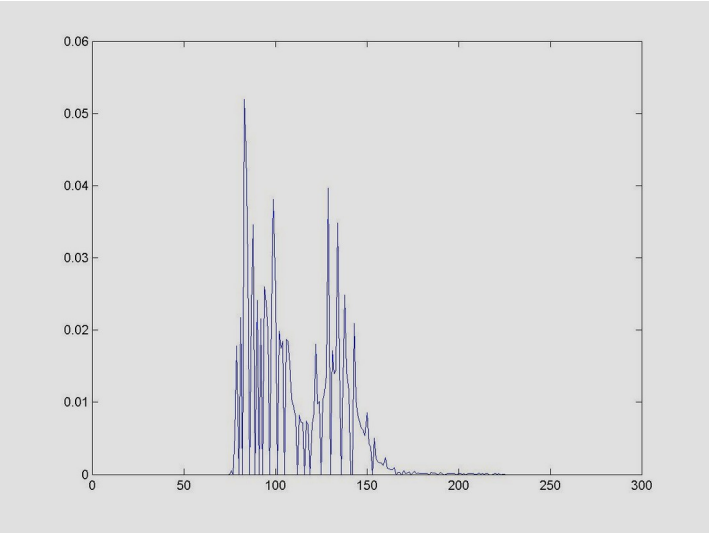


Image histograms



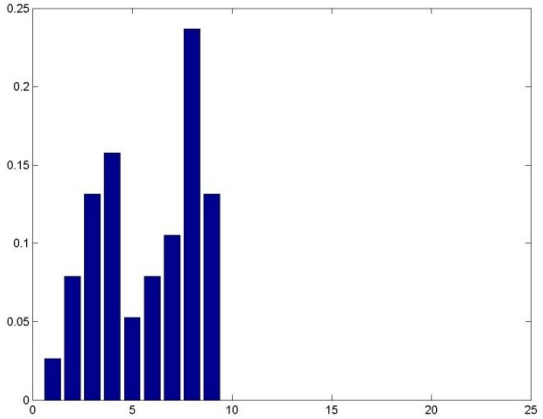
- In general, histograms are about counting members of (sortable) categories. Here we count pixels of different values.
- Histogram processing is a very powerful tool!
- A histogram is a discrete function $p(k) = n_k / n$
  - n_k is the number of pixel with the k-th gray level
  - n is the total number of pixels
  - Histogram entry $p(k)$ gives the probability of gray value k appearing in the image

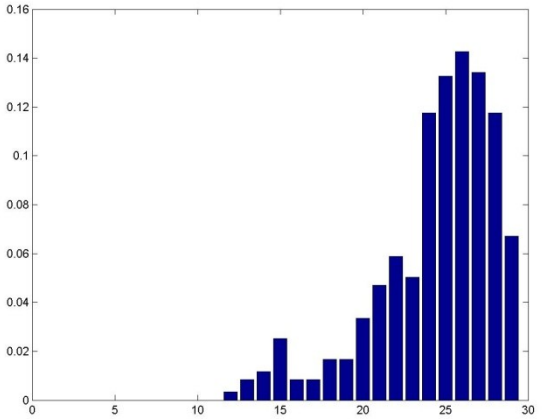


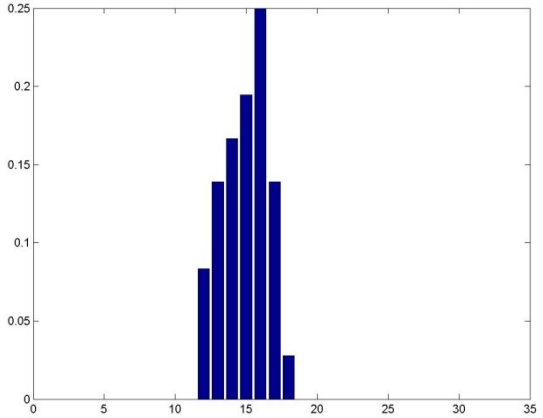


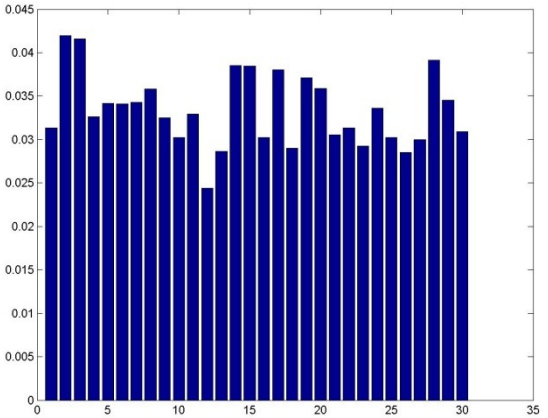
Histogram shapes

- Which of these are:
  - Low brightness?
  - High brightness?
  - Low contrast?
  - High contrast?





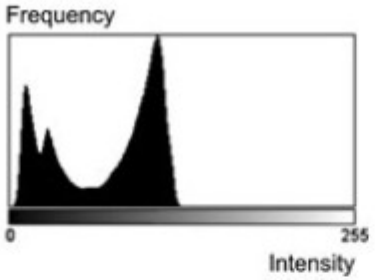




Histogram shapes

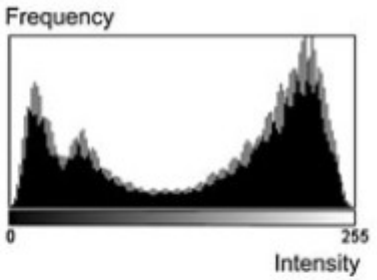


Dark image



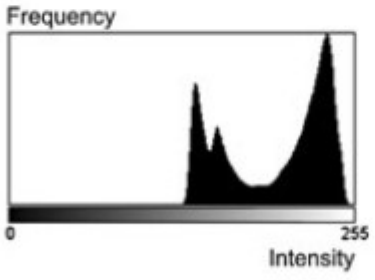


High contrast image



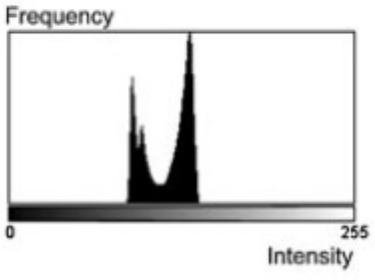


Bright image

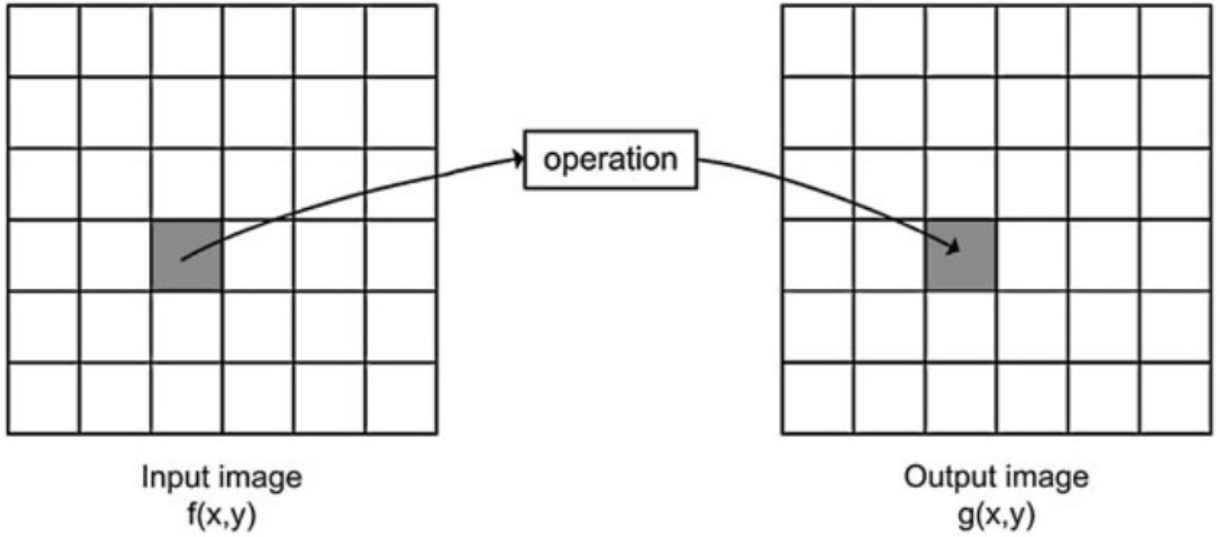




Low contrast image



What is point processing?



- Only one pixel in the input has an effect on the output
- For example: Changing the brightness, thresholding, histogram stretching

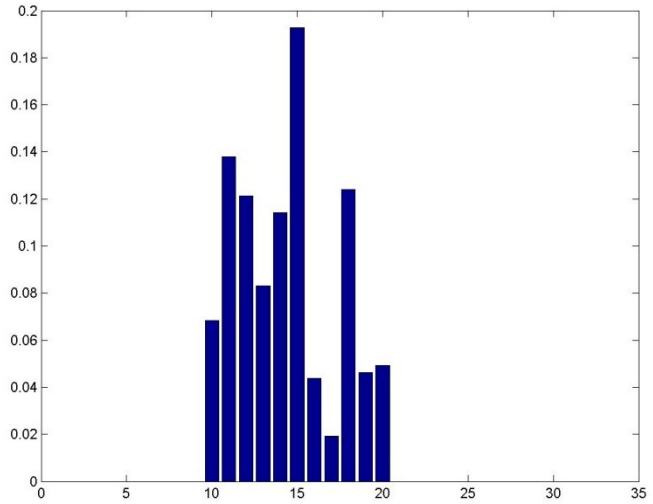
Gray level enhancement

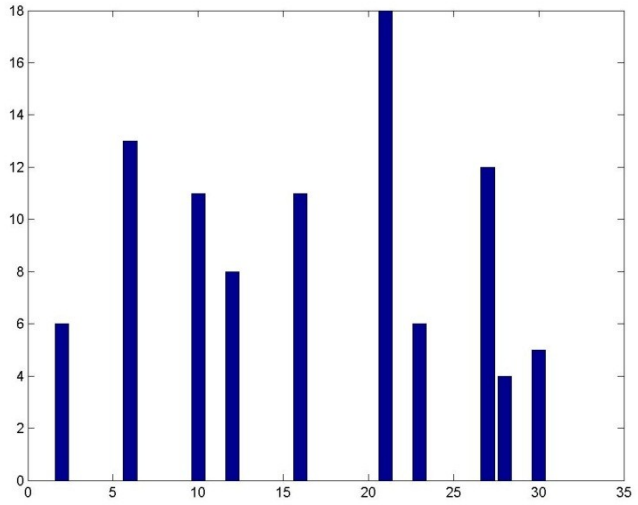
Correct | Too high brightness | Too low brightness | Too high contrast | Too low contrast



Improving contrast

- Humans cannot tell the difference between greylevel values too close to each other
- So: spread out the greylevel values
- This is called histogram stretching



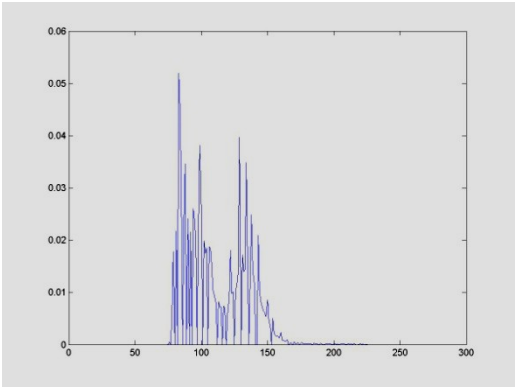


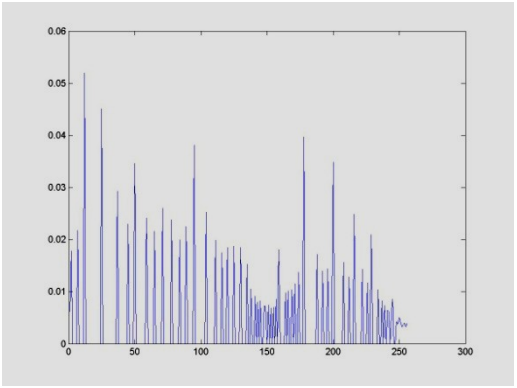
Histogram stretching



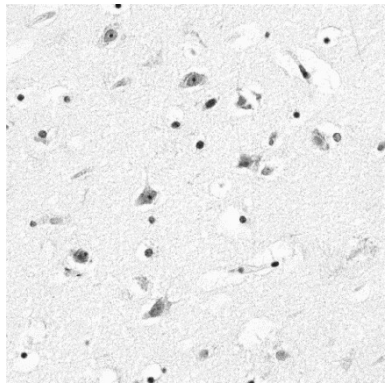








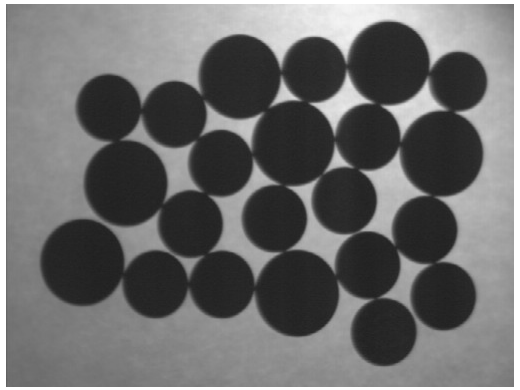
Segmentation



- Segmentation is about image analysis
- Not image manipulation



- The task:
  - Information versus noise
  - Foreground (object) versus background



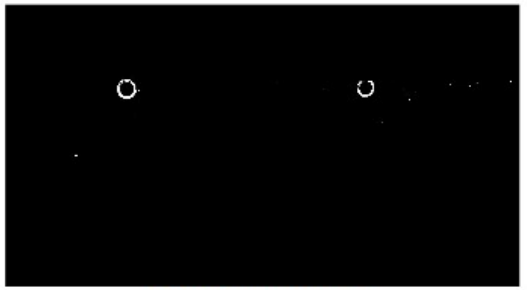
Thresholding

- Use graylevel mapping and the histogram
- When two peaks (modes) of a histogram correspond to object and noise
- Find a **threshold** value, T, that separates the two peaks. This process is called thresholding.

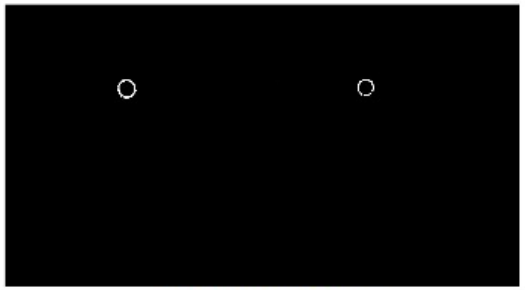
- Algorithm:
  - If f(x,y) > T then g(x,y) = 1, else g(x,y) = 0
  - (or reverse)
- Result: a binary image where object pixels = 1 and noise = 0



(a) Training image



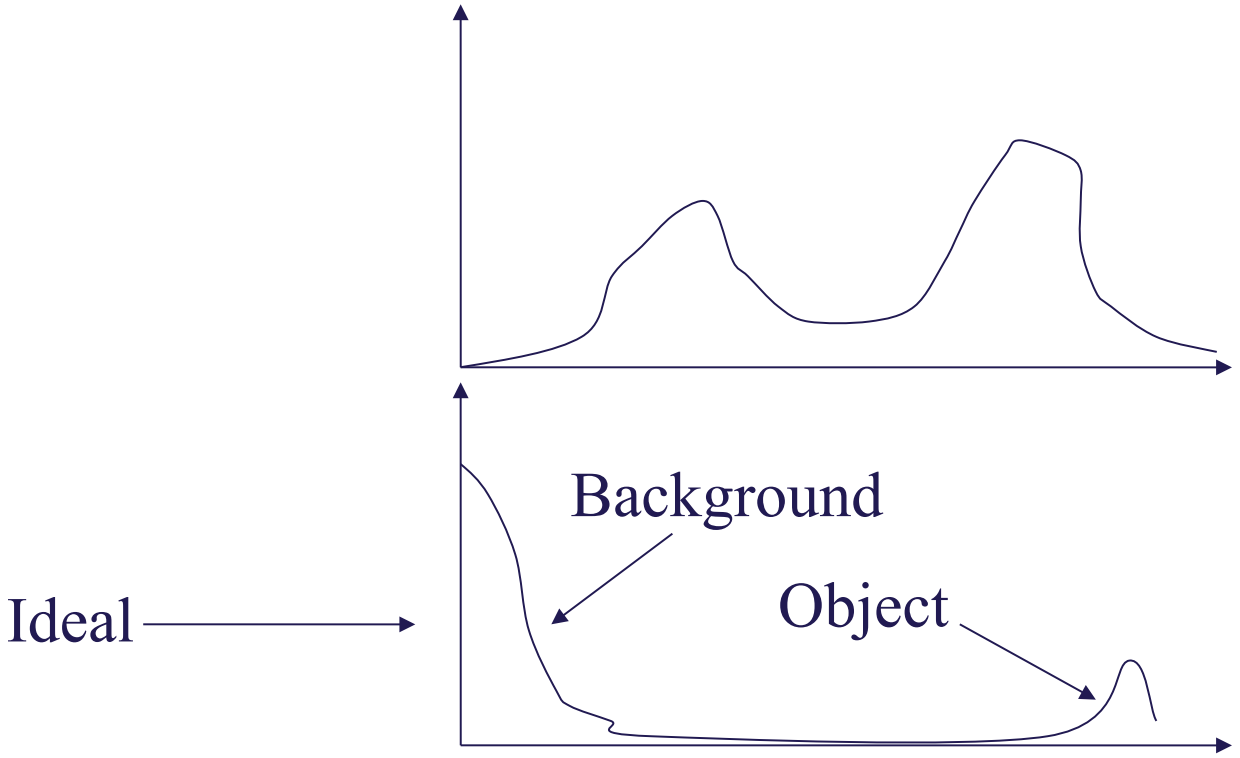
(b) HSV thresholding



(c) RGB thresholding

Segmentation

- Often, obtaining a bi-modal histogram is the ”sole” purpose of the image acquisition:
  - Lighting
  - Setup
  - Camera
  - Lens

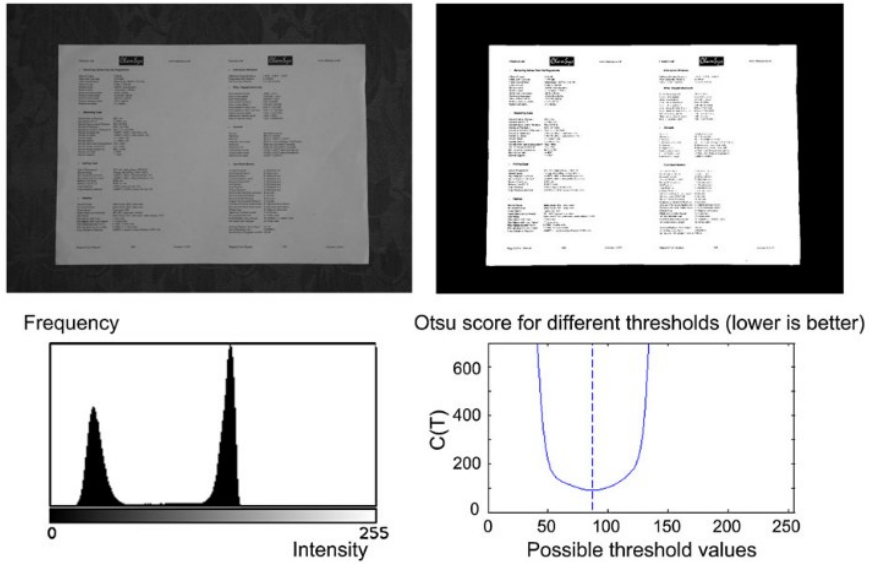


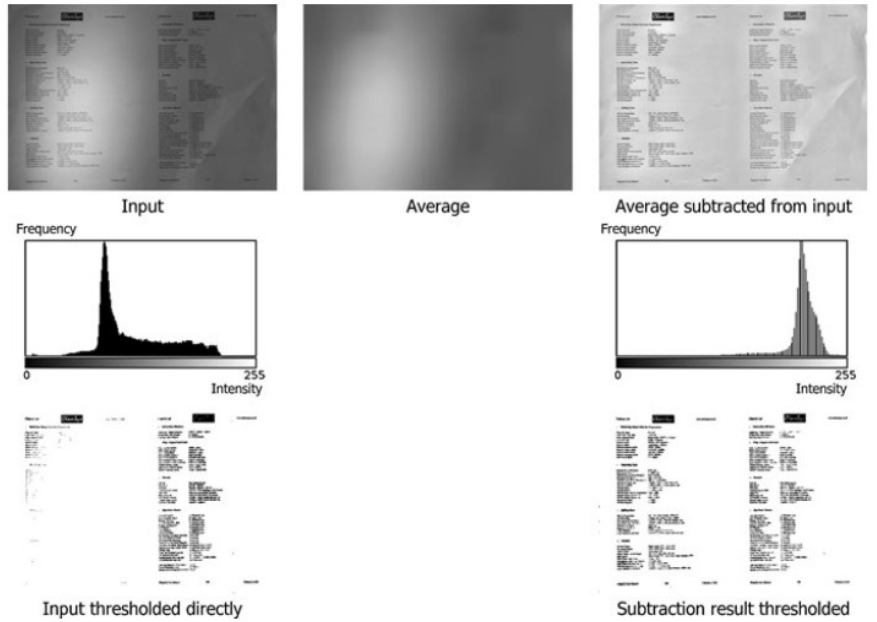
How to find the threshold?

- If we have a good setup, then the threshold is static.
- But the histogram is NEVER static!

- **Find the threshold during training**

- Check out the Otsu-method
- Consider updating the threshold continuously
- Consider pre-processing

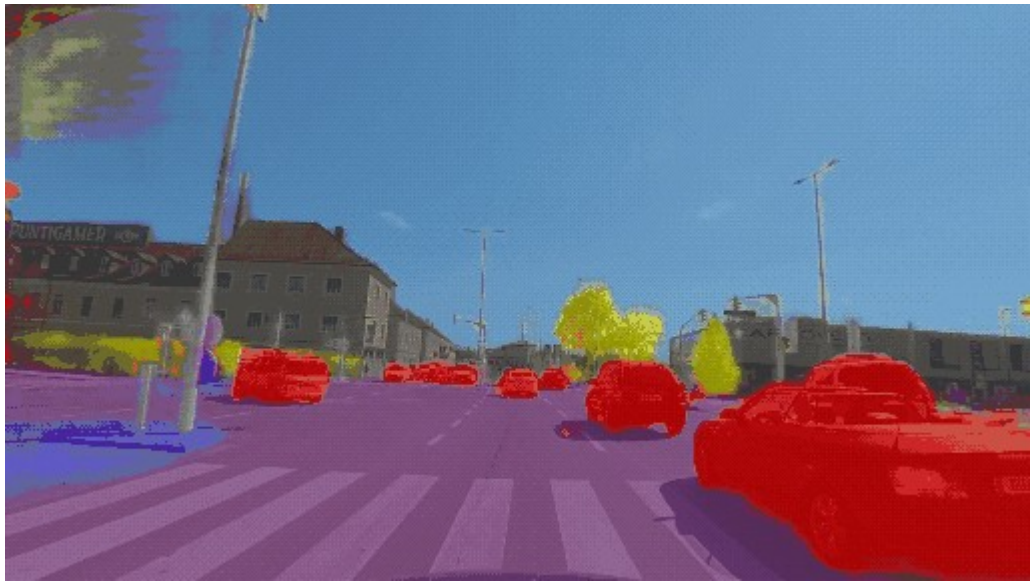




What if this does not work?

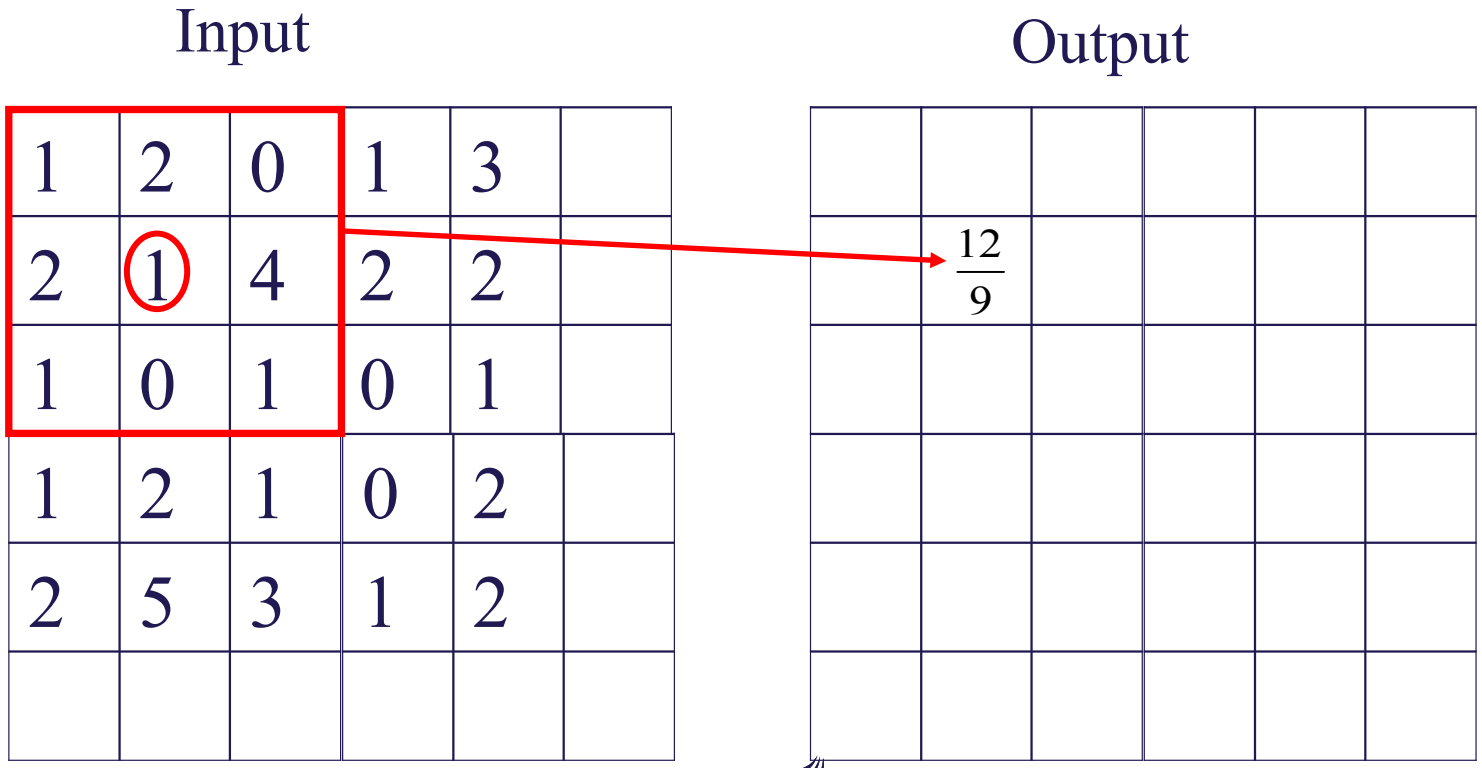
- Thresholding is surprisingly powerful, especially if you control the lighting.
- If you plan to develop something for outside – in the actual real world – it will probably fail horribly.

- This is still an open research area! (don't worry, you can always do a PhD)
- But we will of course cover some more advanced methods later today.

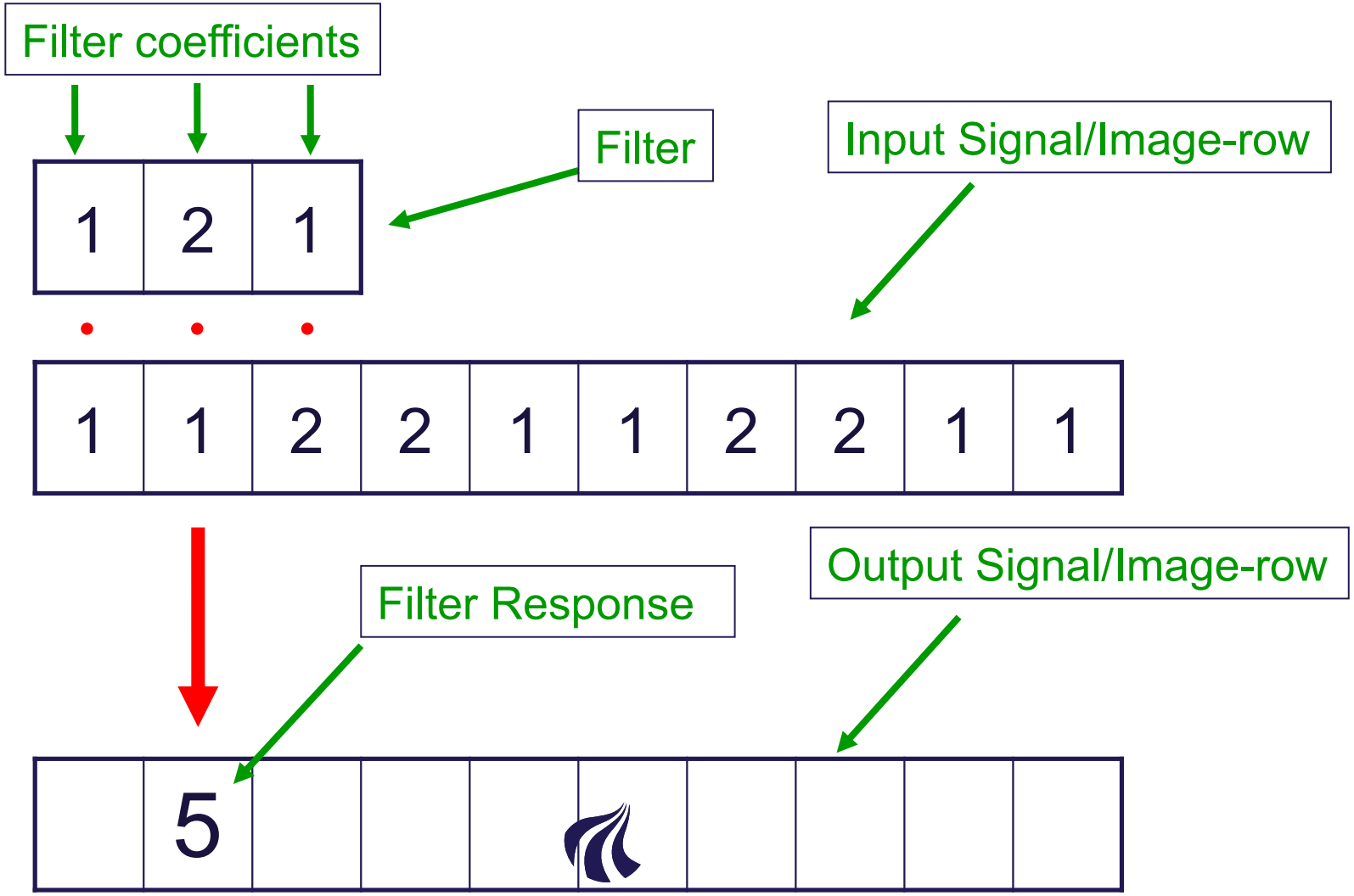


Neighbourhood processing

- Several pixels in the input has an effect on the output:



Correlation (1D)



Normalize filter response

- To avoid overflow, we must normalize.
  - i.e. Make sure that the max value stays within our memory bounds.

Filter coefficients:

A	B	C

Max pixel values:

…	255	255	255	…

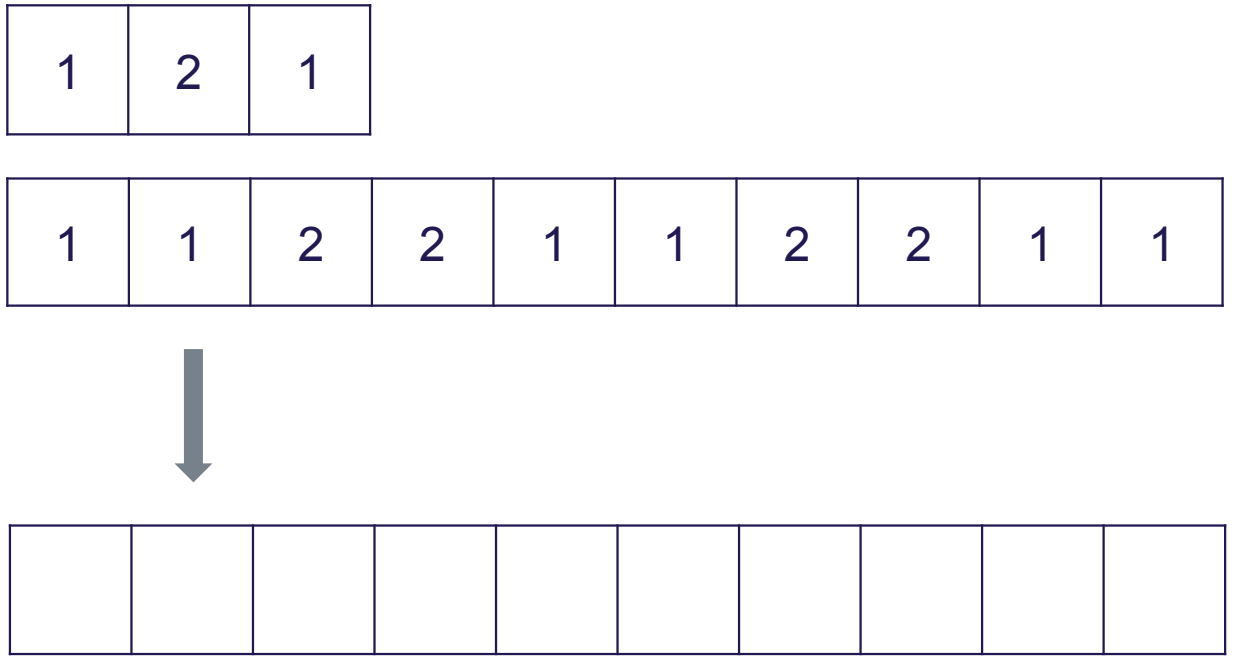
- Max filter response =

$$A \cdot 255 + B \cdot 255 + C \cdot 255 \Leftrightarrow 255 \cdot (A + B + C)$$

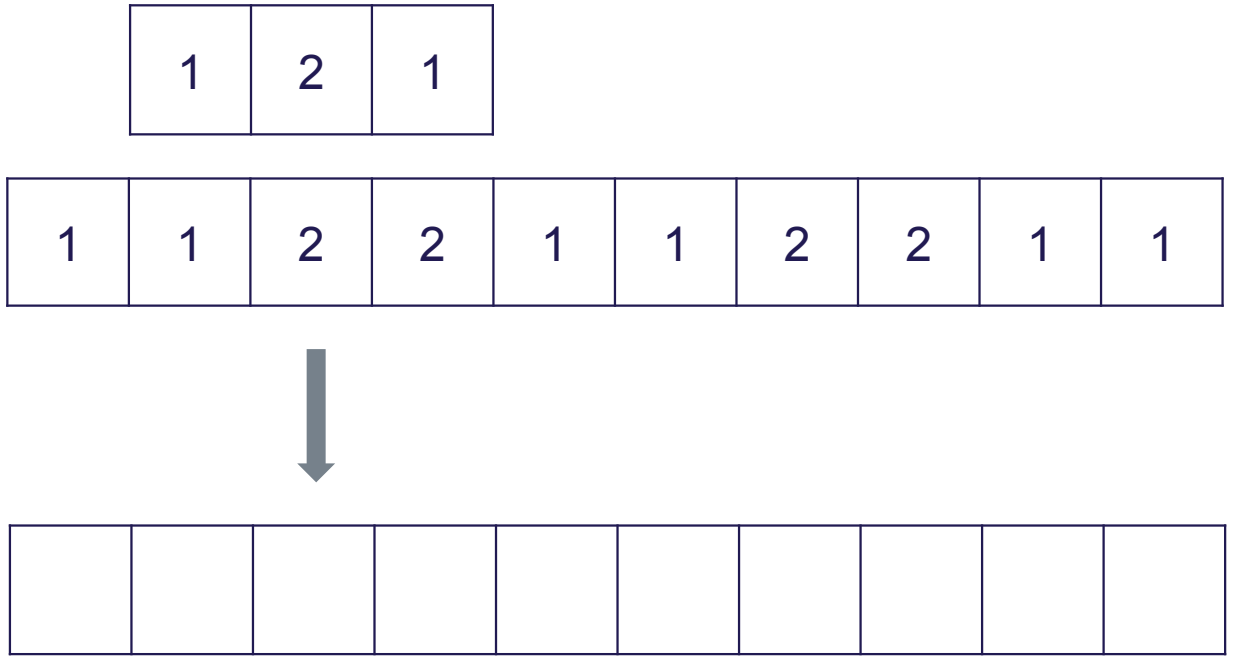
- Normalized filter response =

  Filter response / (A + B + C)

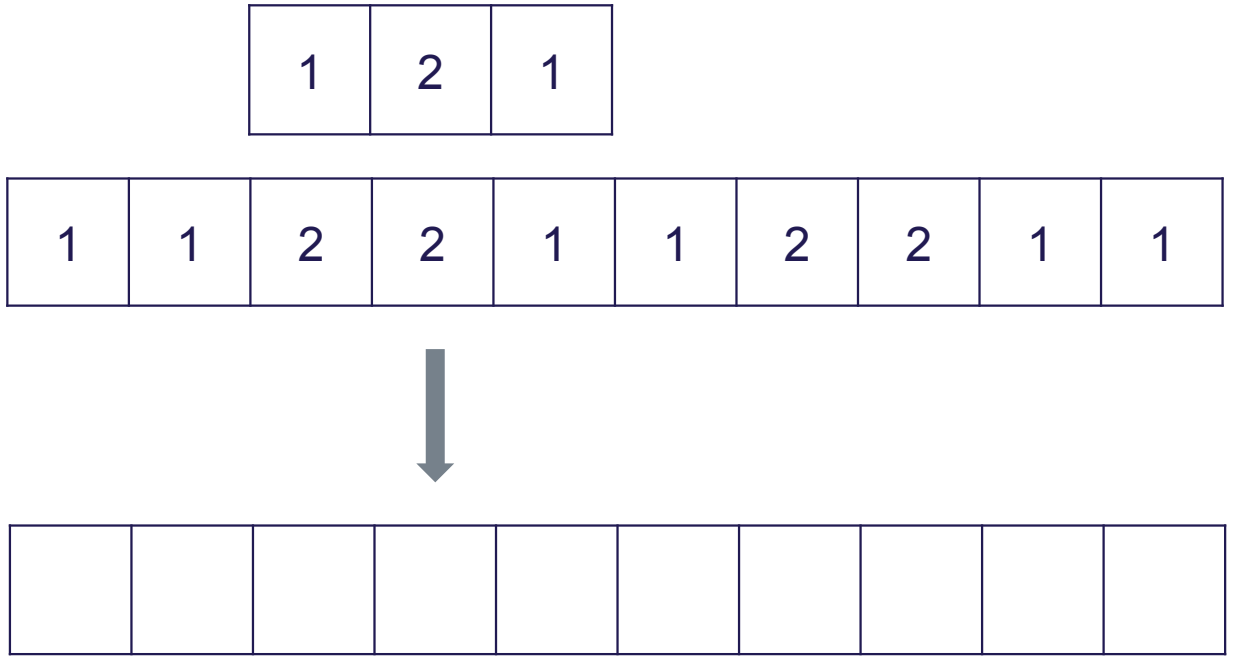
Correlation (1D)



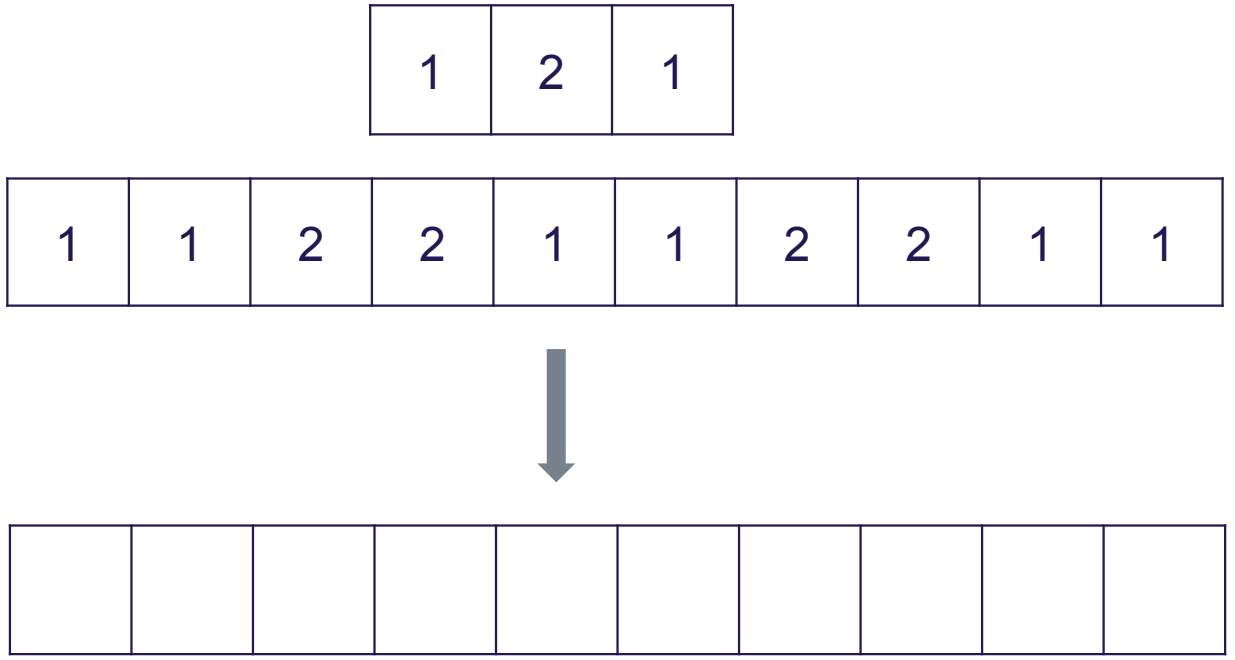
Correlation (1D)



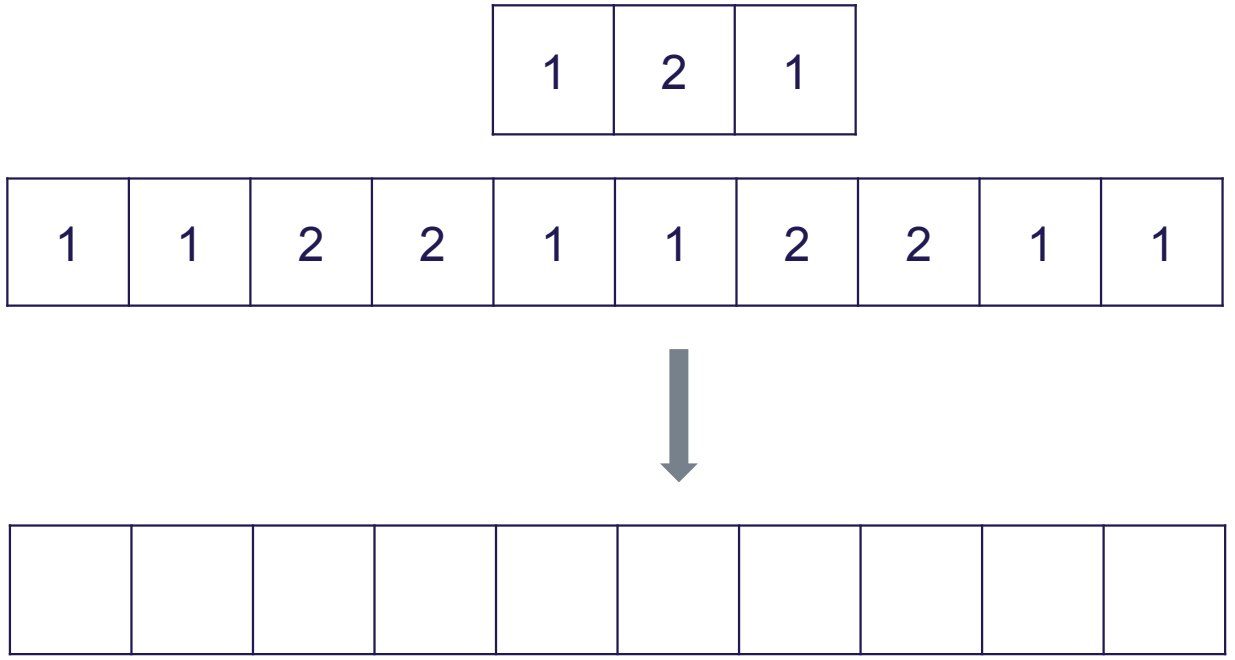
Correlation (1D)



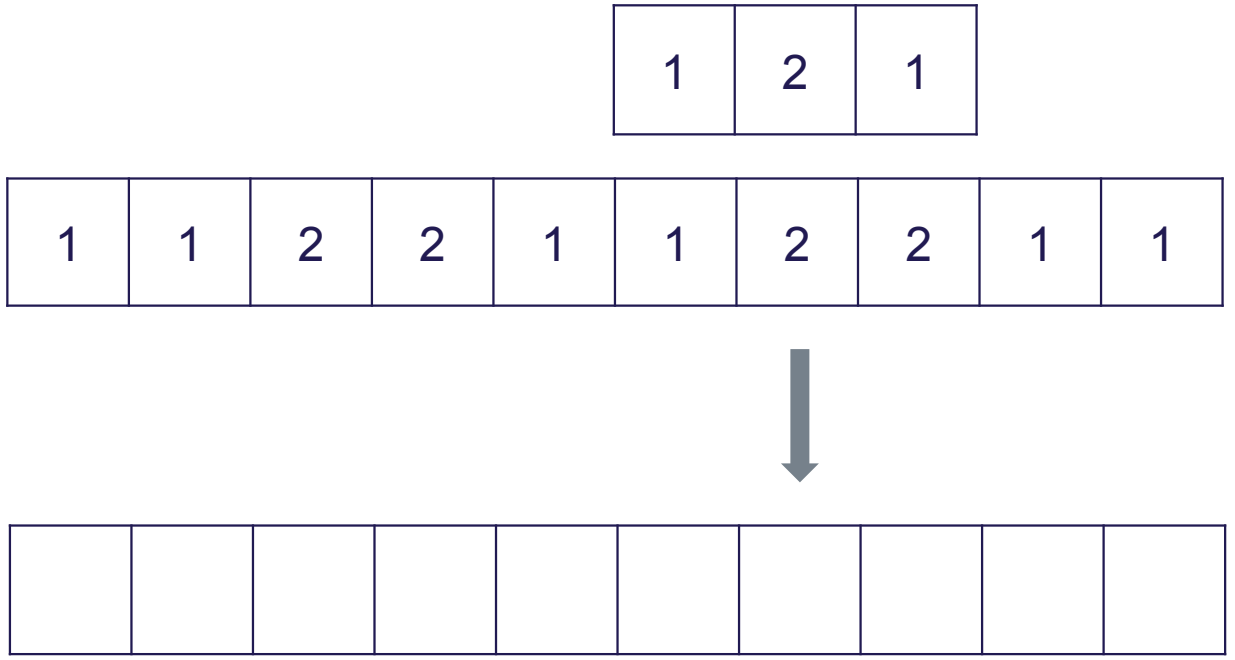
Correlation (1D)



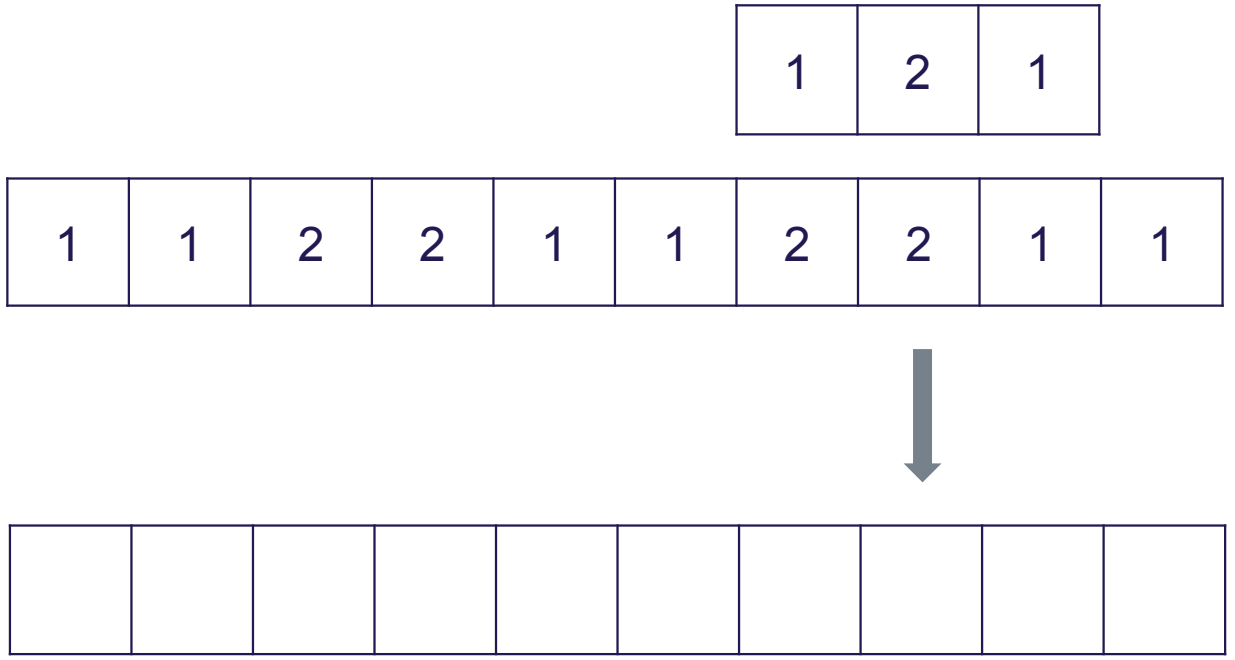
Correlation (1D)



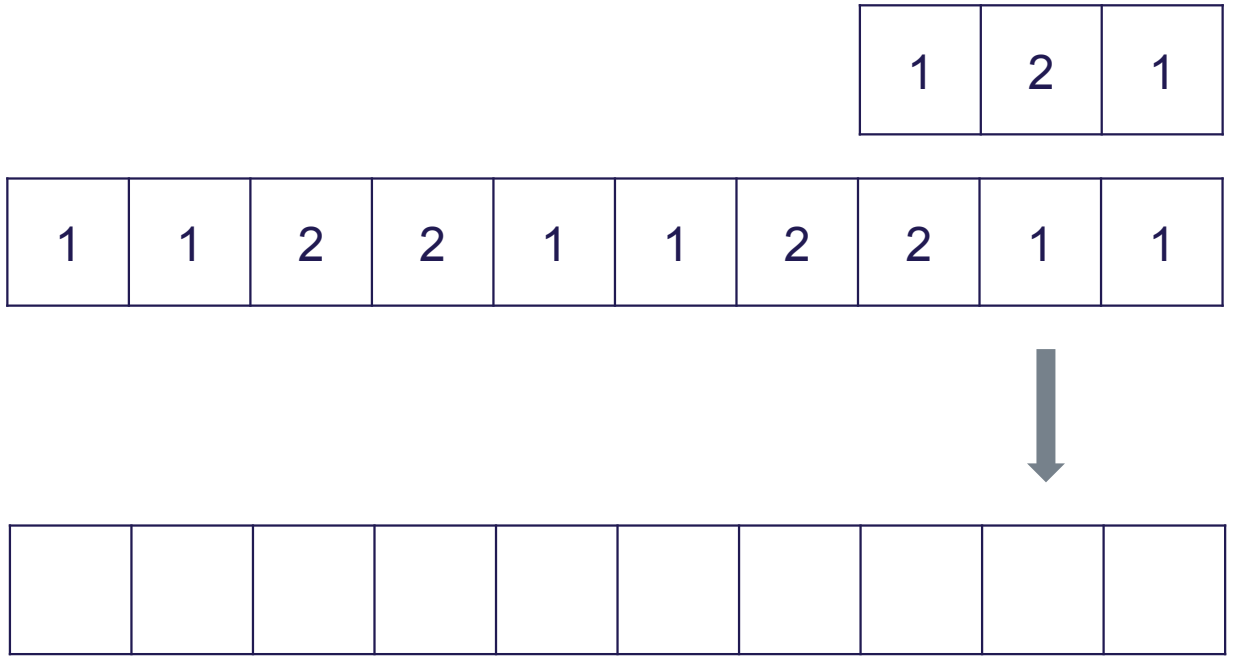
Correlation (1D)



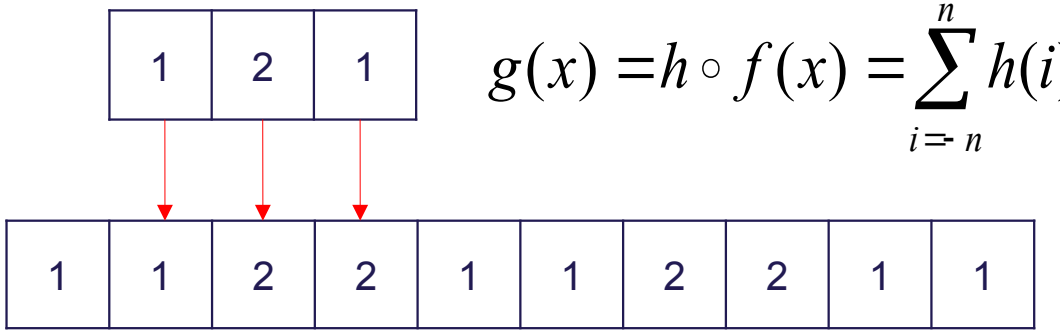
Correlation (1D)



Correlation (1D)



Correlation vs. convolution

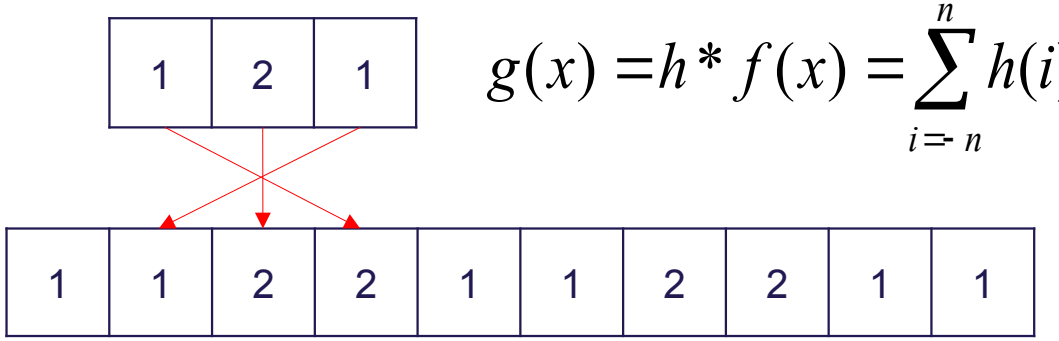


$$g(x) = h \circ f(x) = \sum_{i=-n}^{n} h(i) f(x+i)$$

Correlation

Convolution

$$g(x) = h * f(x) = \sum_{i=-n}^{n} h(i) f(x-i)$$

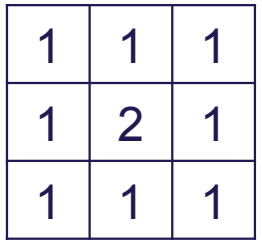


- In image processing we use correlation, but (nearly) always call it convolution! We're silly like that.
- Note: When the filter is symmetric: correlation = convolution!

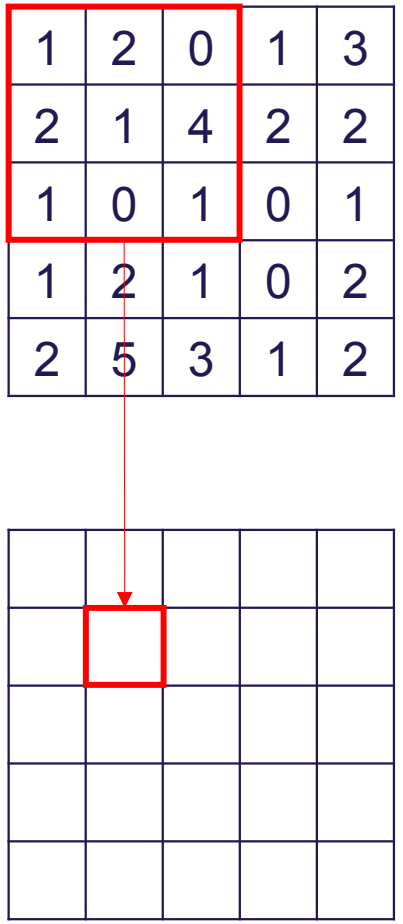
Convolution on images

- The filter is now in 2D
- This is called a kernel, and the coefficients are kernel coefficients.
- They are often square, and always odd-sized: 3x3, 5x5, 7x7, etc.

Normalization



1	1	1
1	2	1
1	1	1



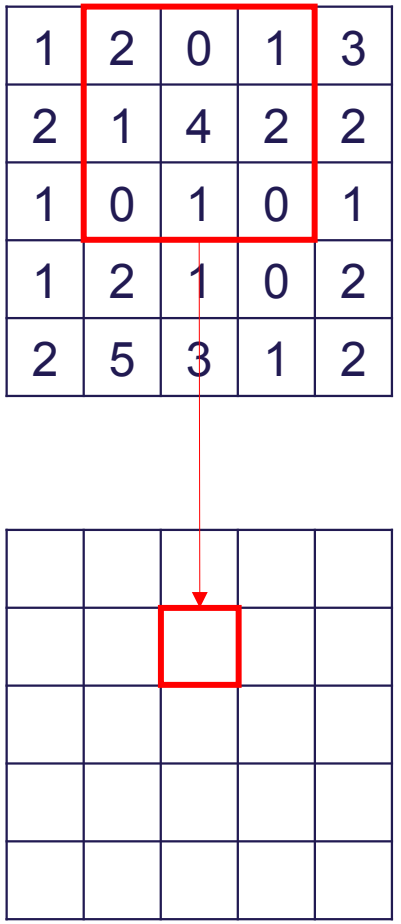
1	2	0	1	3
2	1	4	2	2
1	0	1	0	1
1	2	1	0	2
2	5	3	1	2

Convolution on images

- The filter is now in 2D
- This is called a kernel, and the coefficients are kernel coefficients.
- They are often square, and always odd-sized: 3x3, 5x5, 7x7, etc.

Normalization

1	1	1
1	2	1
1	1	1



Convolution on images

- The filter is now in 2D
- This is called a kernel, and the coefficients are kernel coefficients.
- They are often square, and always odd-sized: 3x3, 5x5, 7x7, etc.

Normalization

1	1	1
1	2	1
1	1	1

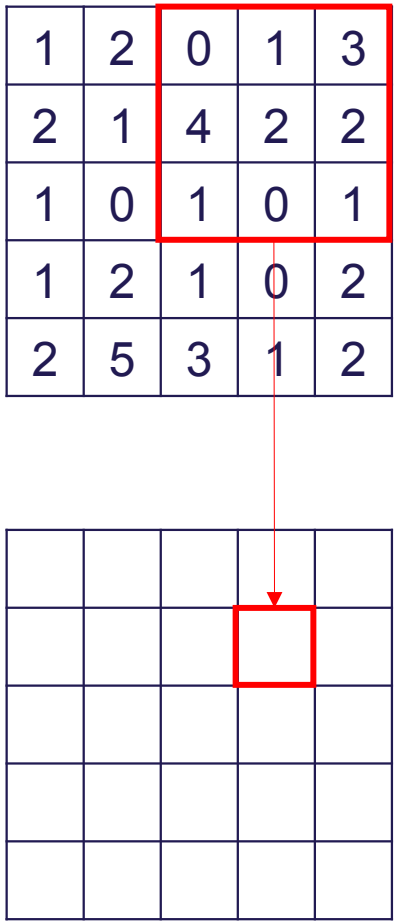


Image blur





Image blur

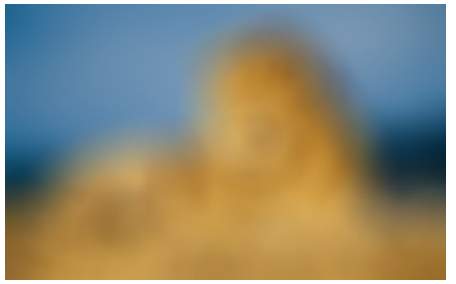


Image size: 384x240, kernel radius: 30

- Also known as: Smoothing kernel, low pass filter

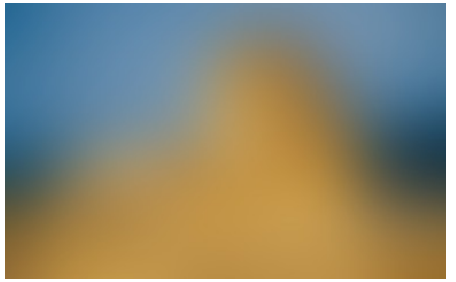
- The simplest filter: Mean filter (spatial low pass filter)

1	1	1
1	1	1
1	1	1



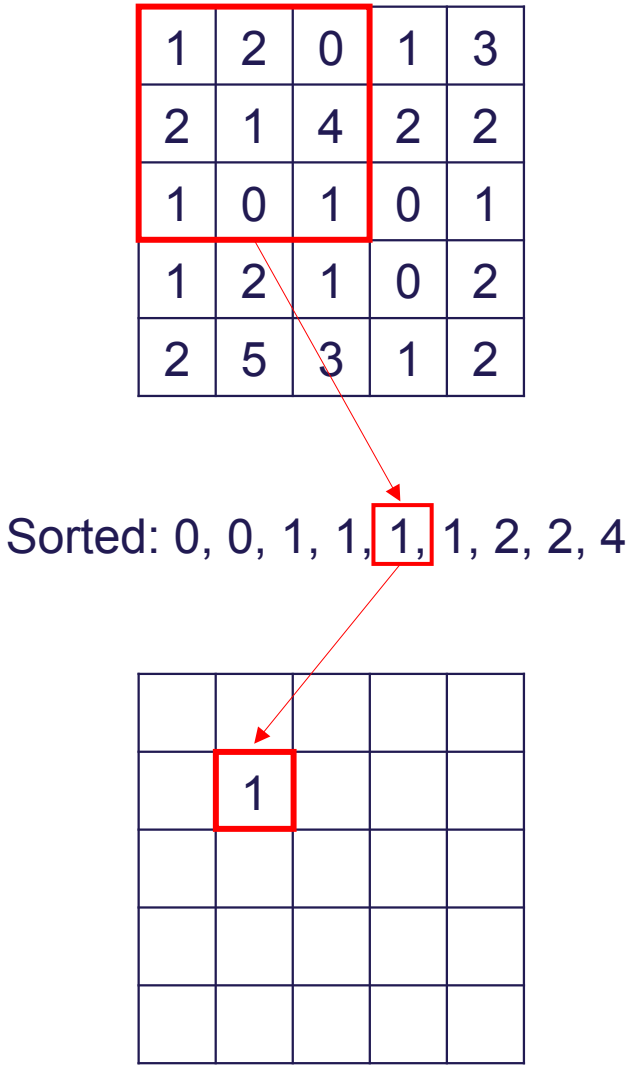
- Another option: Gaussian filter

1	2	1
2	4	2
1	2	1



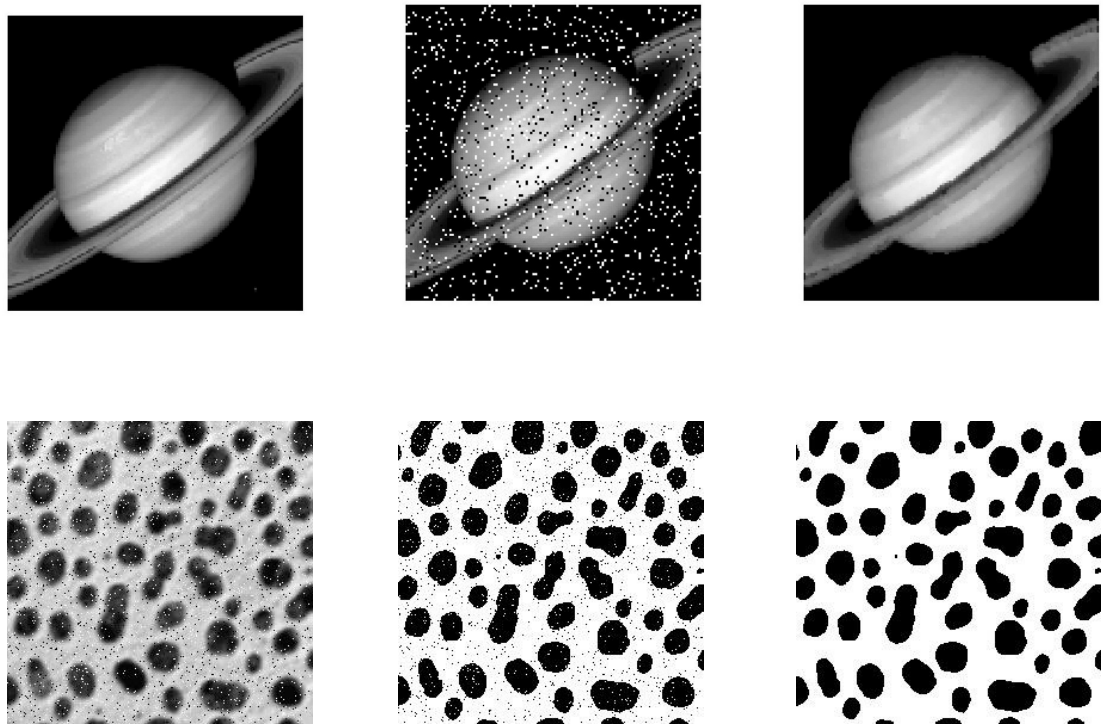
Rank filters

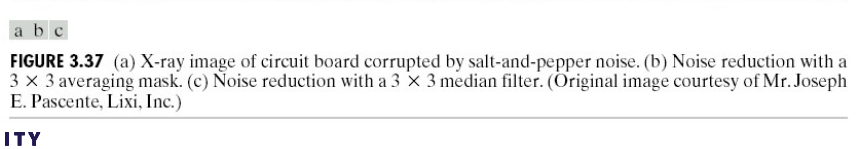
- Aka: order-statistics filters
- Not based on correlation but still neighborhood processing
- Principle:
  - Define a mask/kernel, e.g., 3x3
  - Sort all pixel-values within the mask into ascending order
  - Select a pixel-value according to the filter type: Median, min., max., range, …
- Example to the right: Median filter



Applications of median filter

- Good for cleaning salt-and-pepper noise
- Good at removing noise in binary images
- Better than the mean filter as blurring is minimized and edges stay sharp





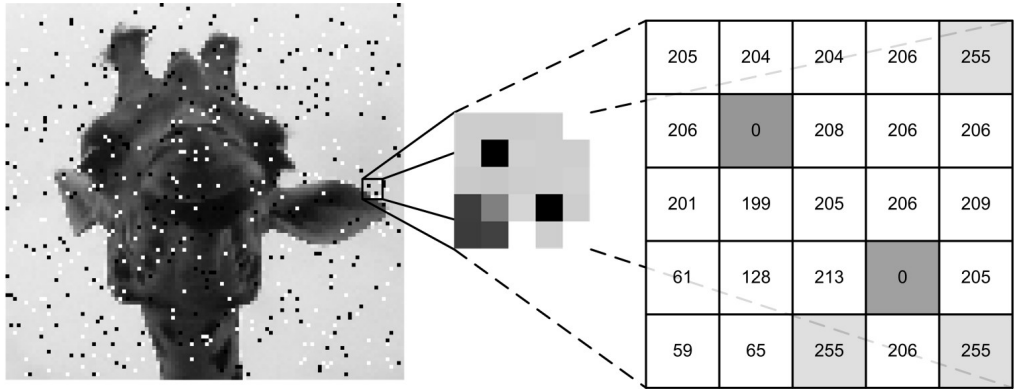
a b c

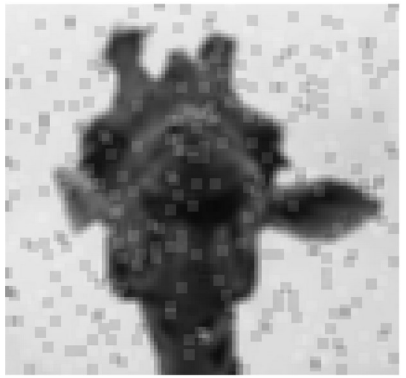
FIGURE 3.37 (a) X-ray image of circuit board corrupted by salt-and-pepper noise. (b) Noise reduction with a 3×3 averaging mask. (c) Noise reduction with a 3×3 median filter. (Original image courtesy of Mr. Joseph E. Pascente, Lixi, Inc.)

Difference between mean and median filter

- Apply a Median and a Mean filter (use a 3x3 kernel) on the image below. Think about the differences.

1	2	0
2	2	4
1	202	1

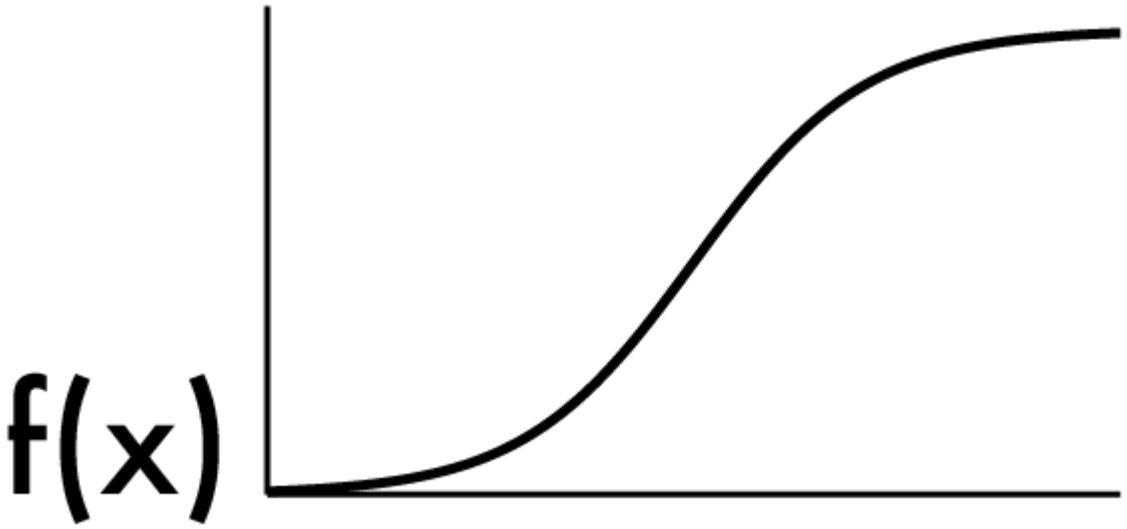



Mean filtered


Median filtered

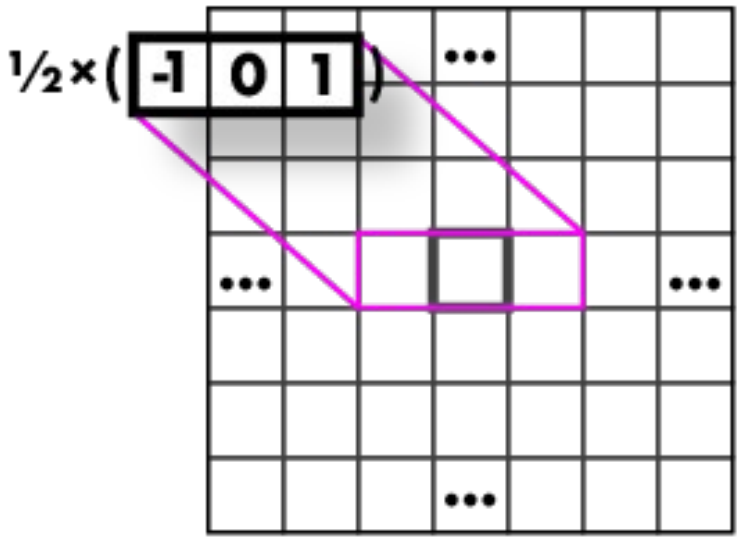
Edge detection

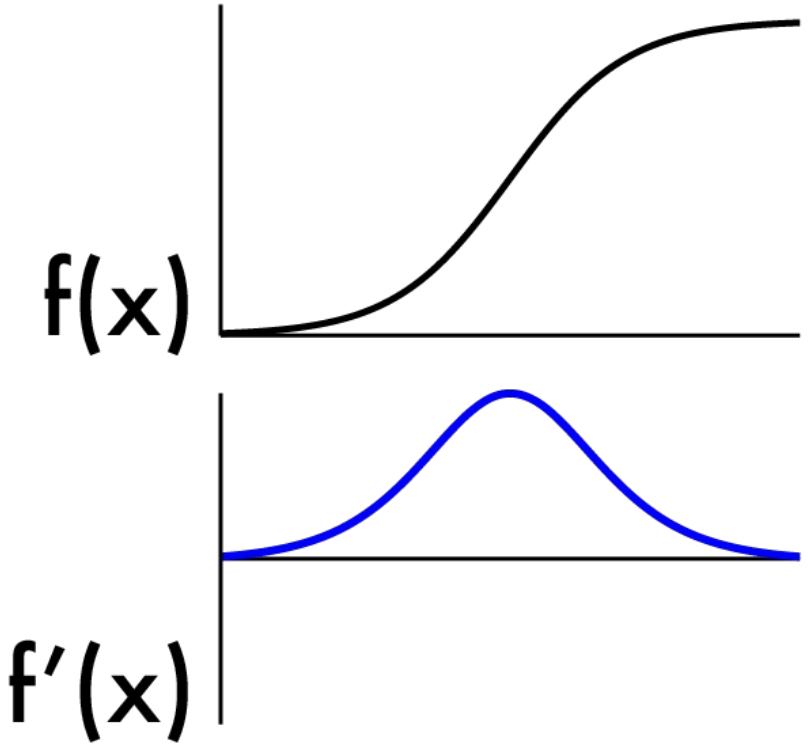
- Image is a function
- Edges are rapid changes in this function



Edge detection

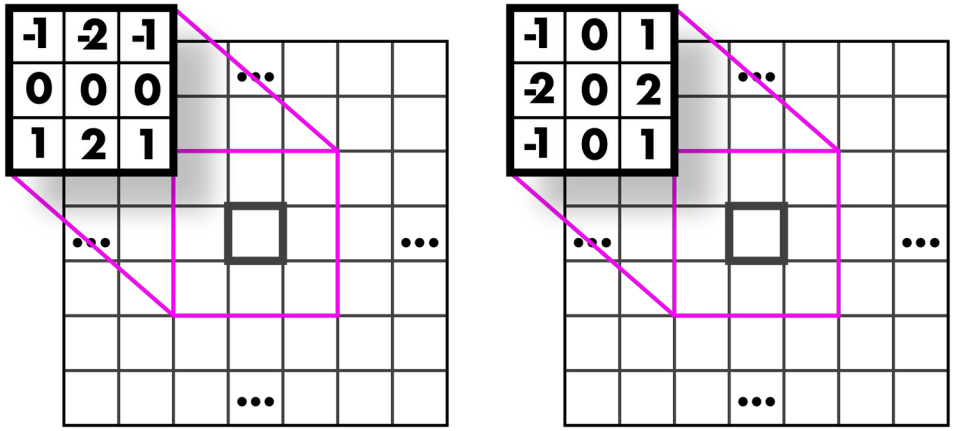
- Could take derivative
- Edges = high response

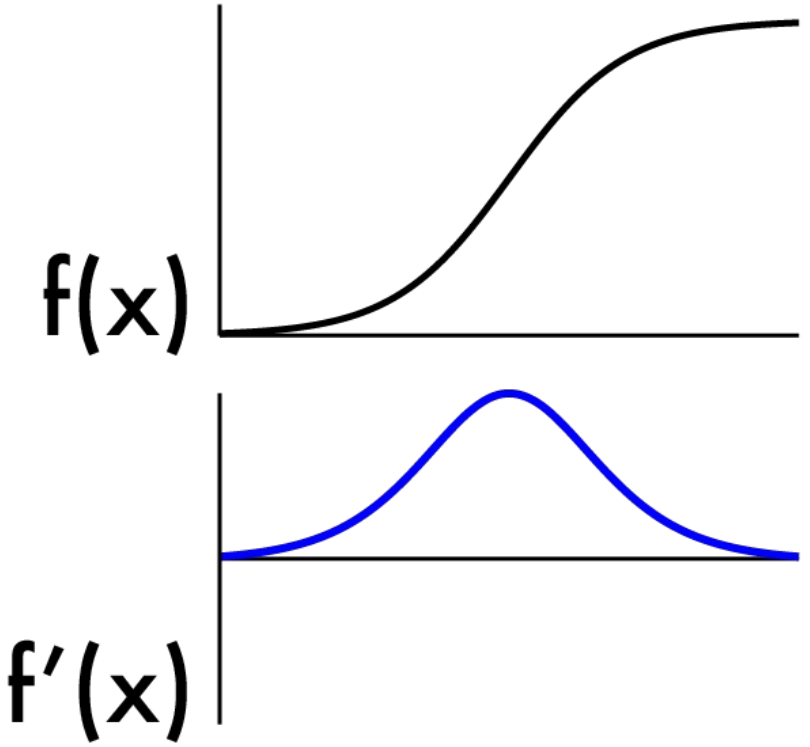




Edge detection

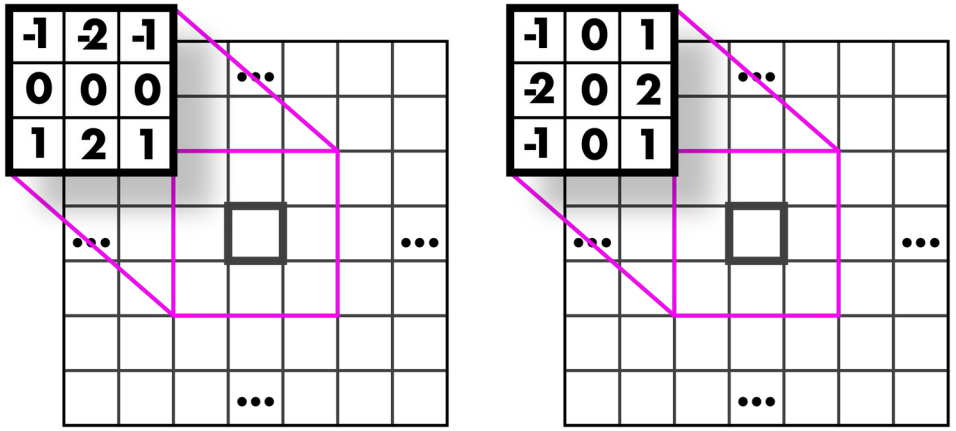
- Sobel filters
  - Gradient + smoothing
- What is an edge?
  - High response

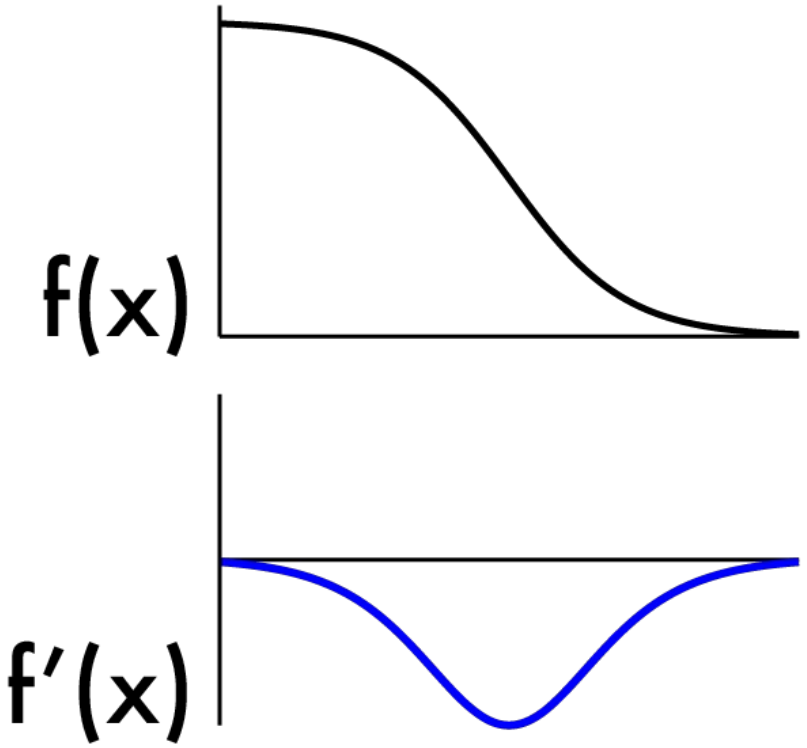




Edge detection

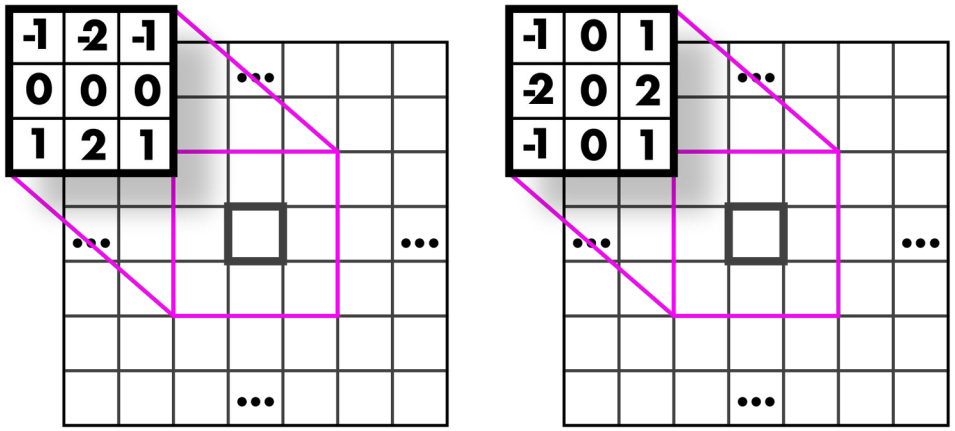
- Sobel filters
- What is an edge?
  - ~~High response~~
  - Extrema





Edge detection

- Back to gradients: Use magnitude of gradients
- Sobel kernels

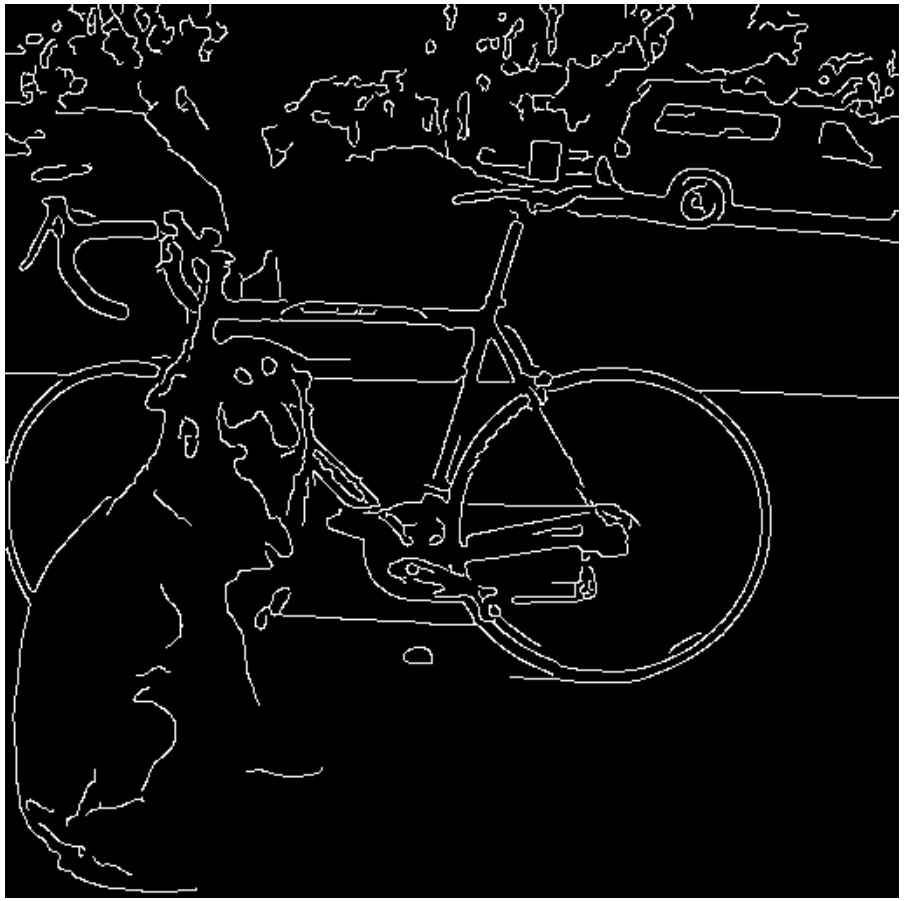




Edge detection

- What we want: Line drawing!





Canny edge detector

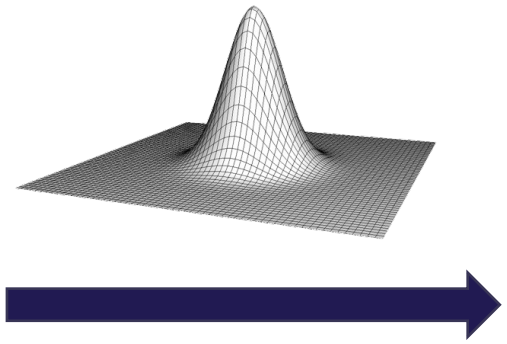
Algorithm:

- Smooth image (Gaussian filter)
- Calculate gradient direction and magnitude
- Non-maximum suppression perpendicular to edge
- Threshold into strong, weak, no edge
- Connect components

Canny edge detector

- Smoothing – Gaussian filter

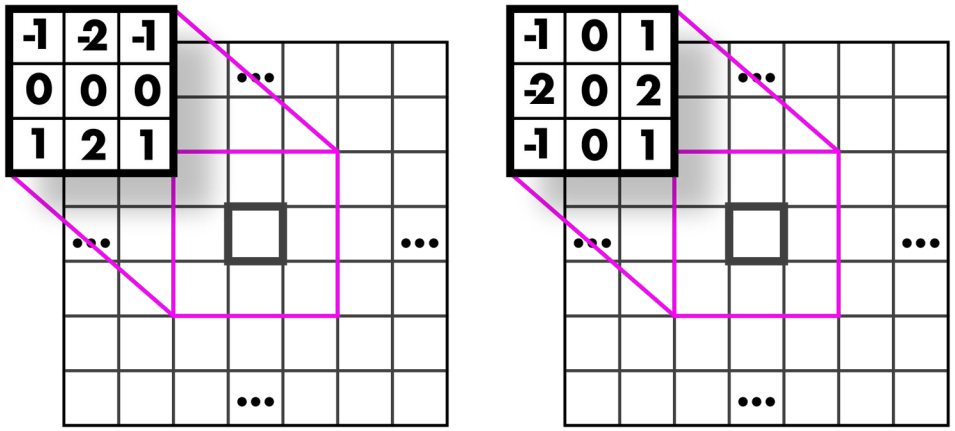






Canny edge detector

- Gradient magnitude and direction
  - Sobel filter



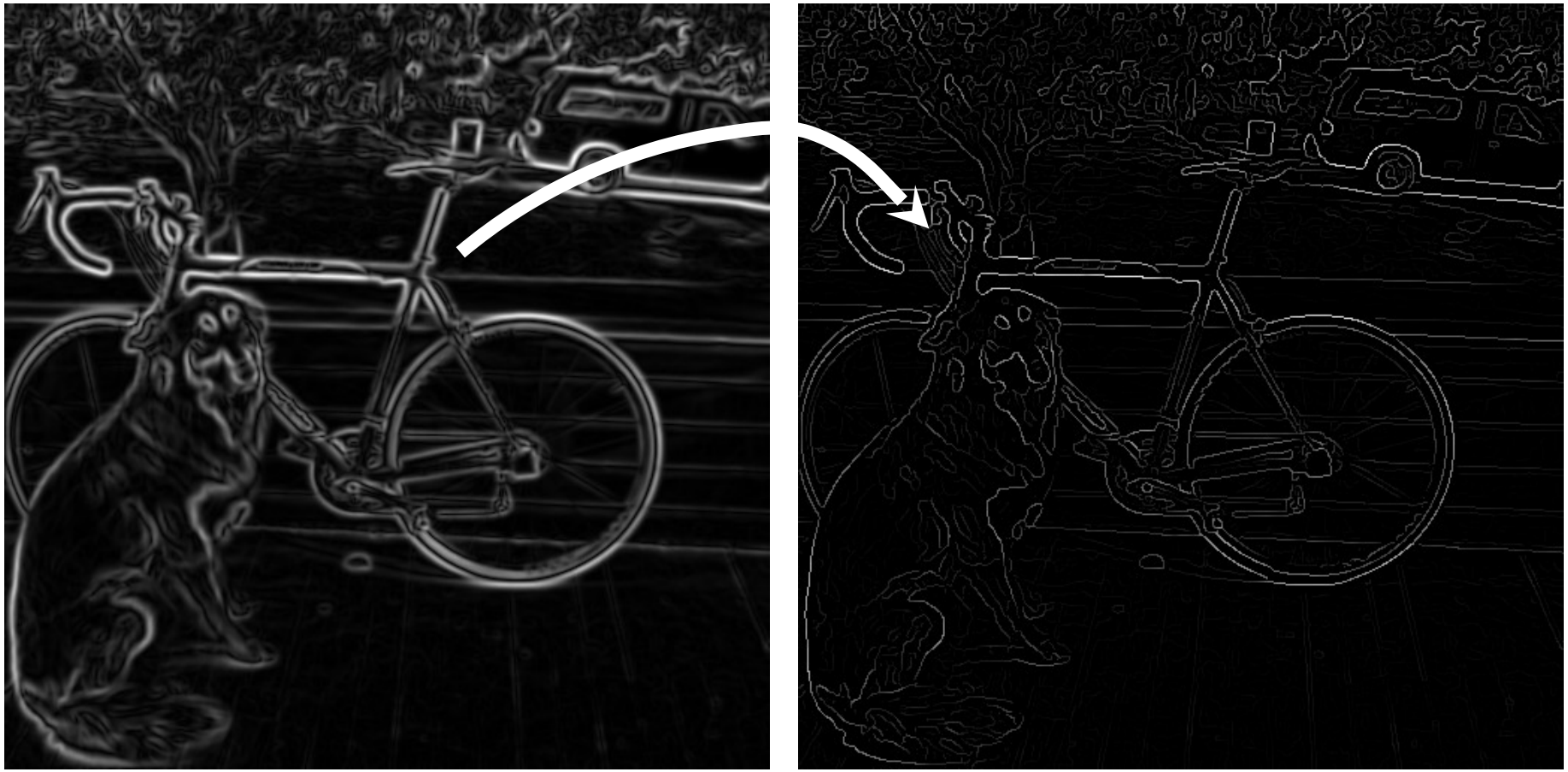


Canny edge detector

- Non-maximum suppression
  - Goal: Single pixel edges, not thick blurry lines
  - Need to check nearby pixels and keep only highest respons



Canny edge detector



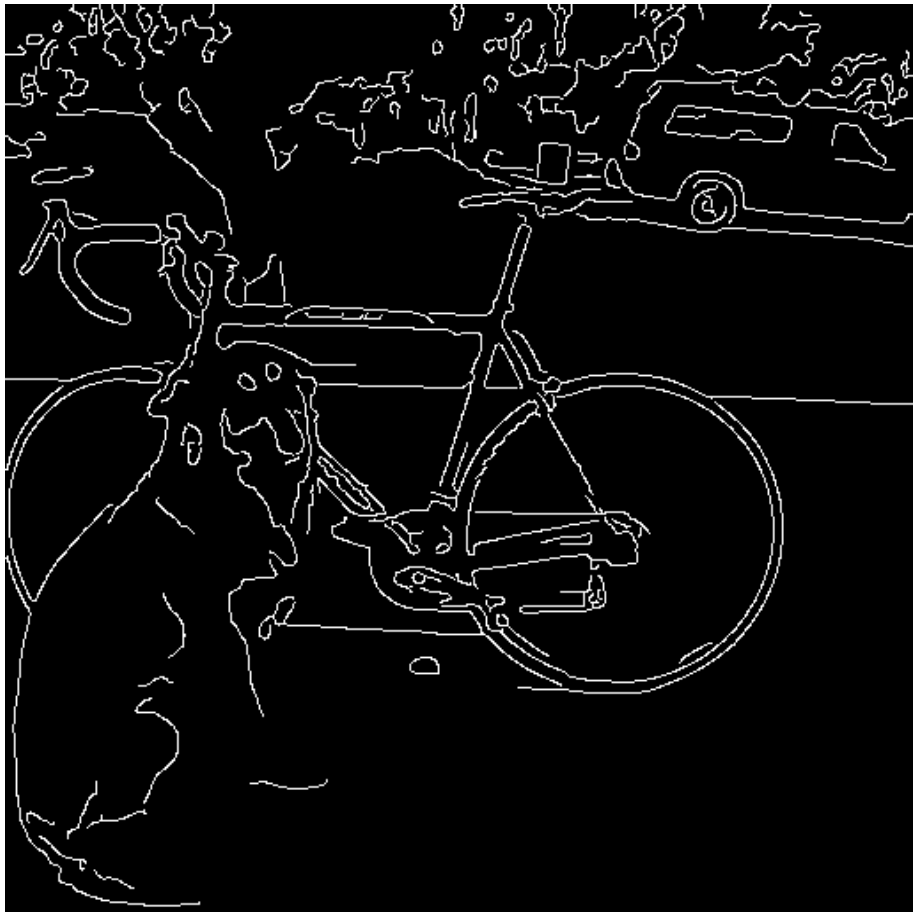
Canny edge detector

- Still some noise
- Only want strong edges
- 2 thresholds – 3 cases
  - PixVal > T_{high}: strong edge
  - T_{high} > PixVal > T_{low}: weak edge
  - PixVal < T_{low}: no edge



Canny edge detector

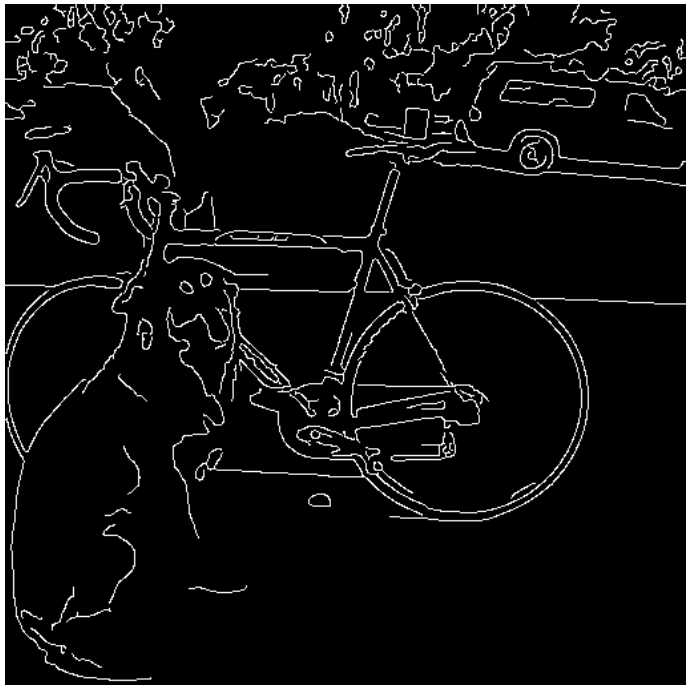
- Strong edge are edges!
- Weak edges are edges if they connect to strong edges
  - Look in some neighborhood (usually 8 closest)
  - Hysteresis



Canny edge detector

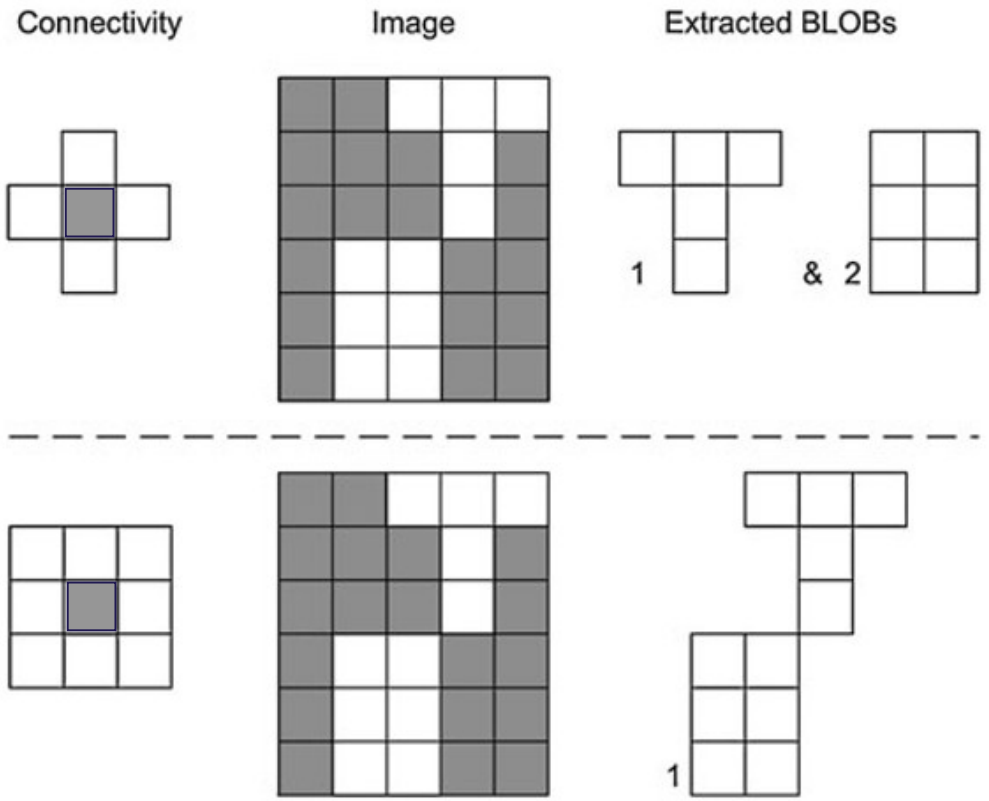
- Tunable parameters: Sigma (Gaussian), thresholds T_{high} and T_{low}
- Demo: http://bigwww.epfl.ch/demo/ip/demos/edgeDetector/





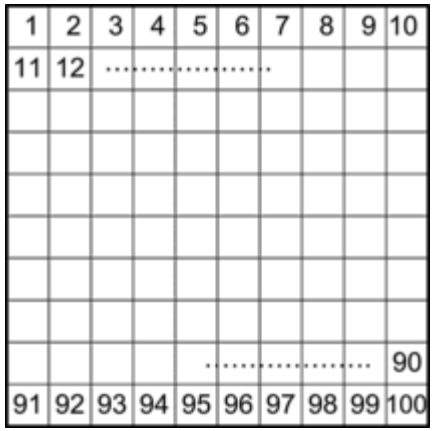
Isolating blobs

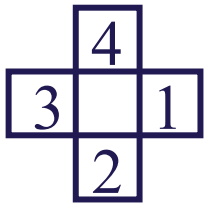
- What we want:
  - For each object in the image, a list with its pixels
- How do we get that?
  - Connected component analysis
  - Region growing
- Define: **Connectivity**
  - Who are my neighbors?
  - 4-connected
  - 8-connected

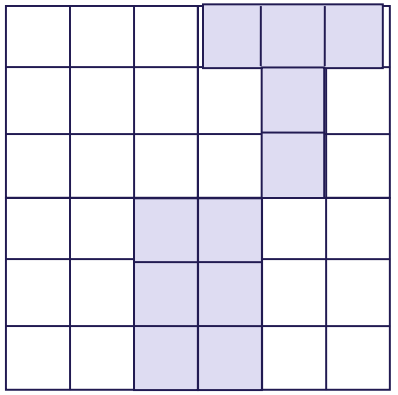


Connected component analysis

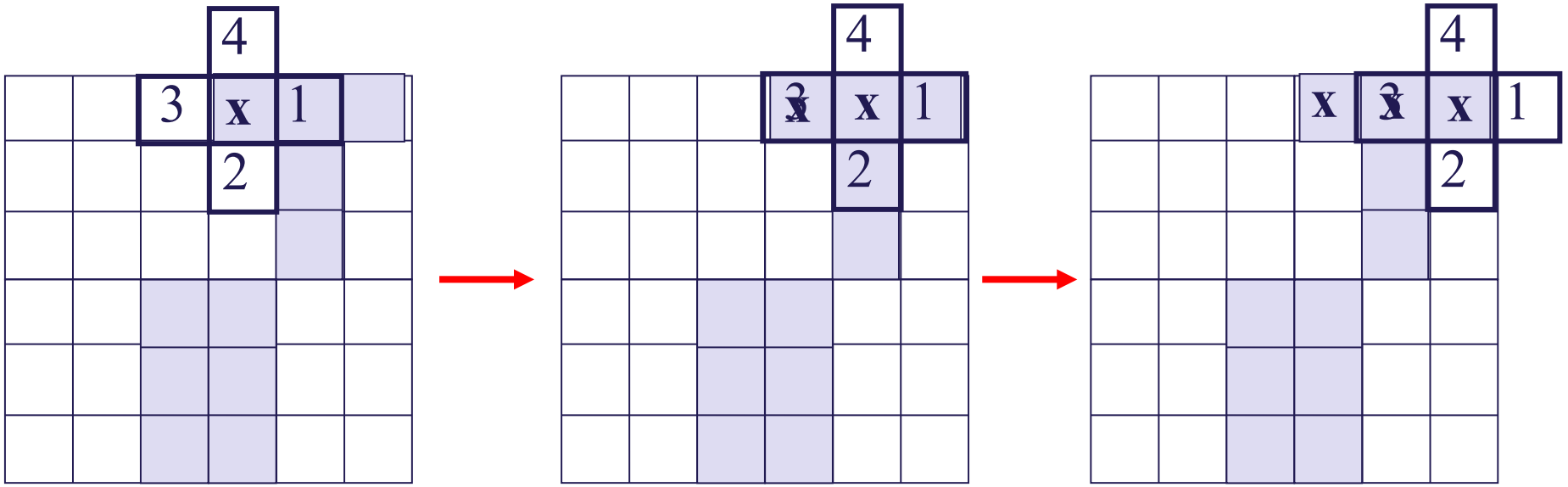
- We are working with binary images.
- The task is to give all groups of connected pixels a unique ID.
- We use the grassfire-algorithm:
  - Start in the corner
  - “Burn” (and name) all visited object pixels
  - When an object is encountered, burn all associated pixels before continuing
- Can be implemented either in a recursive or sequential manner, but the concept is the same (this is essentially a depth-first vs. breadth-first graph search question).



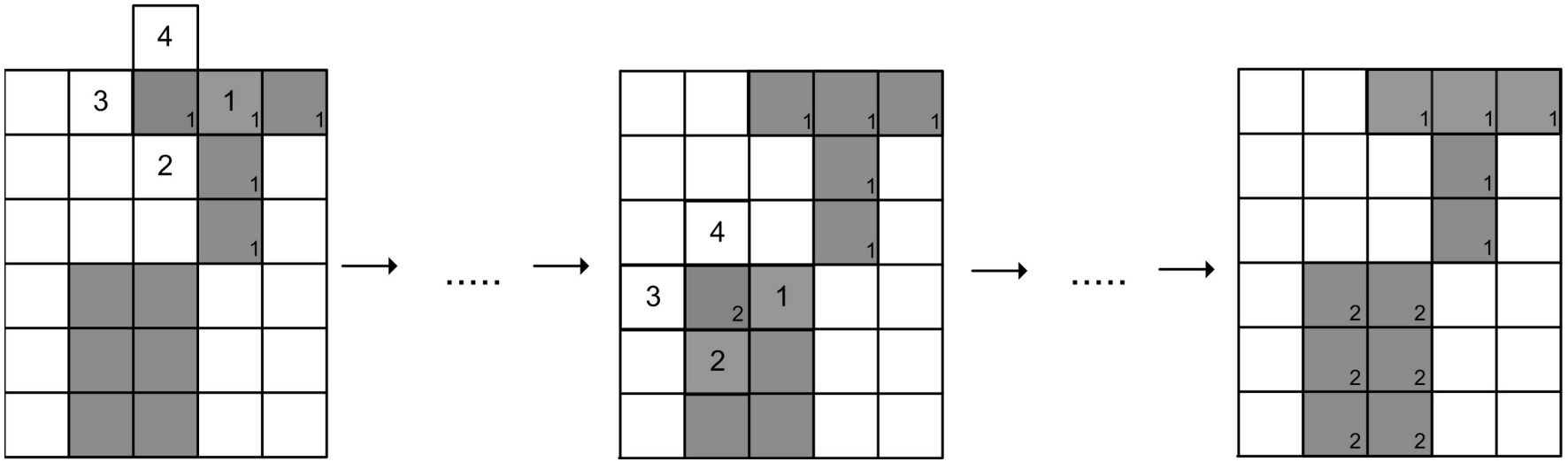




Connected component analysis



Connected component analysis



Exercise 1 (10 min): Count the coins

- Segment and count objects with threshold and connected components (see handout).
- Get the count as close as possible to the true number of coins (24). Look at the Otsu result first: does anything look wrong?
- Filter out tiny components by area. Why does this help?
- Stretch: use cv2.Canny instead and compare. When is edge-based better than threshold-based?

https://github.com/andreasmoegelmose/cv_primer

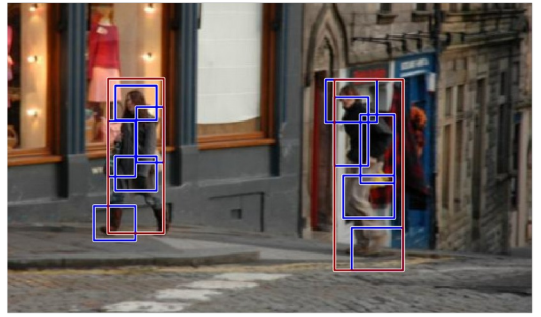


Section 2: Features

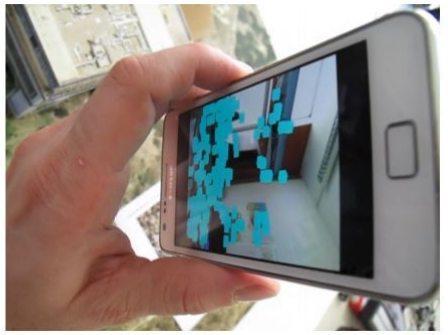
Features

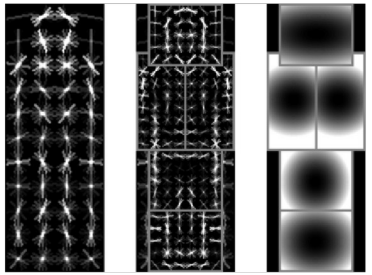
- Feature descriptors: Describing local regions
  - Also just called features

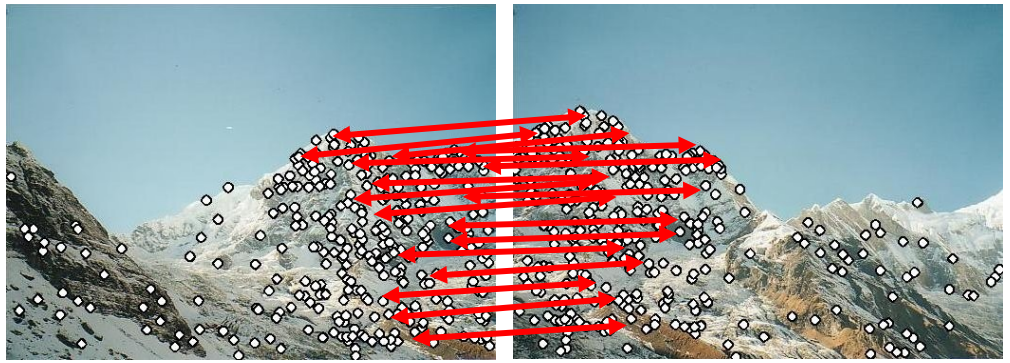
- Useful for:
  - Matching
  - Recognition
  - Detection
  - …

















Features

Objectives:

- Want to find patches in image that are useful or have some meaning
- For object detection, want a patch that is common to that object but not in general
- For panorama stitching, want patches that we can find easily in another image of same place
- Good features are unique!
  - Can find the “same” feature easily
  - Not mistaken for “different” features

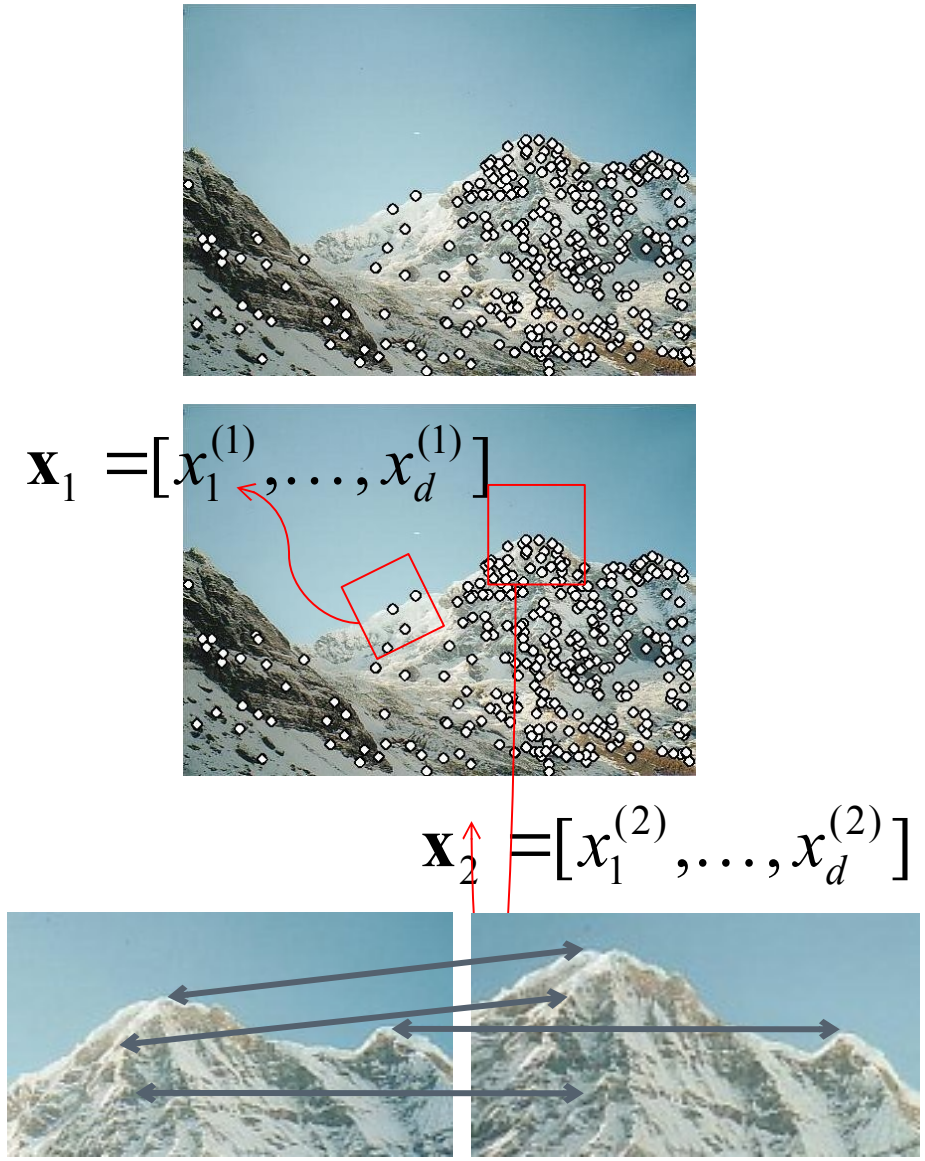


Features – main components

1) Detection: Identify the interest point.

2) **Description**: Extract vector feature descriptor surrounding each interest point.

3) Matching: Determine correspondence between descriptors in two views or between model and image



Features

Application example:

- Stitching a panorama: Looking for patches in image to match other image





Features

- Unique patches?
- Sky: bad
  - Very little variation
  - Could match any other sky



Features

- Unique patches?
- Sky: bad
  - Very little variation
  - Could match any other sky

- Edge: ok
  - Variation in one direction
  - Could match other patches along same edge



Features

- Unique patches?
- Sky: bad
  - Very little variation
  - Could match any other sky

- Edge: ok
  - Variation in one direction
  - Could match other patches along same edge

- Corners: good!
  - Only one alignment matches



Features

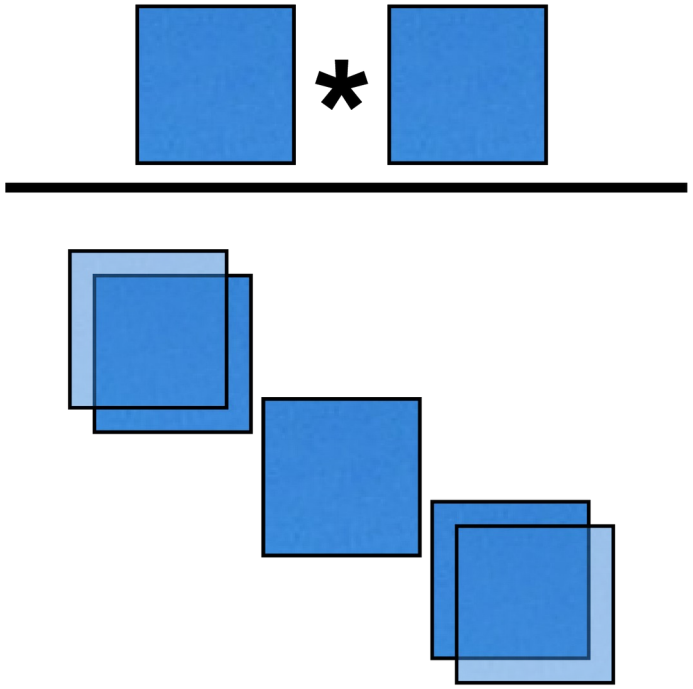
- How to find the unique patches?

  a) Distance (difference) between patch and every other patch
     - Computationally heavy!

  b) Auto-correlation: How well does the image patch match a shifted version of itself?
     - Measure of self-difference
     - $\sum_d \sum_{x,y} (I(x,y) - I(x+dx, y+dy))^2$

Features

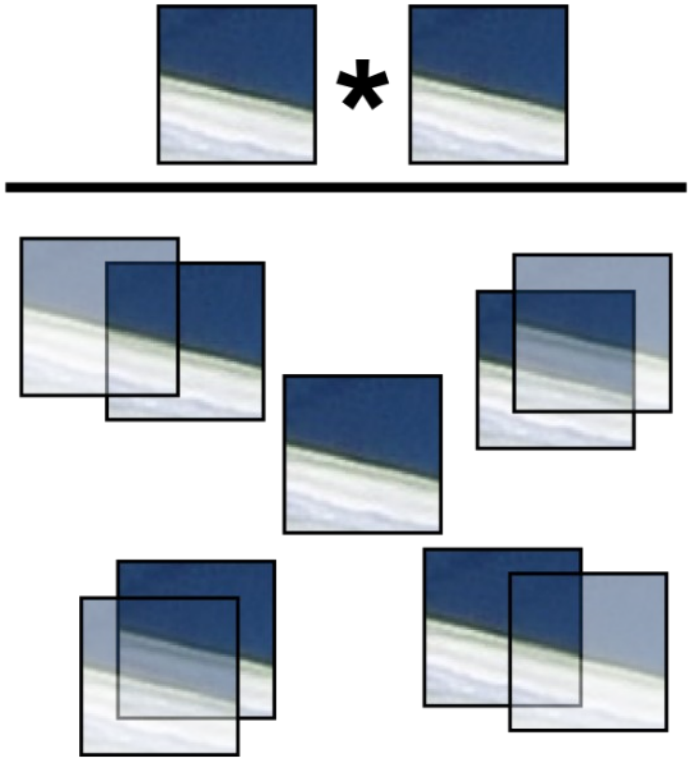
- Sky: Self-difference low everywhere





Features

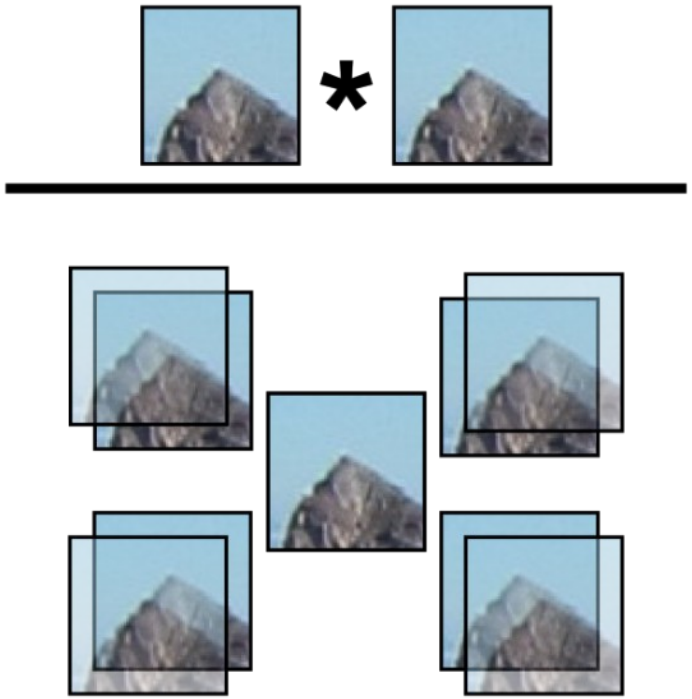
- Edge: Self-difference low along edge





Features

- Corner: Self-difference mostly high





Features

- How to find the unique patches?

  a) Distance between patch and every other patch
     - Computationally heavy!

  b) Auto-correlation: How well does the image match a shifted version of itself?
     - Measure of self-difference
     - $\sum_d \sum_{x,y} (I(x,y) - I(x+dx, y+dy))^2$
     - Still computationally expensive – need an approximation

  c) Gradients, x- and y-direction
     - Zero: plain image patch
     - One direction: Edge
     - Two directions: Corner, good patch!

Features

- Analysis of gradients: Eigen vectors/- values!
- Need structure matrix for patch, just a weighted sum of nearby gradient information

- $$S_w[p] = \begin{bmatrix} \sum_r w[r]\,(I_x[p-r])^2 & \sum_r w[r] I_x[p-r] I_y[p-r] \\ \sum_r w[r] I_x[p-r] I_y[p-r] & \sum_r w[r]\big(I_y[p-r]\big)^2 \end{bmatrix}$$

- Not as complex as it looks, weighted sum of gradients near pixel
  - Can use Gaussian weighting

Features

- $$S_w[p] = \begin{bmatrix} \sum_r w[r]\,(I_x[p-r])^2 & \sum_r w[r] I_x[p-r] I_y[p-r] \\ \sum_r w[r] I_x[p-r] I_y[p-r] & \sum_r w[r]\big(I_y[p-r]\big)^2 \end{bmatrix}$$

- Eigen vectors/values of this matrix summarize the distribution of the gradients nearby
  - λ_1 and λ_2 are eigenvalues
  - λ_1 and λ_2 both small: no gradient
  - λ_1 >> λ_2: gradient in one direction
  - λ_1 and λ_2 similar: multiple gradient directions, corner

Harris Corner Detection

- Calculate derivatives I_x and I_y
- Calculate $I_x I_x$, $I_y I_y$, $I_x I_y$
- Calculate weighted sums (S)
- Estimate response based on smallest eigenvalue
- Non-maximum suppression (like Canny)

- Approximation of smallest eigenvalue:

$$\lambda_{min} \approx \frac{\lambda_1 \lambda_2}{(\lambda_1 + \lambda_2)} = \frac{\det(S)}{trace(S)}$$

or

$$R = \det(S) - k\big(trace(S)\big)^2 = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$$

AALBORG UNIVERSITY
DENMARK

C. Harris and M. Stephens: A combined corner and edge detector. Proceedings of the 4th Alvey Vision Conference, pages 147-151, 1988

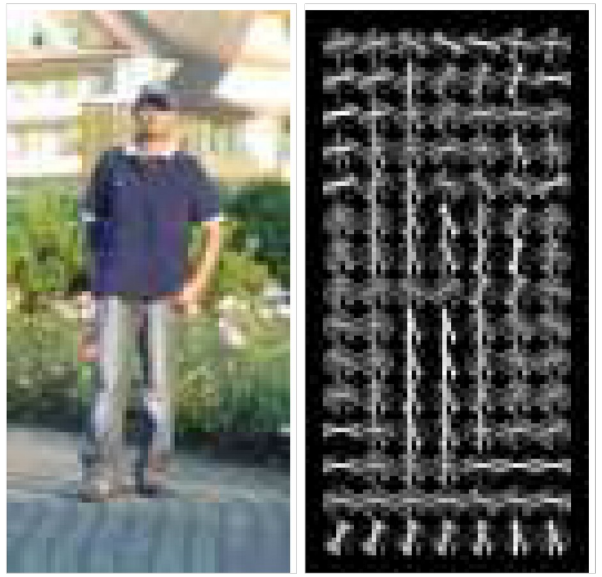


Histogram of Oriented Gradients (HOG)

Idea: Objects can be described by the distribution of local intensity gradients (edges)
Originally proposed for human detection

Algorithm:

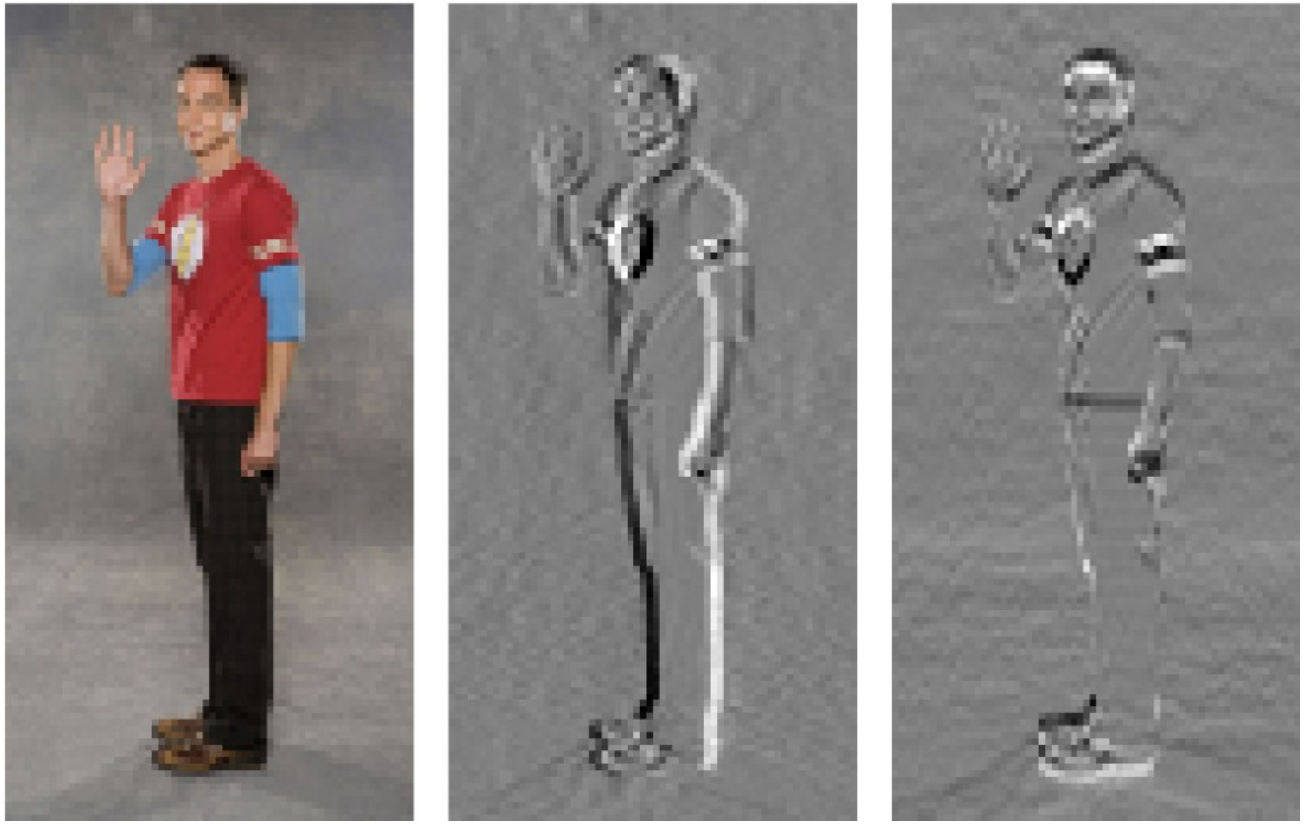
1. Normalize colour
2. Compute gradients
3. Weighted vote into spatial and orientation cells
4. Contrast normalize over overlapping spatial blocks
5. Collect HOG's over detection window
6. *Run linear SVM classifier*



Dalal and Triggs: "Histogram of Oriented Gradients for Human Detection". CVPR 2005

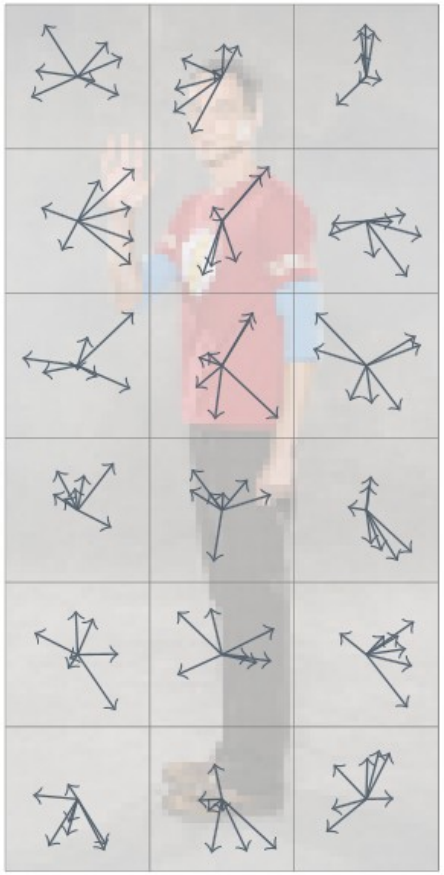
Histogram of Oriented Gradients (HOG)

- Compute gradients
- Kernels: $[-1, 0, 1]$ $[-1, 0, 1]^T$



Histogram of Oriented Gradients (HOG)

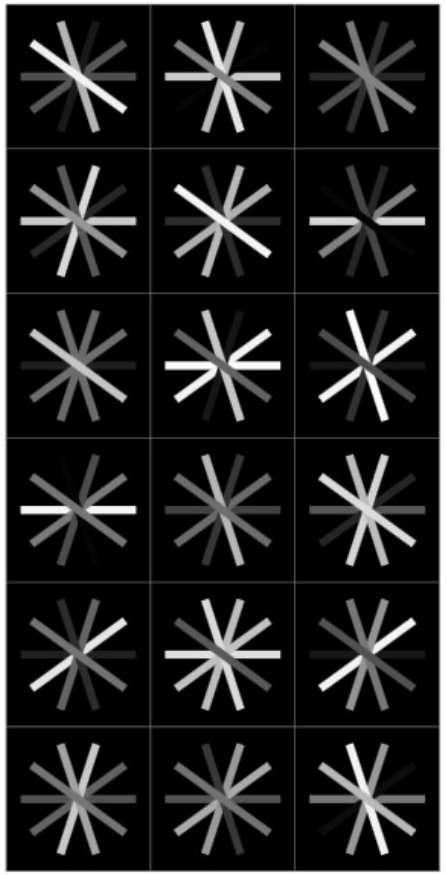
Compute gradient orientations in cells



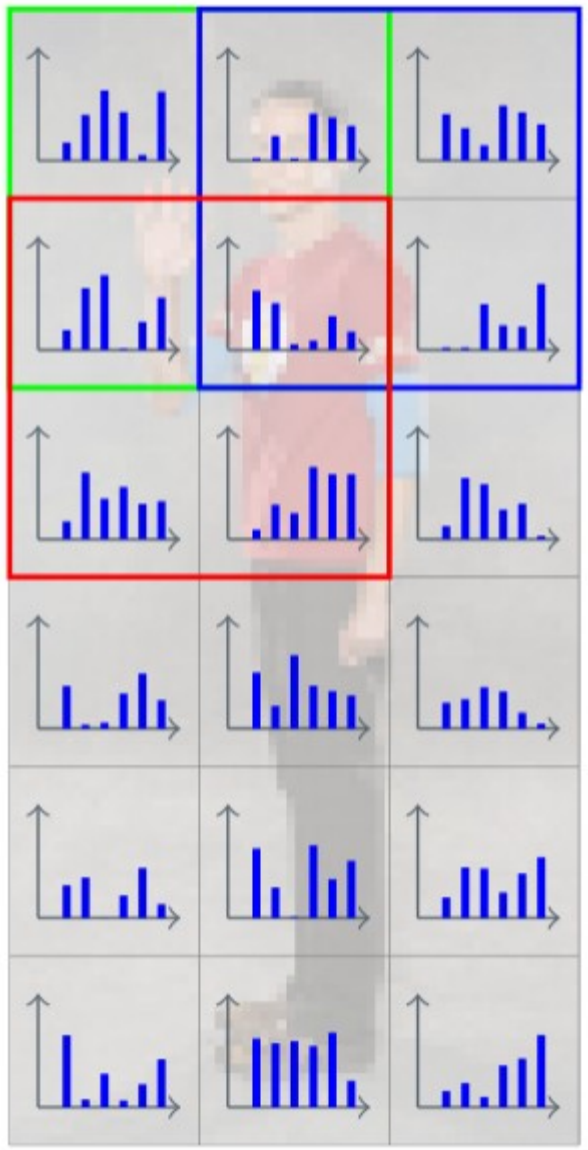
Assemble histograms



Visualized



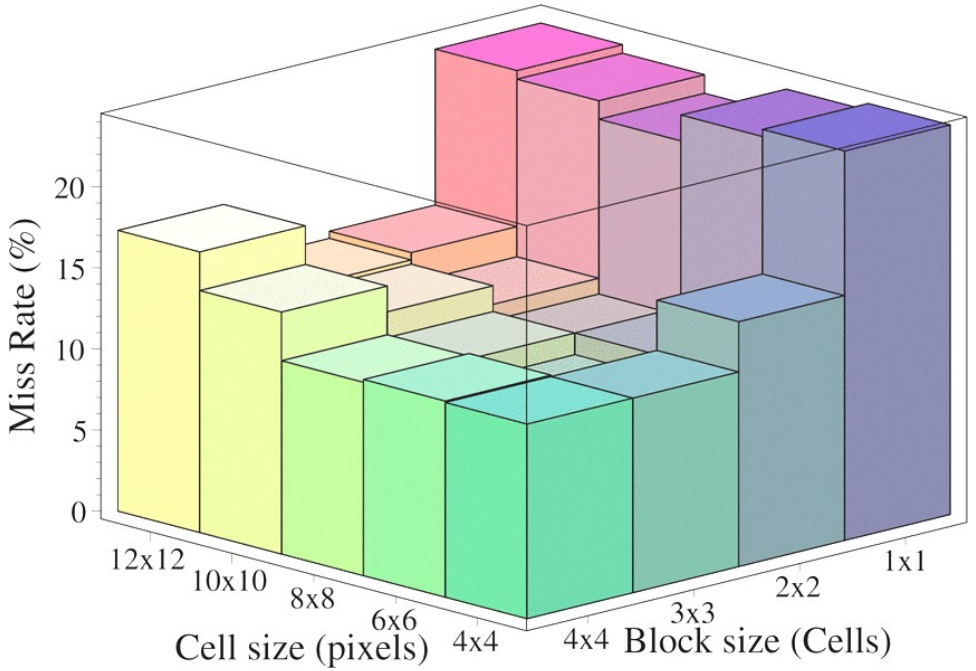
Histogram of Oriented Gradients (HOG)



- Normalization is done in (overlapping) blocks, which span multiple cells
- L2-norm, including a regularization term ϵ:

$$v \rightarrow v/(\|v\|_2^2 + \epsilon^2)$$

where **v** is the unnormalized combined block feature vector

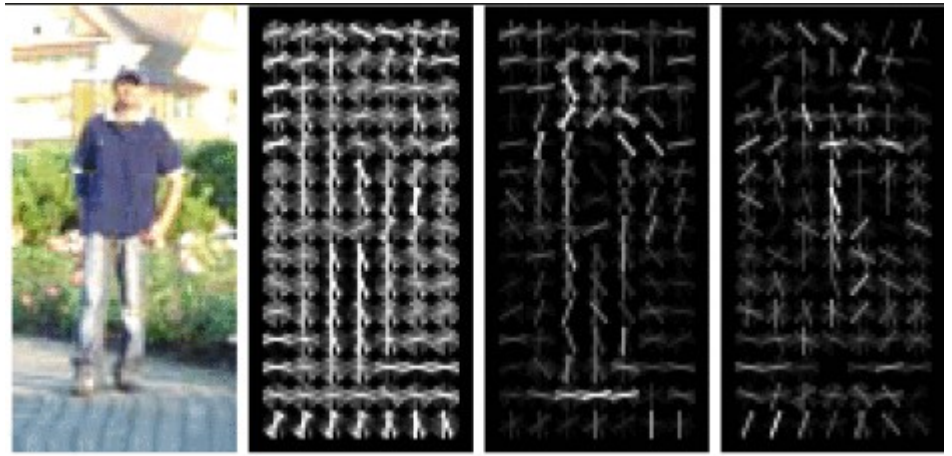


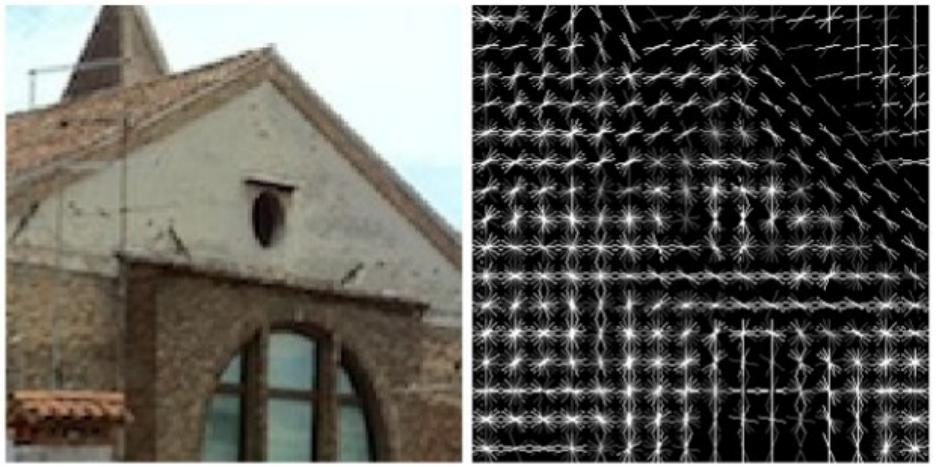
Histogram of Oriented Gradients (HOG)

Idea: Objects can be described by the distribution of local intensity gradients (edges)
Originally proposed for human detection

Algorithm:

1. Normalize colour
2. Compute gradients
3. Weighted vote into spatial and orientation cells
4. Contrast normalize over overlapping spatial blocks
5. Collect HOG's over detection window
6. Run linear SVM classifier





Dalal and Triggs: "Histogram of Oriented Gradients for Human Detection". CVPR 2005

Robust feature descriptors

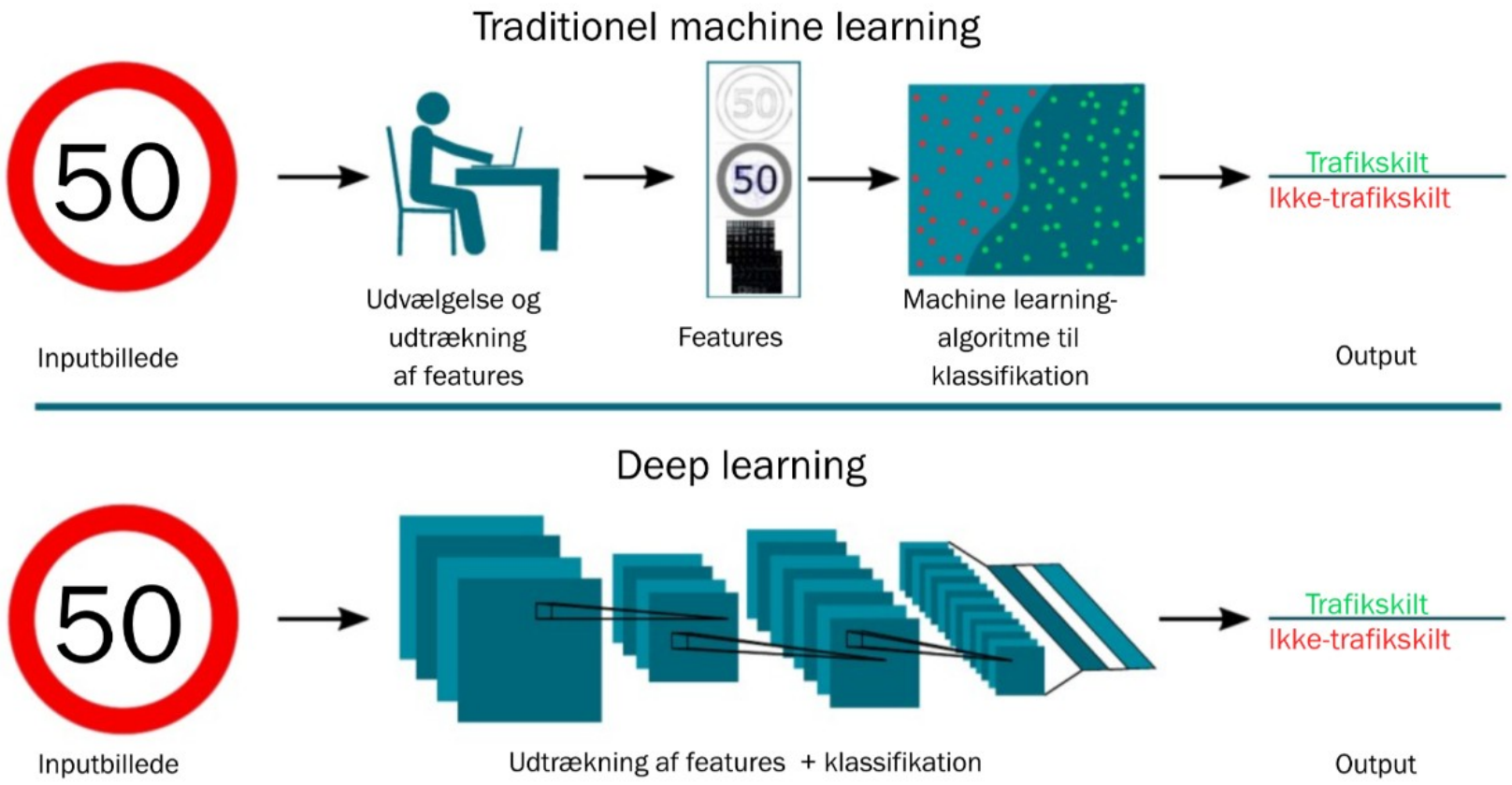
- Harris corners
  - Rotation invariant ☺
  - Not invariant to scale :-(
- HOG features
  - Lighting invariant ☺
  - Scale invariant – depending on detection window ☺
  - Not rotation invariant :-(
- We need a more general feature descriptor which is both rotation and scale invariant!

Feature detectors/descriptors

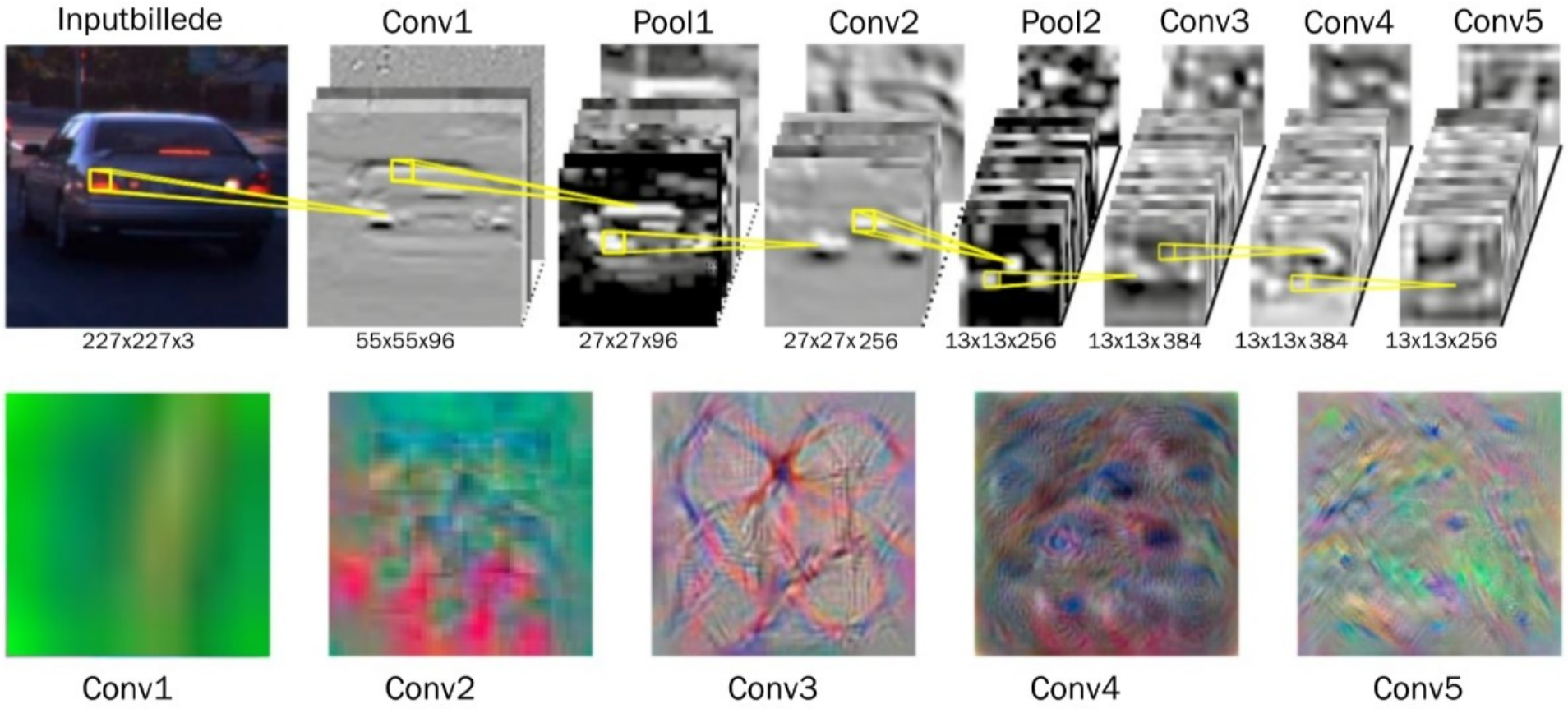
So many algorithms proposed, different pros and cons!

- SIFT
- SURF
- ORB
- BRIEF
- BRISK
- AKAZE
- FAST
- LATCH
- …

Deep features



Deep features

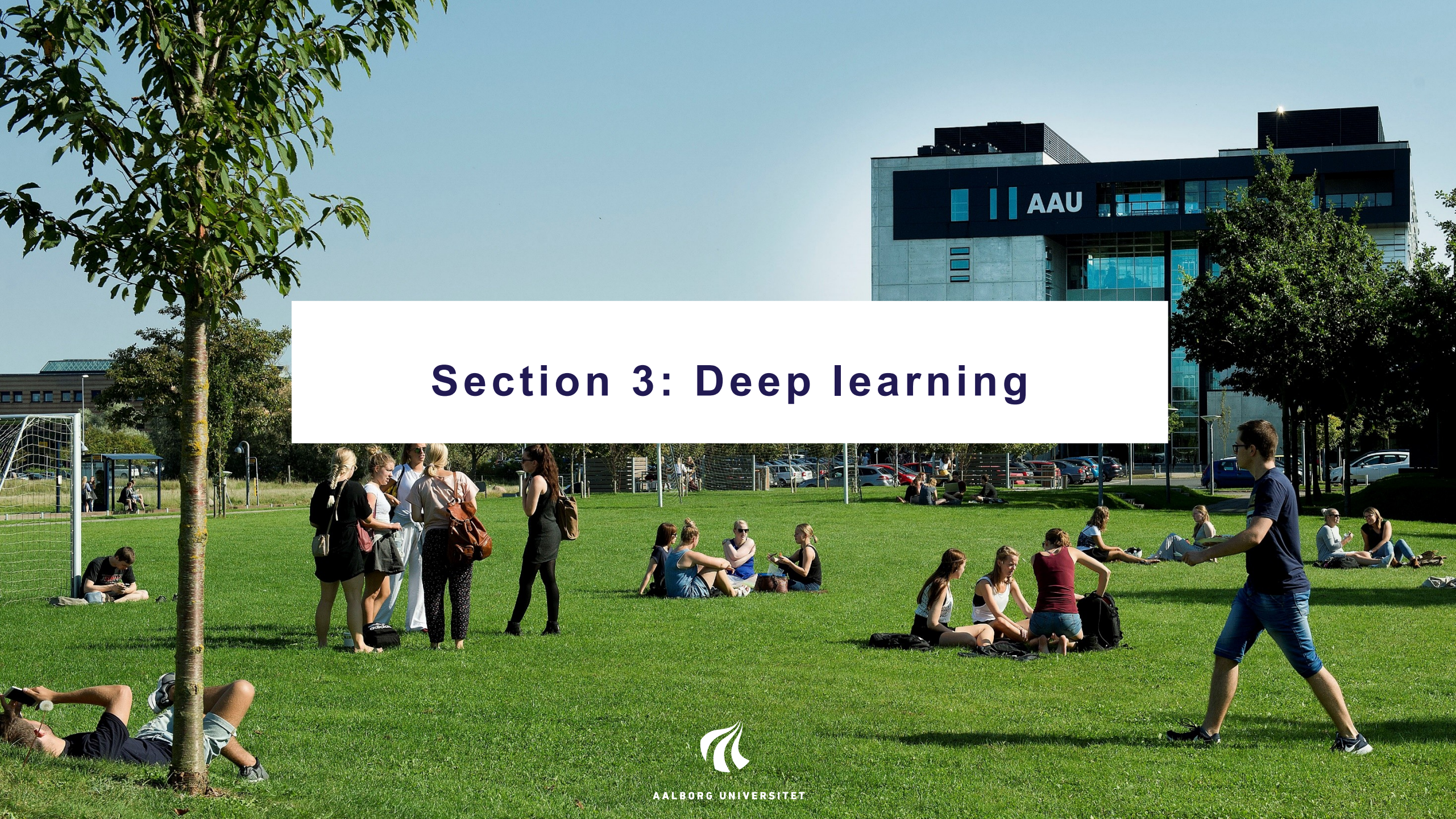


Exercise 2 (15 min): Build a Harris corner detector

- Build Harris from Section 1 tools: Sobel gradients, Gaussian window, structure tensor, corner response (see handout).
- Flat, edge or corner (4 min): predict λ1 and λ2 at the three checkerboard points, then run step 1. Does it match the lecture?
- Harris response (5 min): complete $R = \det - k \cdot \text{trace}^2$. Do you get 49 checkerboard corners? How close is R to cv2.cornerHarris?
- Window size (6 min): compare sigma 1, 2 and 4, then rel_thresh 0.001 and 0.1. Which corners survive a large window? What does that say about scale?
- Stretch: try k = 0.04, 0.15 and 0.25. Why does everything disappear at 0.25? Or visualise HOG on the astronaut.

https://github.com/andreasmoegelmose/cv_primer





Section 3: Deep learning

Deep learning

- Handcrafted models
  - Handcrafted features
  - Rule-based classification



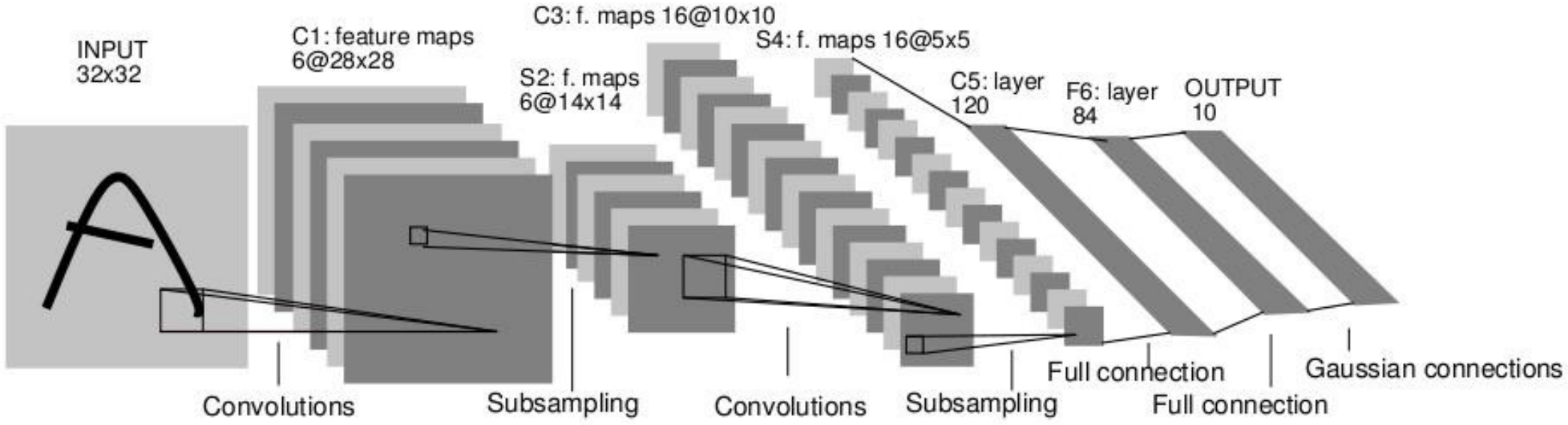
- Machine learning
  - Handcrafted features
  - Learned classification



- Deep learning
  - Learned features
  - Learned classification
  - Handcrafted networks

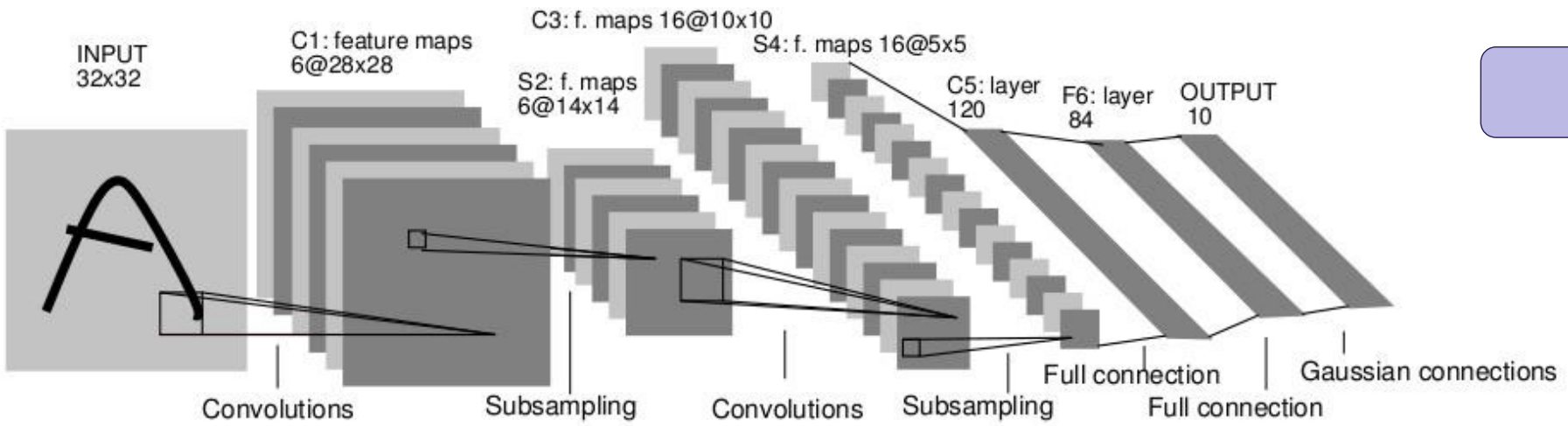
ConvNet architecture

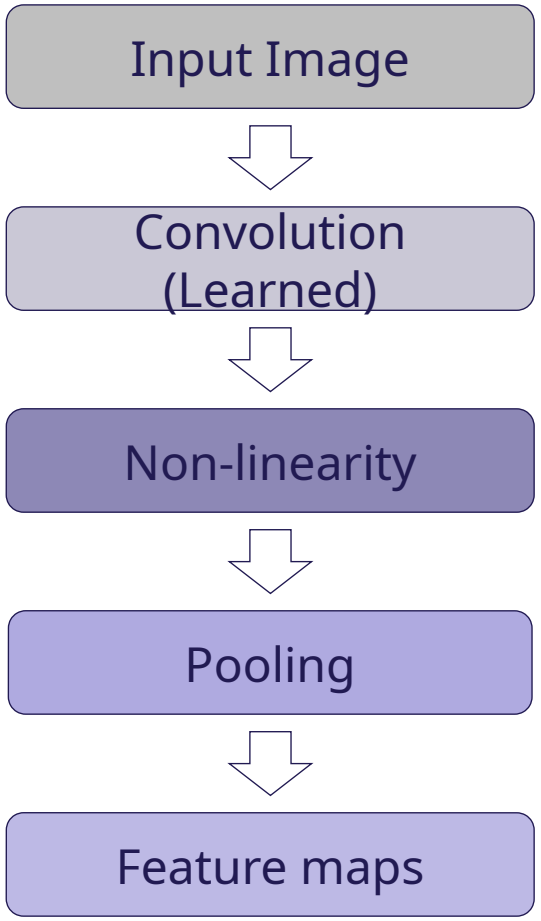
- Stack multiple stages of simple cells
- Higher stages compute more global, more invariant features
- Classification layer on top



ConvNet architecture

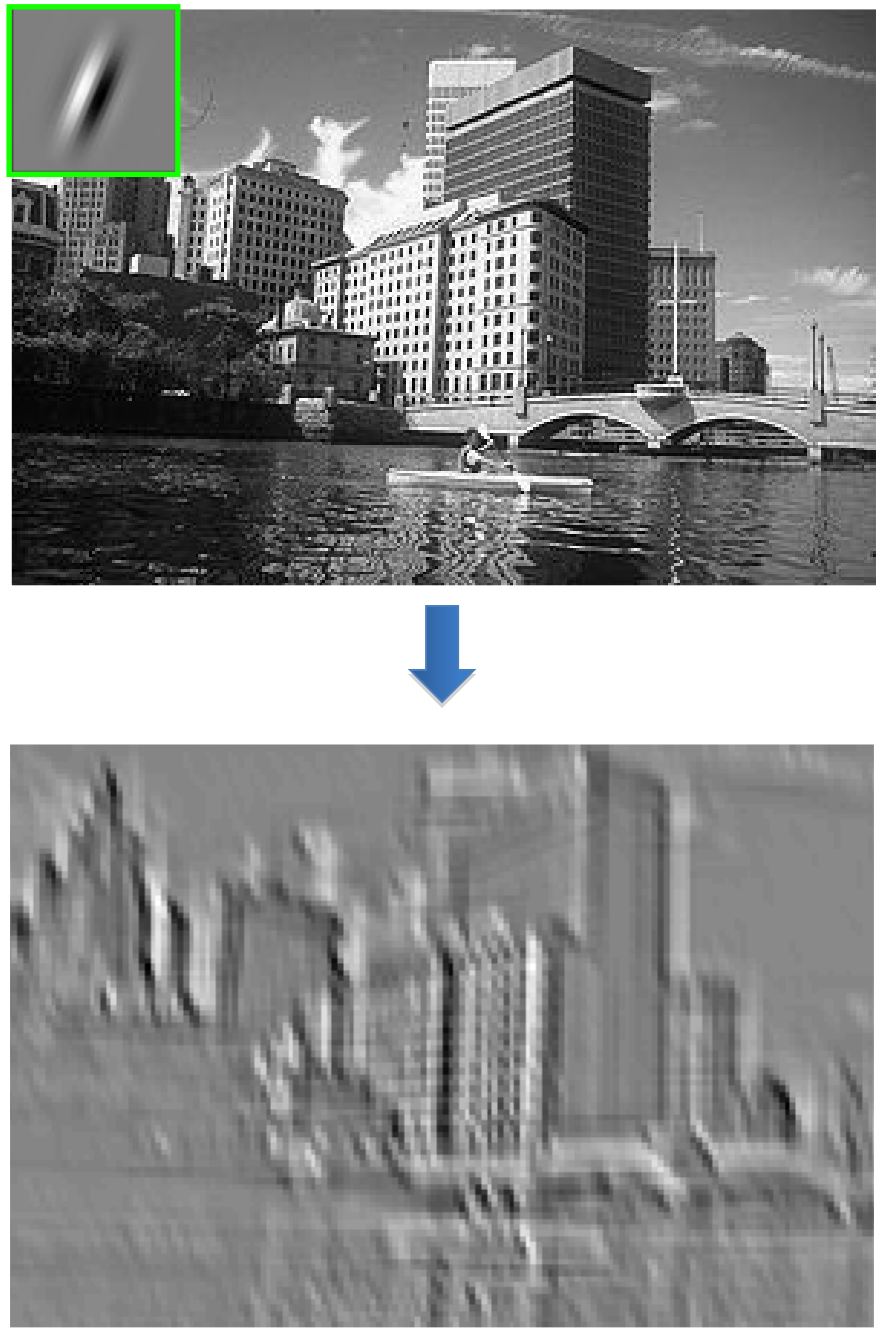
- Feed-forward
- Fully supervised
- Train convolutional filters by back-propagating classification errors





Filtering / Convolution

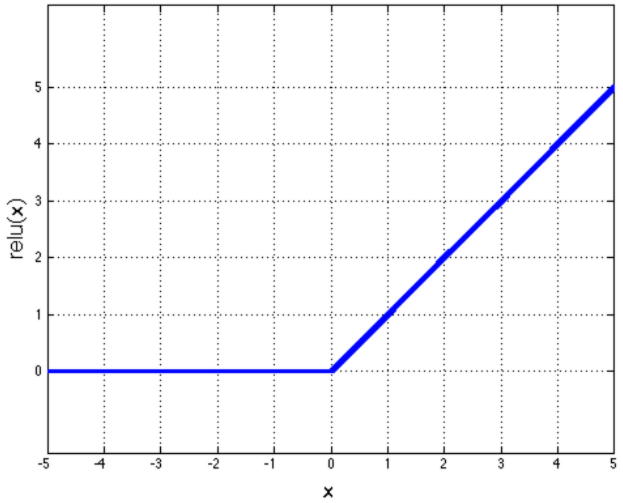
- Linear combination of input and filter
- Same filter at each location
- # Parameters: F x F x D x K + K
  - F filter size
  - D input depth
  - K number of filters



Non-linear activation functions

- Applied per-pixel
- ReLU most popular – faster training, more efficient

- ReLU: f(x) = max(0, x)

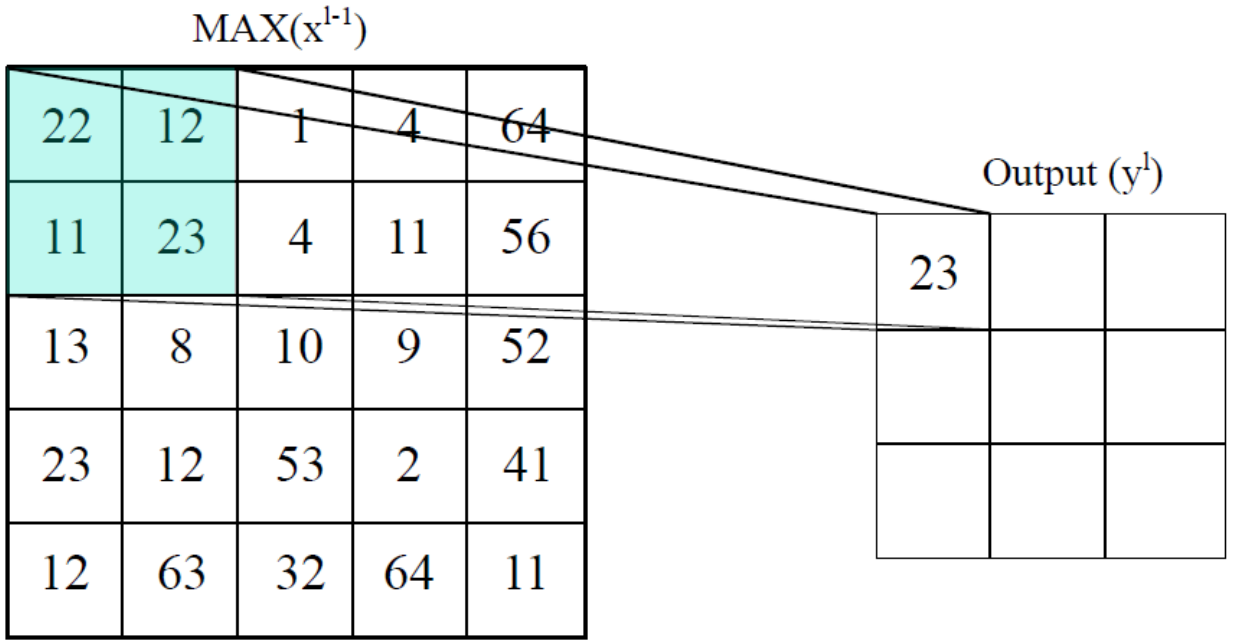




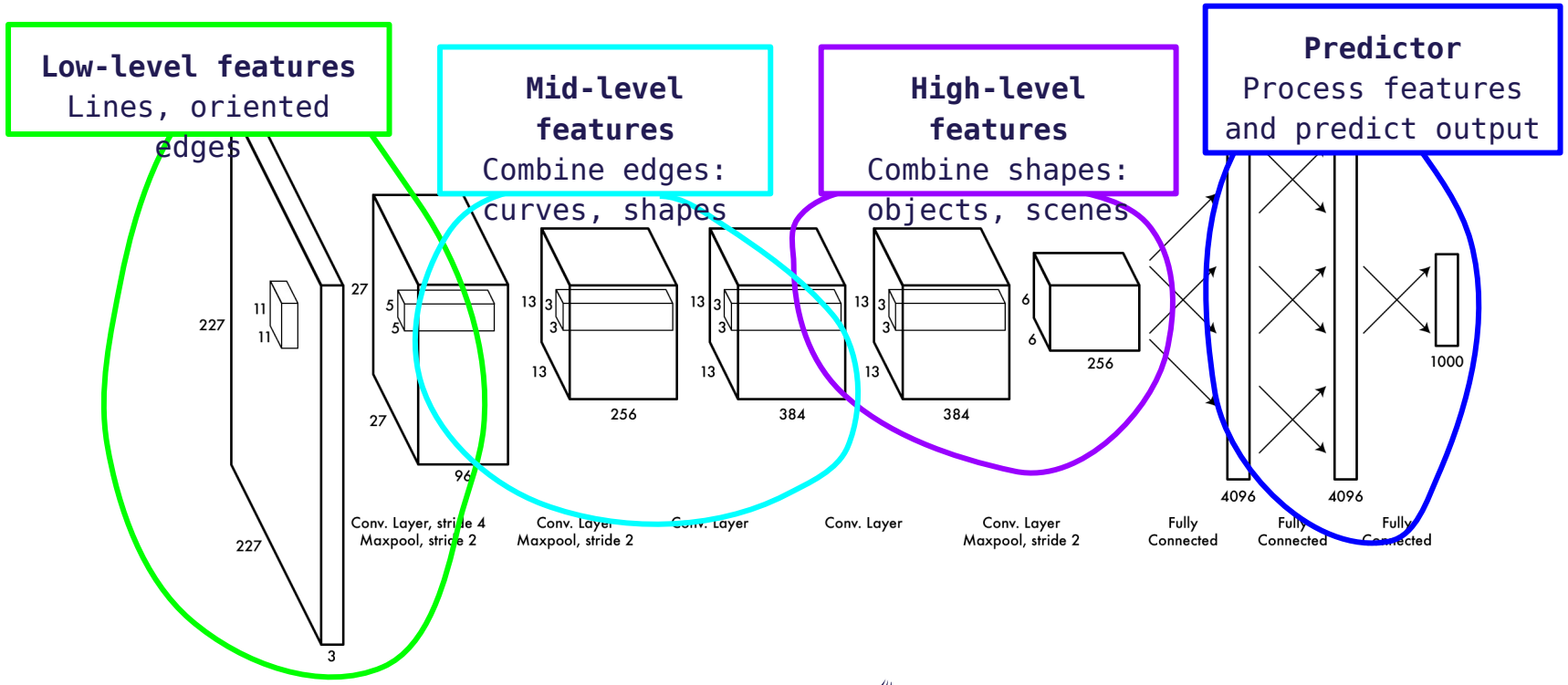


Pooling

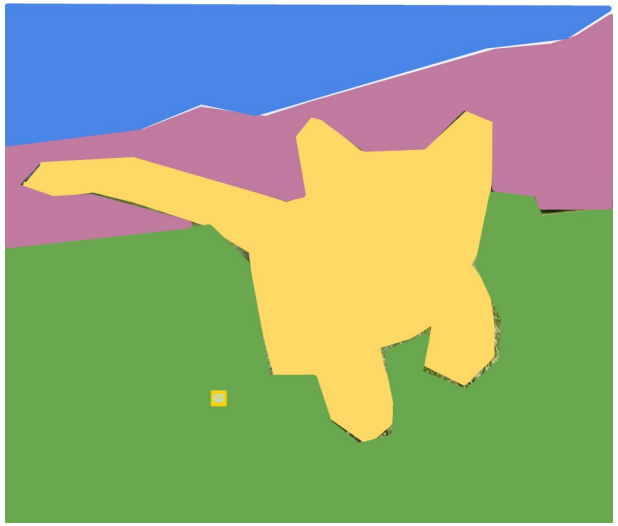
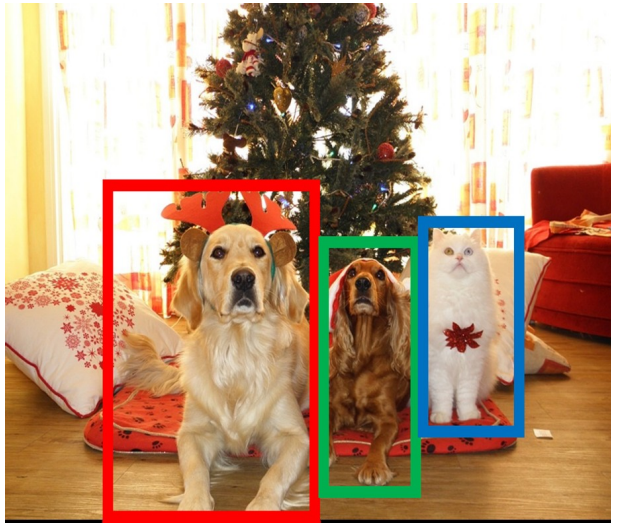
- Spatial pooling
  - Non-overlapping or overlapping regions
  - **MAX-pooling**
  - Average pooling
  - Sum-pooling
- Hyper-parameter – not learnable



Understanding neural networks



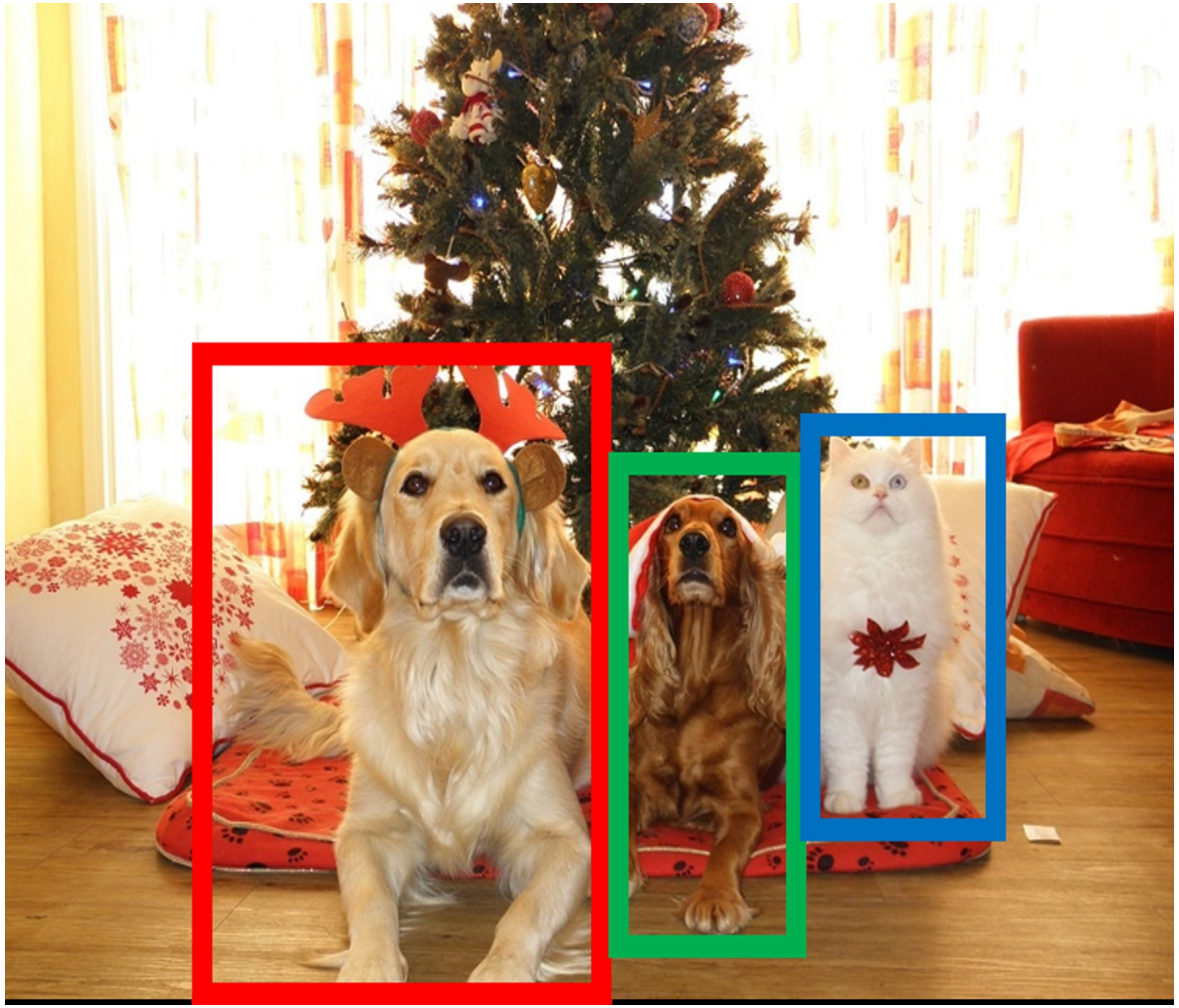
Computer Vision Tasks

Classification	Semantic Segmentation	Object detection	Instance segmentation
			
CAT	GRASS, CAT, TREE, SKY	DOG, DOG, CAT	DOG, DOG, CAT
No spatial extent	No objects, just pixels	Multiple objects	

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Object detection: Task definition

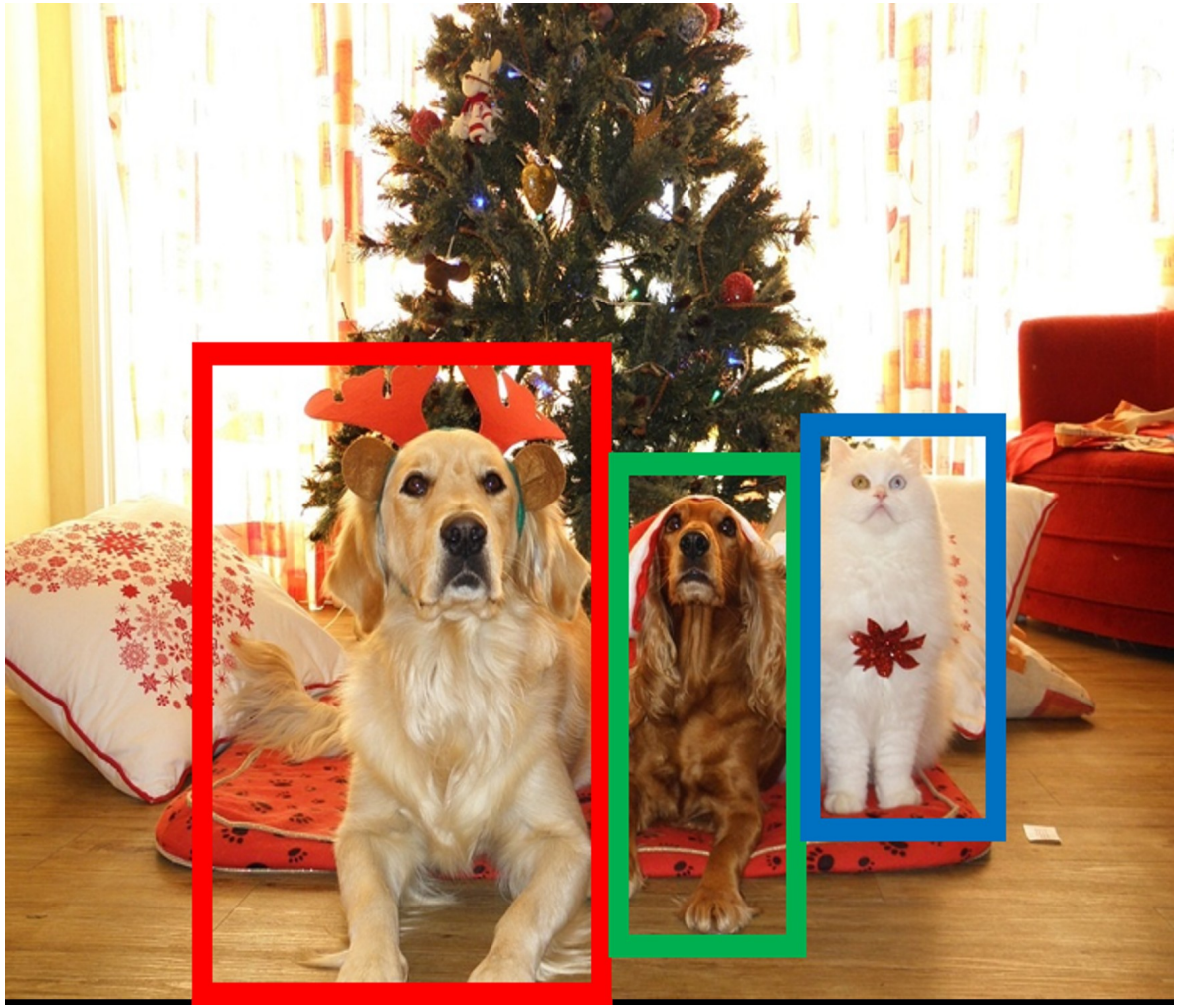
- Input: Single RGB Image
- Output: A *set* of detected objects.
  For each predicted object:
  - Category label (class)
    From fixed set of categories
  - Bounding box
    x, y, width, height



Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Object detection: Challenges

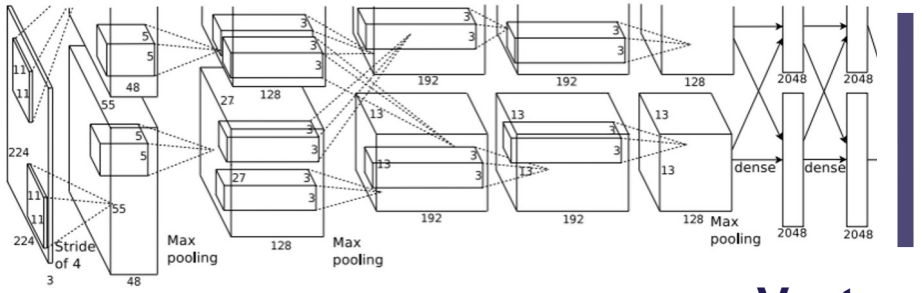
- **Multiple outputs:**
  Need to output variable number of objects per image
- **Multiple types of output:**
  Need to predict ”what” (class) as well as “where” (bounding box)
- **Large images:**
  Classification works at 224x224; need higher resolution for detection, at least 640x640



Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Detecting a single object





Vector:
4096

Fully Connected:
4096 to 1000

Class scores
Cat: 0.9
Dog: 0.05
Car: 0.01
…

Softmax loss

Correct label:
Cat

Multitask loss

Weighted sum → **Loss**

Fully Connected:
4096 to 4

Box coordinates
(x, y, w, h)

L2 loss

Correct box:
(x', y', w', h')

"What"

"Where"

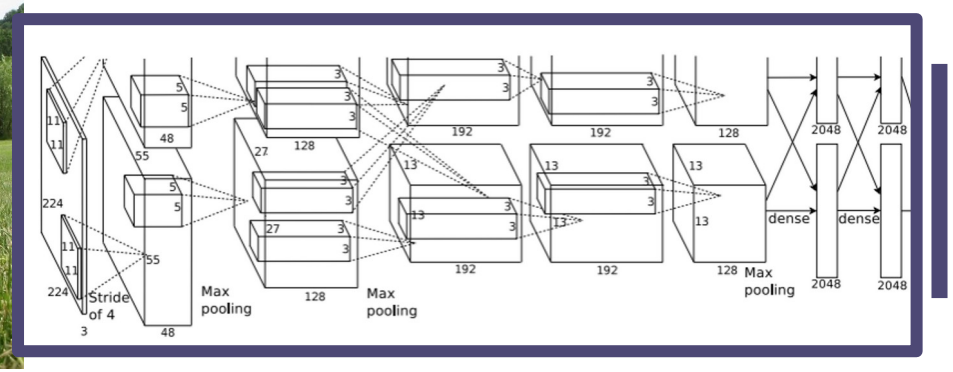
What: Classification
Where: Regression

AALBORG UNIVERSITY
DENMARK

Detecting a single object

Often pretrained on ImageNet (transfer learning)





- Only works with 1 object per image!
- We need to detect multiple objects per image

"What"

Fully Connected: 4096 to 1000

Class scores
Cat: 0.9
Dog: 0.05
Car: 0.01
…

Softmax loss

Correct label:
Cat

Multitask loss

Weighted sum → **Loss**

Vector: 4096

Fully Connected: 4096 to 4

Box coordinates (x, y, w, h)

L2 loss

Correct box:
(x', y', w', h')

"Where"

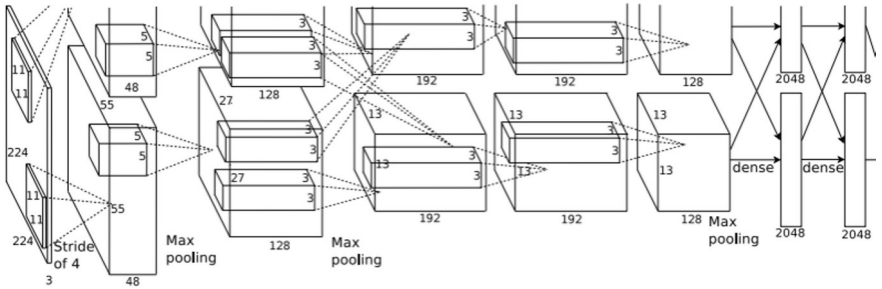
AALBORG UNIVERSITY
DENMARK

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Detecting multiple objects

Need variable number of outputs per image

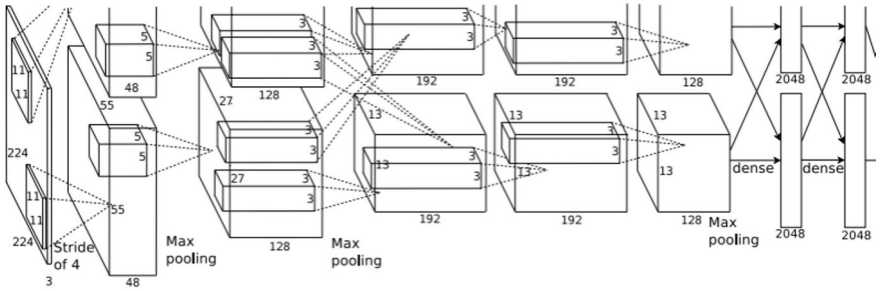




CAT: (x, y, w, h)

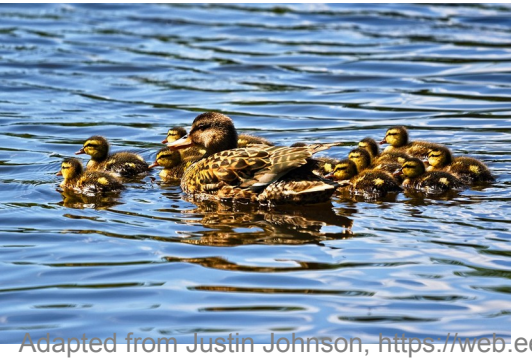
4 numbers

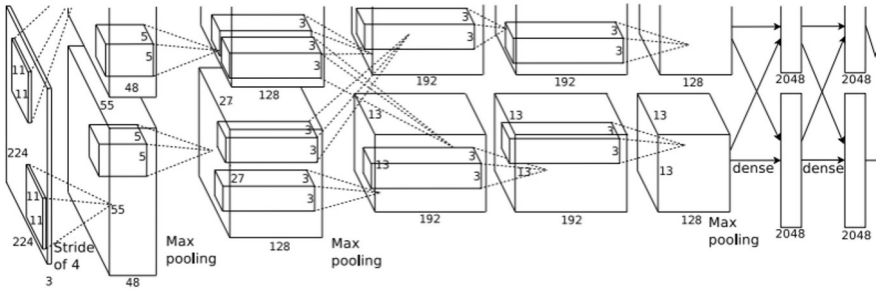




DOG: (x, y, w, h)
DOG: (x, y, w, h)
CAT: (x, y, w, h)

12 numbers





DUCK: (x, y, w, h)
DUCK: (x, y, w, h)
….

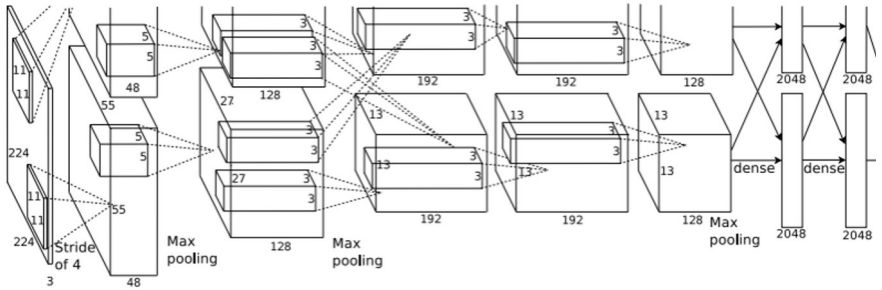
k x 4 numbers

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Detecting multiple objects: Sliding window



- For each window position:
  - Warp image, apply a CNN
  - Classification problem (with extra background category)



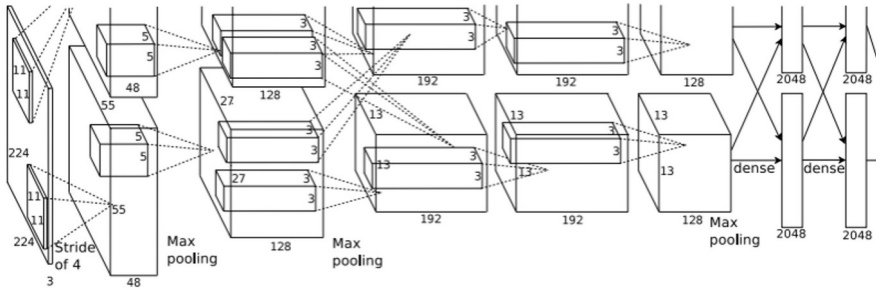
Dog? No
Cat? No
Car? No
Background? Yes

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Detecting multiple objects: Sliding window

- For each window position:
  - Warp image, apply a CNN
  - Classification problem (with extra background category)





Dog? Yes
Cat? No
Car? No
Background? No

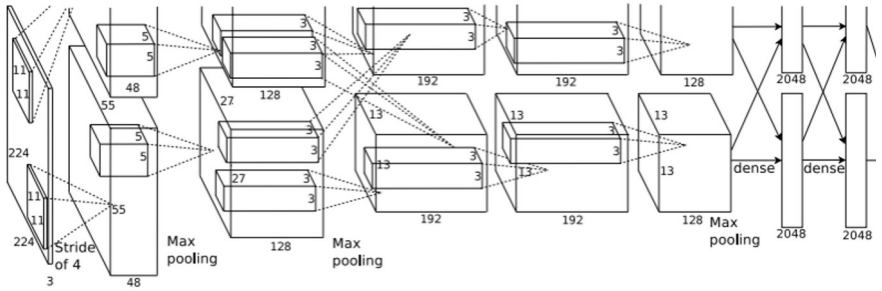
AALBORG UNIVERSITY
DENMARK

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Detecting multiple objects: Sliding window



- For each window position:
  - Warp image, apply a CNN
  - Classification problem (with extra background category)



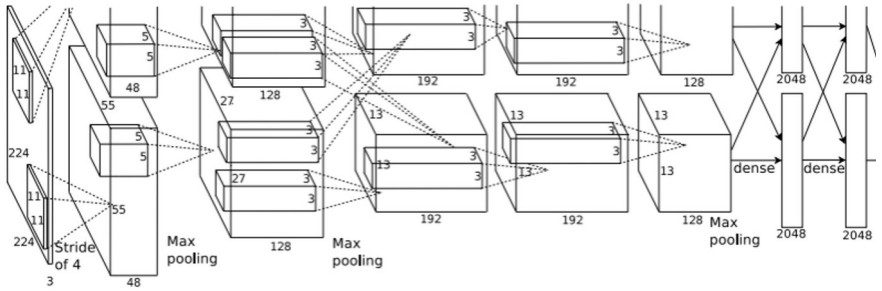
Dog? Yes
Cat? No
Car? No
Background? No

AALBORG UNIVERSITY
DENMARK

Detecting multiple objects: Sliding window



- For each window position:
  - Warp image, apply a CNN
  - Classification problem (with extra background category)



Dog? No
Cat? Yes
Car? No
Background? No

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Detecting multiple objects: Sliding window



- How many boxes to evaluate?
- Parameters
  - Image width (W), height (H)
  - Box width (w), height (h)

- 800 x 600 image
  ~ 58M boxes!

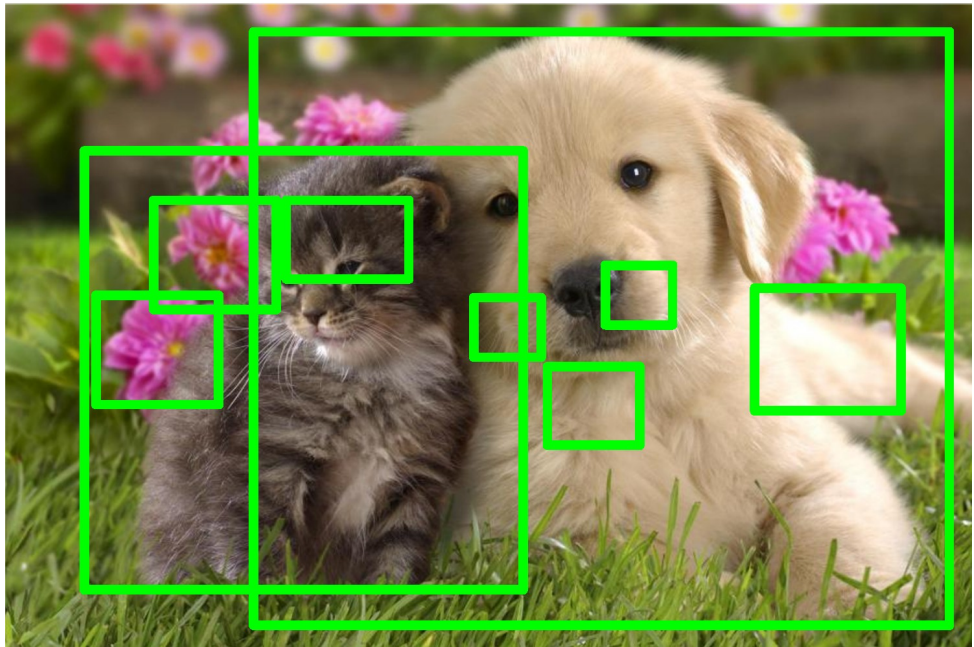
- Infeasible to evaluate all box positions and sizes

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Detecting multiple objects: Region proposals

- Find a small set of boxes that are likely to cover all objects
- Often based on heuristics
  - Look for “blob-like” image regions
- Relatively fast to run
  - **Selective Search** (2013) gives 2000 region proposals in a few seconds on CPU

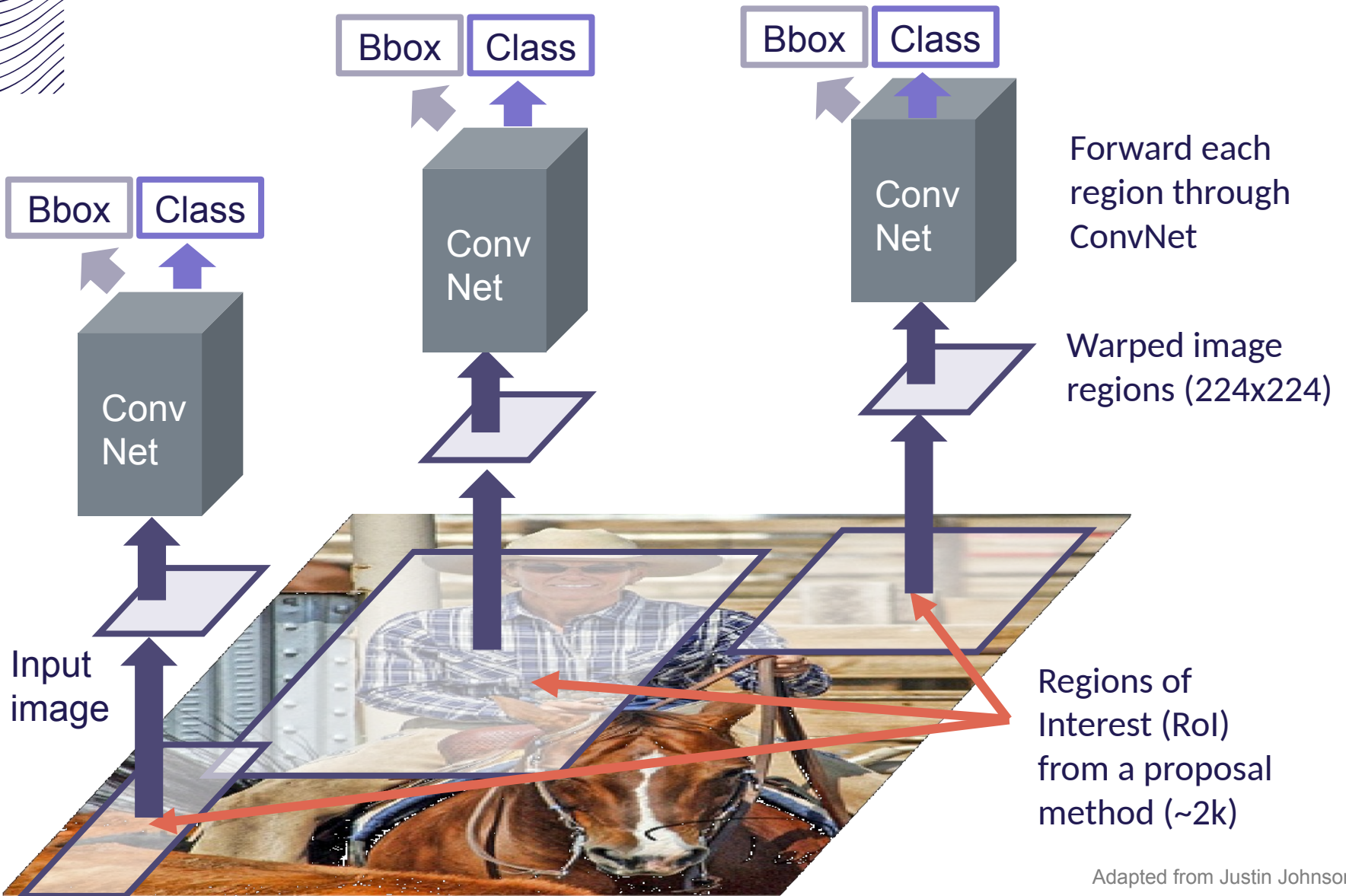




Alexe et al, “Measuring the objectness of image windows”, TPAMI 2012
Uijlings et al, “Selective Search for Object Recognition”, IJCV 2013
Cheng et al, “BING: Binarized normed gradients for objectness estimation at 300fps”, CVPR 2014
Zitnick and Dollar, “Edge boxes: Locating object proposals from edges”, ECCV 2014

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

R-CNN: Region-based CNN



Forward each region through ConvNet

Warped image regions (224x224)

Regions of Interest (RoI) from a proposal method (~2k)

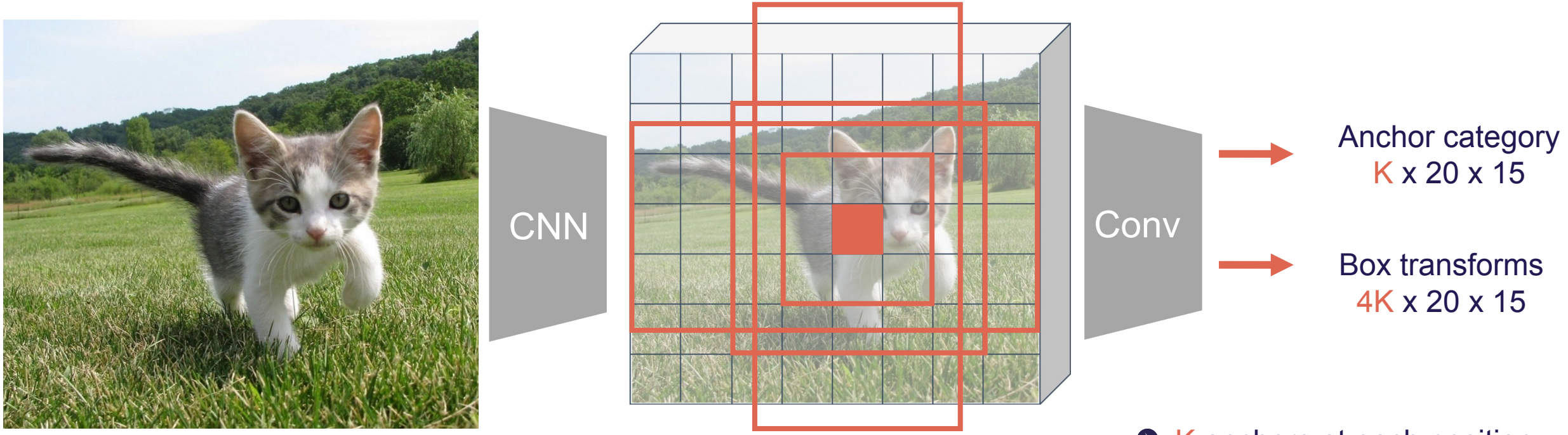
Classify each region

Bounding box regression: Predict "transform" to correct the Region-of-interest (ROI). 4 numbers (t_x, t_y, t_h, t_w)

Single-stage Object Detection

- RPN: Classify each anchor as object / not object

Run backbone CNN to get features aligned to input image



Input image
(e.g. 3 x 640 x 480)

Image features
(e.g. 512 x 20 x 15)

- K anchors at each position in image feature map

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Single-stage Object Detection

- RPN: Classify each anchor as object / not object
- Single-stage detector: Classify each object as one of C categories (or background)

Run backbone CNN to get features aligned to input image



Input image
(e.g. 3 x 640 x 480)

CNN

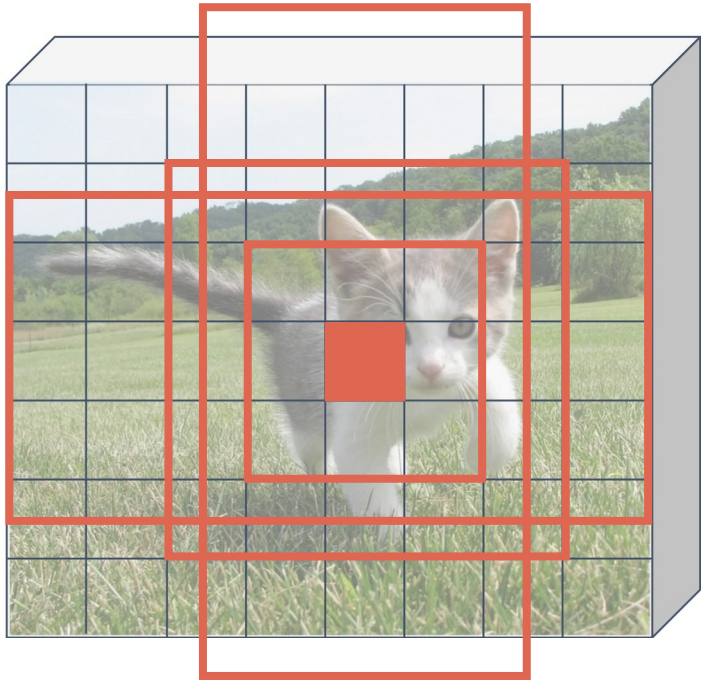


Image features
(e.g. 512 x 20 x 15)

Conv

→ Anchor category
(C+1) x K x 20 x 15

→ Box transforms
4K x 20 x 15

- K anchors at each position in image feature map

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

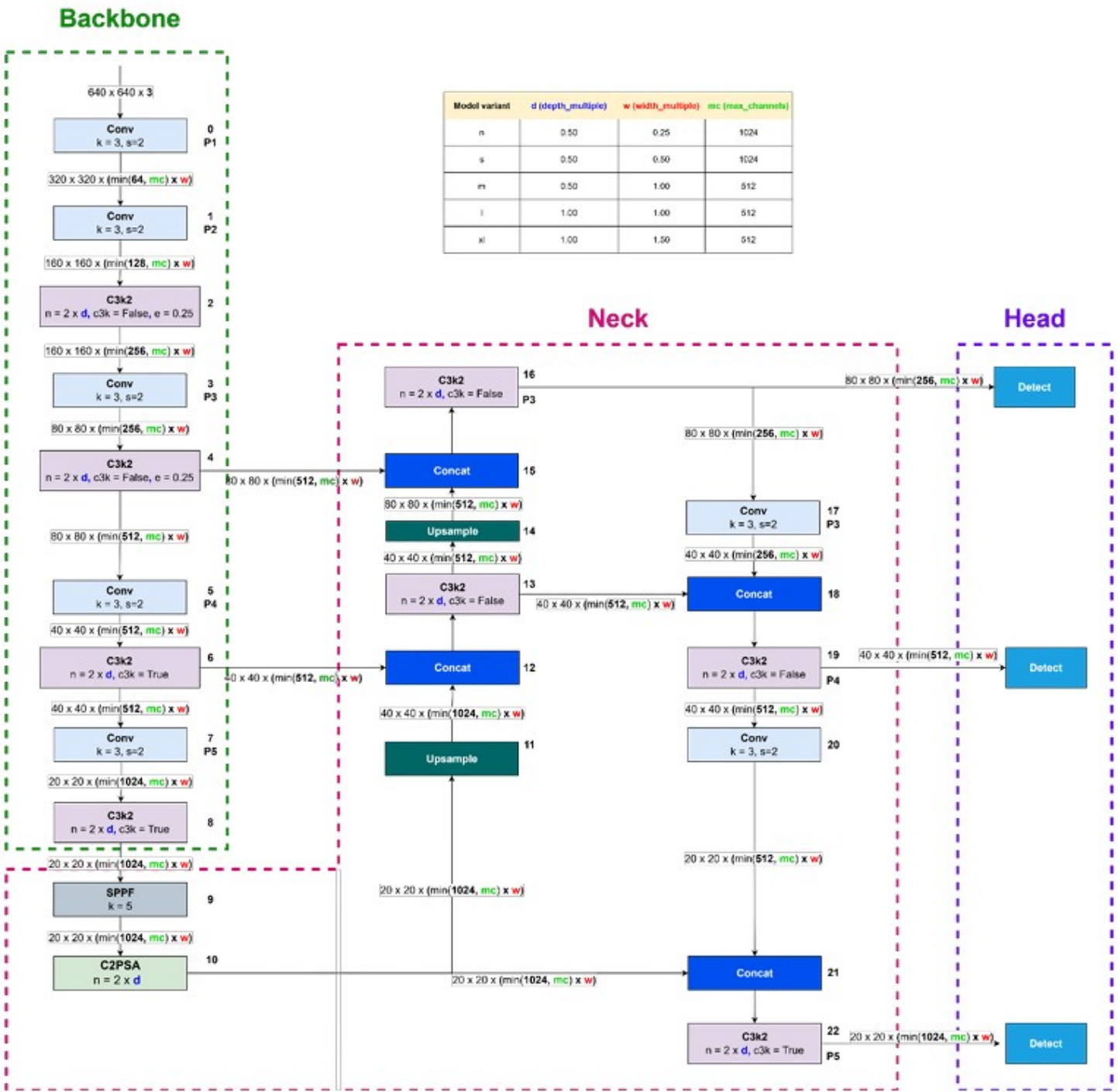
Single-stage Object Detection

- Classical single-stage architectures:
  - YOLO (v1, v2, …, v12, v26)
  - SSD
  - …

- Conventional thoughts:
  - Two-stage methods are more accurate, but slower
  - Single-stage methods are faster, but less accurate
- Today:
  - Single-stage methods are faster, and sometimes more accurate!



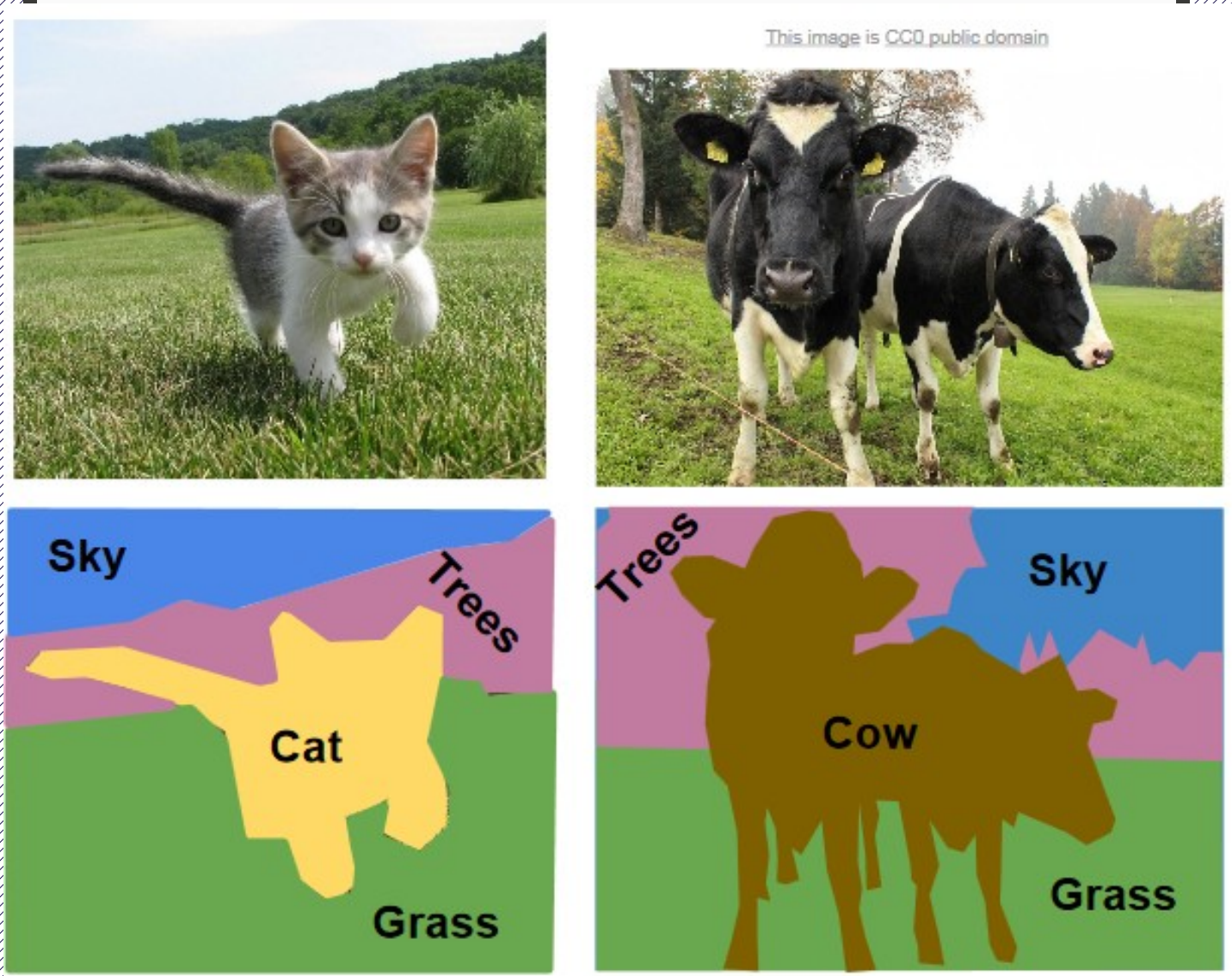
YOLO-v11



Model variant	d (depth_multiple)	w (width_multiple)	mc (max_channels)
n	0.50	0.25	1024
s	0.50	0.50	1024
m	0.50	1.00	512
l	1.00	1.00	512
xl	1.00	1.50	512

Semantic segmentation

- Label each pixel in the image with a category label
- Don’t differentiate instances, only care about pixels

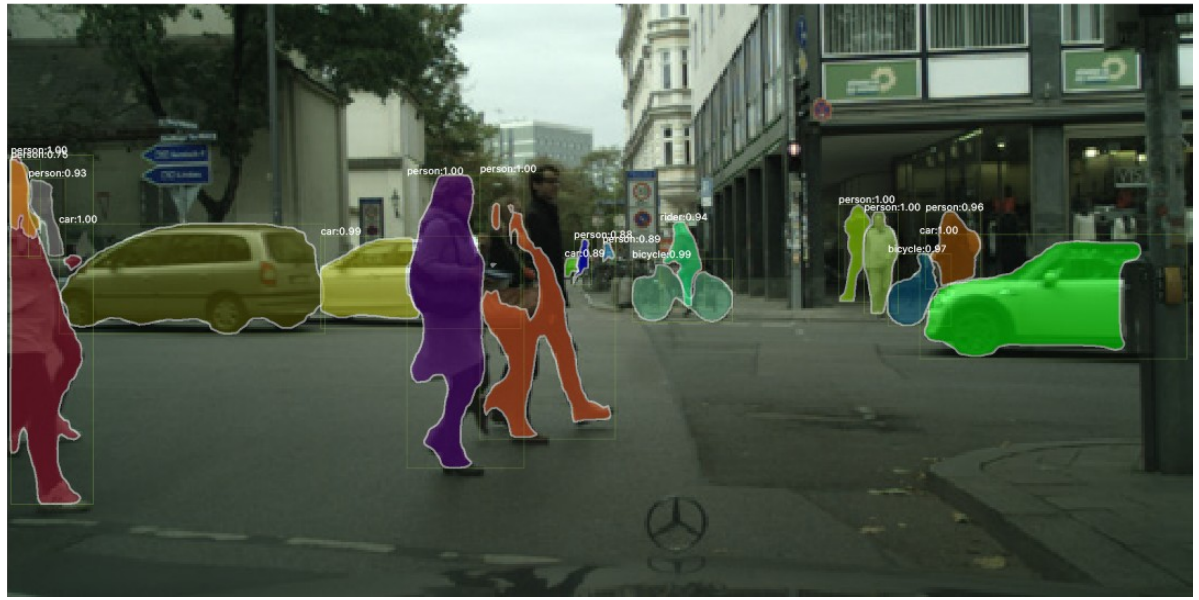


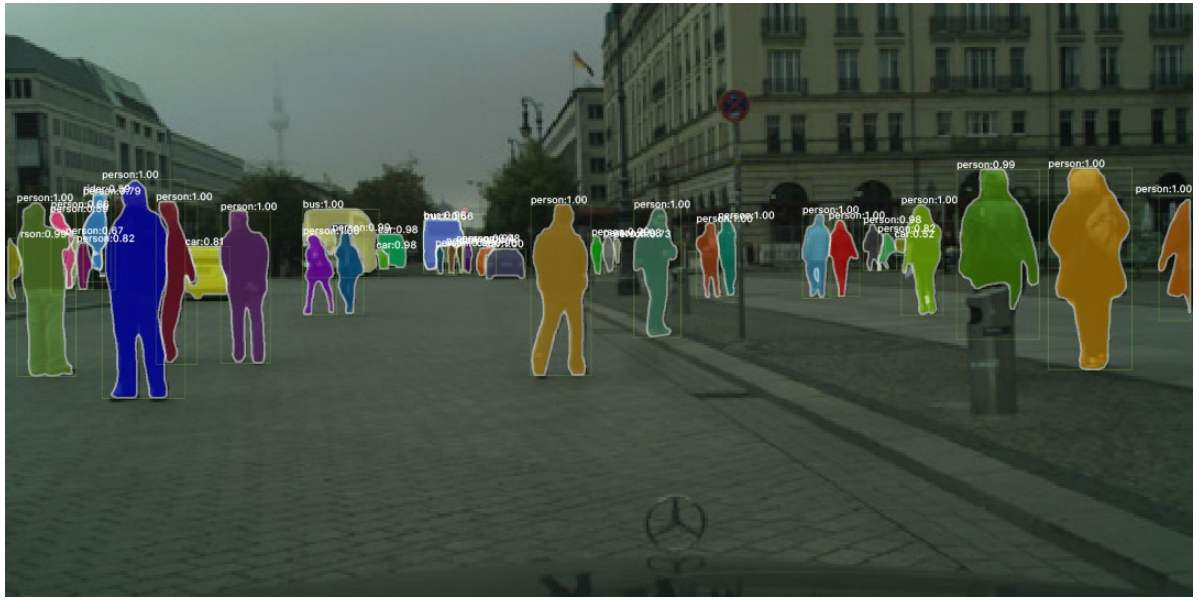
Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Instance segmentation

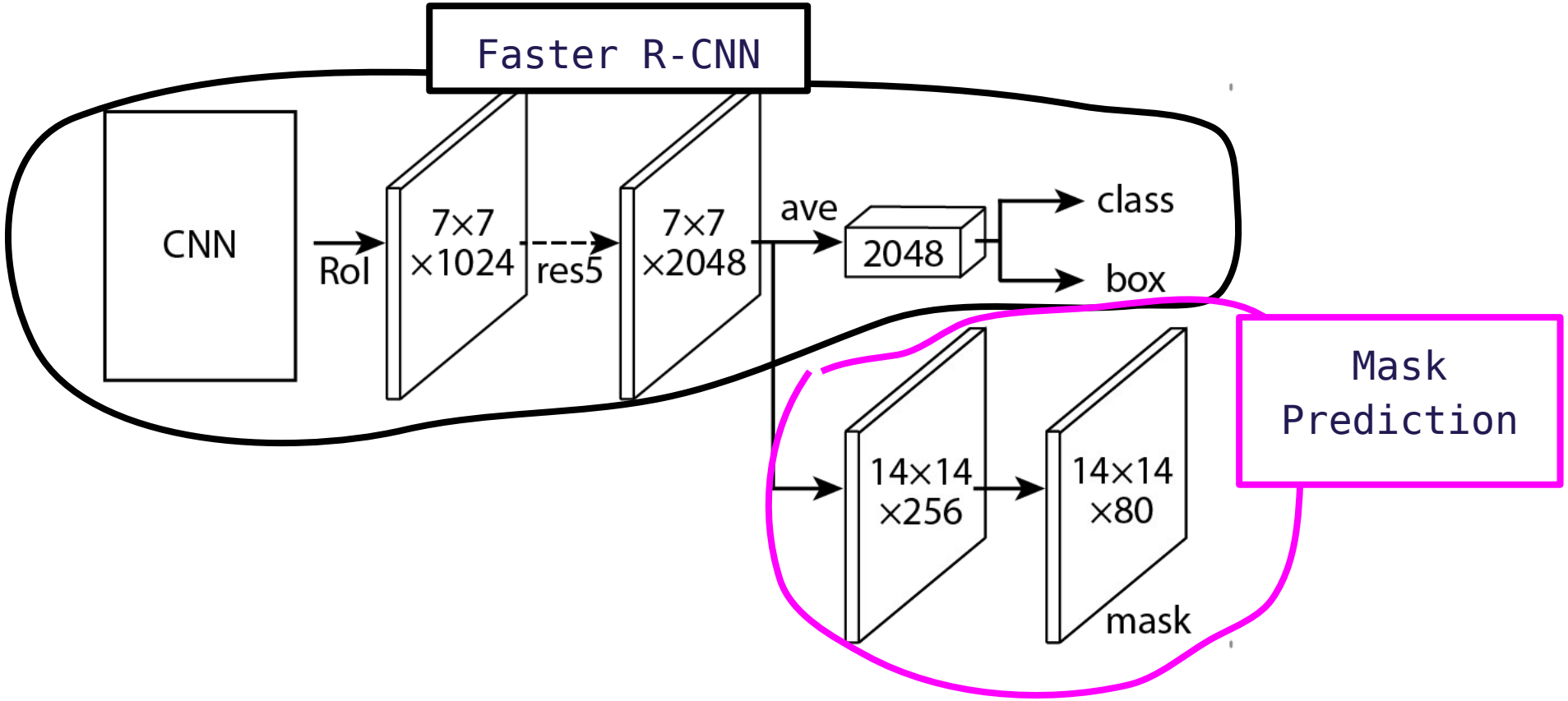
Given an image produce instance-level segmentation

- Which class does each pixel belong to
- Also which instance





Instance segmentation – Mask R-CNN



Mask R-CNN:
https://arxiv.org/pdf/1703.06870.pdf

Basic performance metrics

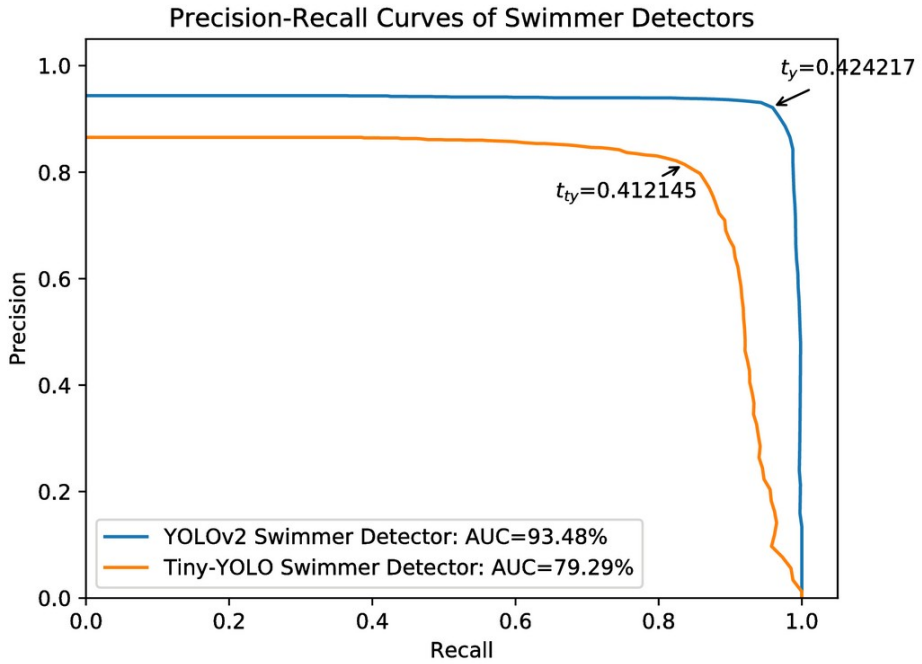
- TP = true positives
- FP = false positives
- FN = false negatives
- (TN = true negatives)

- Recognition rate = TP/number of tested samples
- Error % = 100 % - (Recognition rate x 100 %)

- Precision = TP/(TP+FP)
- Recall = TP/(TP+FN)

Describing system performance

- Precision-recall curves:



Precision:

$$\frac{TP}{TP+FP}$$

"how much is relevant?"

Recall (same as FPR):

$$\frac{TP}{TP+FN}$$

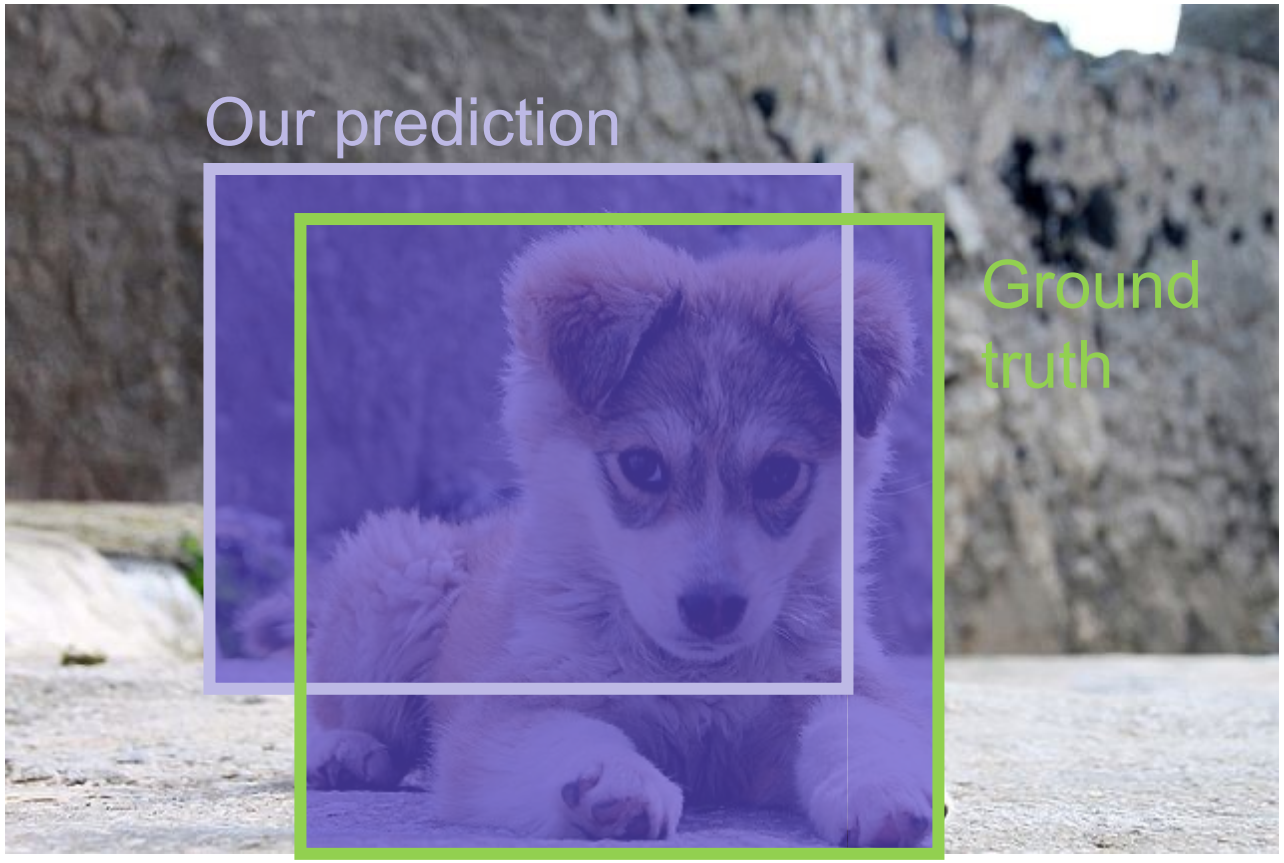
"do we find all the things?"

- The more similar a curve is to the shape: ⌐ the better the system

Evaluating detectors

- Let us assume that we have previously defined n ground-truth boxes
- How can we compare our prediction to the ground-truth box?
- Intersection Over Union

$$\frac{\text{Area of intersection}}{\text{Area of union}}$$



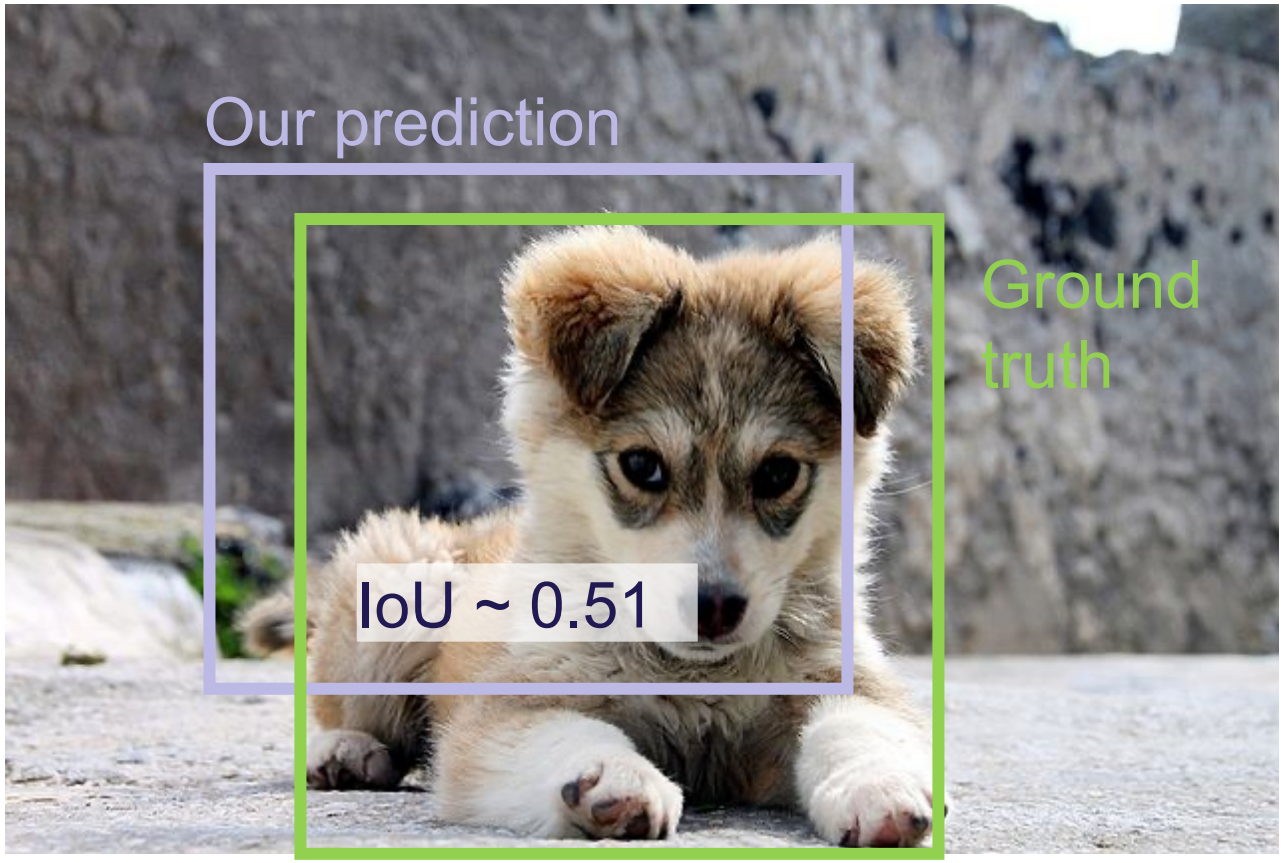
Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Evaluating detectors

- Let us assume that we have previously defined n ground-truth boxes
- How can we compare our prediction to the ground-truth box?
- Intersection Over Union

$$\frac{\text{Area of intersection}}{\text{Area of union}}$$

- IoU > 0.5: “decent”



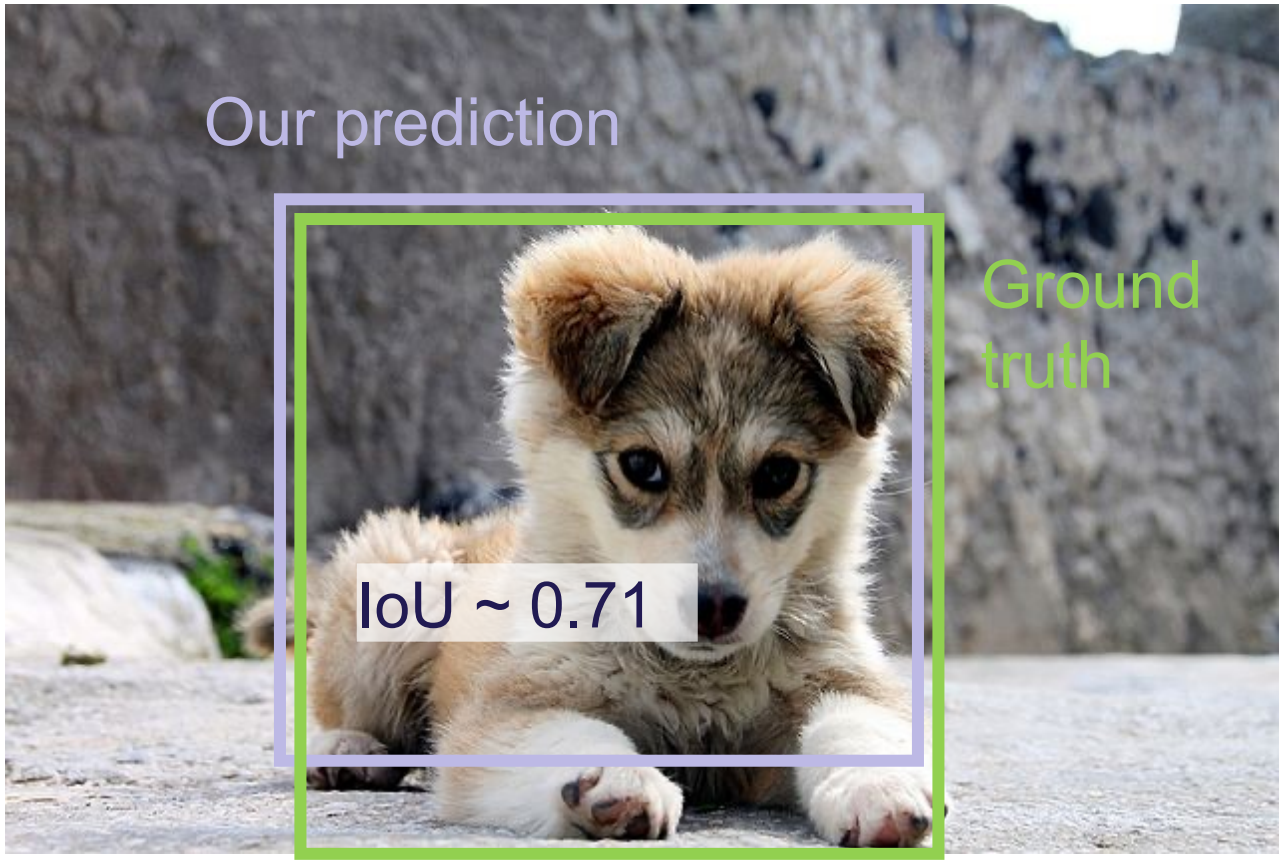
Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Evaluating detectors

- Let us assume that we have previously defined n ground-truth boxes
- How can we compare our prediction to the ground-truth box?
- Intersection Over Union

$$\frac{\text{Area of intersection}}{\text{Area of union}}$$

- IoU > 0.5: “decent”
- IoU > 0.7: “pretty good”



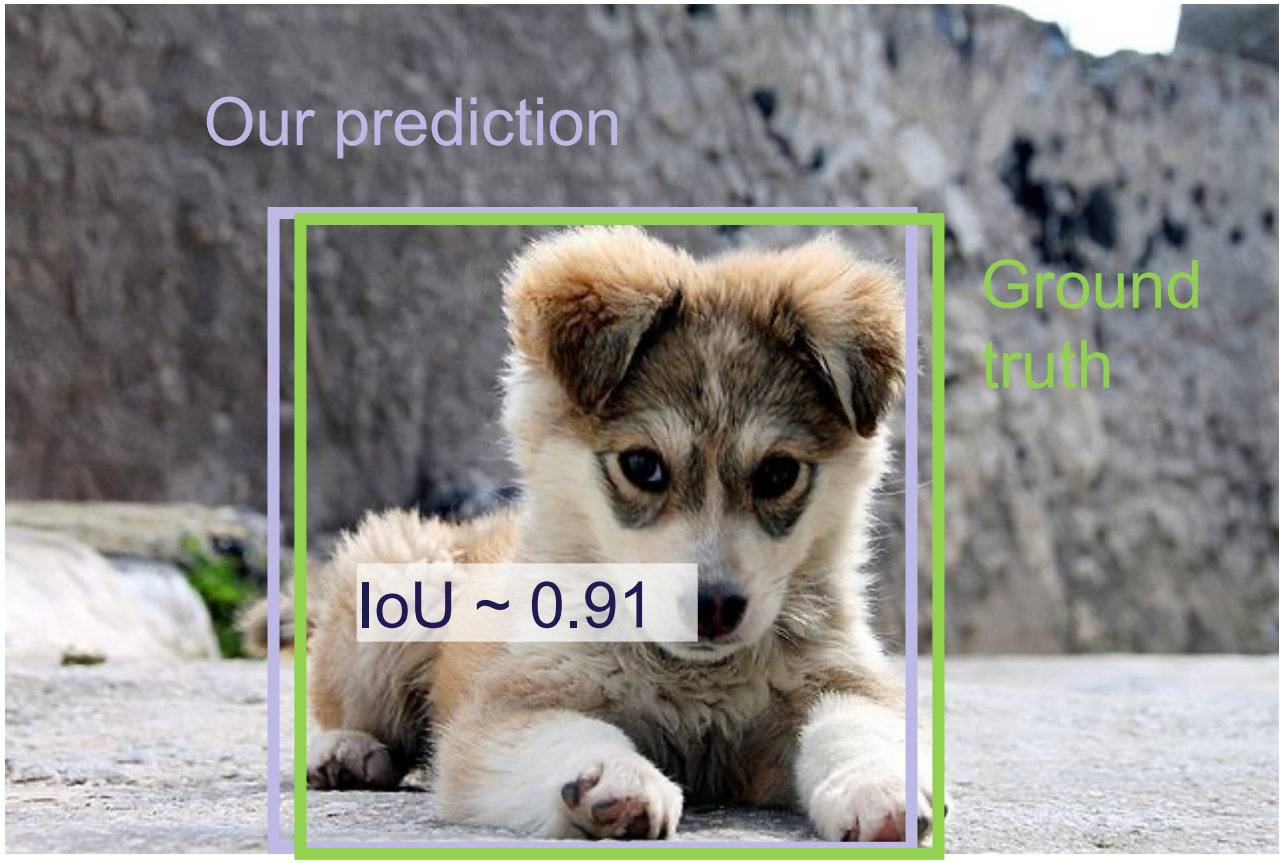
Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Evaluating detectors

- Let us assume that we have previously defined n ground-truth boxes
- How can we compare our prediction to the ground-truth box?
- Intersection Over Union

$$\frac{\text{Area of intersection}}{\text{Area of union}}$$

- IoU > 0.5: “decent”
- IoU > 0.7: “pretty good”
- IoU > 0.9: “almost perfect”

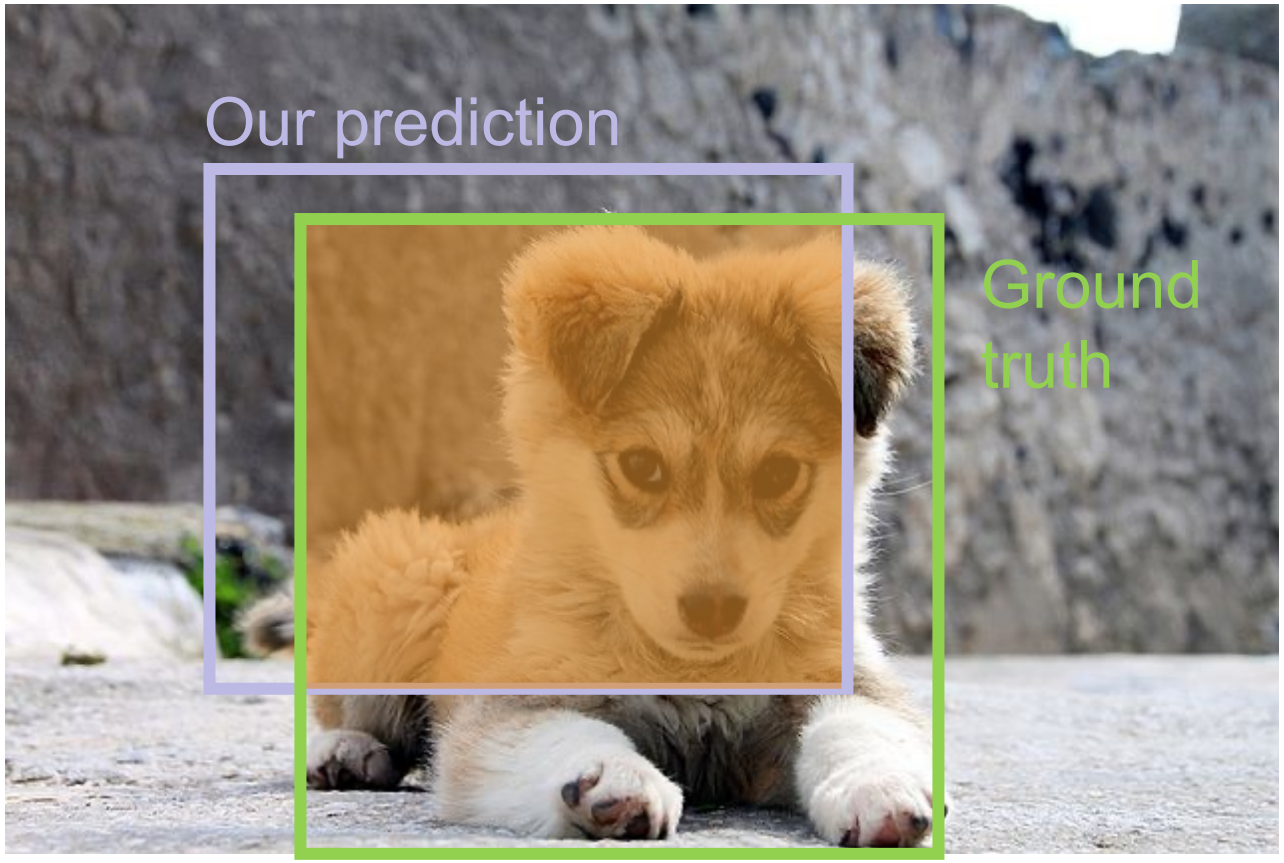


Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Evaluating detectors

- Let us assume that we have previously defined n ground-truth boxes
- How can we compare our prediction to the ground-truth box?
- Intersection Over Union

$$\frac{\text{Area of intersection}}{\text{Area of union}}$$



Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Performance measures

- Investigate which performance measures are common to use for your current tasks

- Examples:
- Detection: Intersection over Union, Precision, Recall, Mean Average Precision, …

- Classification: Confusion matrix, Accuracy, Precision, Recall, Sensitivity, Specificity, F1 Score, …

- Tracking: Average center error, Average overlap, Tracking length, HOTA, MOTA, MOTP, ID changes, …



Take-home messages

- Image quality comes from the camera: optics, exposure, lighting and sensor noise matter a lot. Garbage in, garbage out.
- Classical methods: fast, interpretable, no training data, but brittle and hand-tuned.
- Deep learning: powerful and general, but data-hungry and opaque, and it fails quietly out of distribution.
- Often times complete system use both classical methods and trained perception.



Next steps

- Szeliski: Computer Vision: Algorithms and Applications
- Torralba, Isola, Freeman: Foundations of Computer Vision
- Moeslund: Introduction to Video and Image Processing
- The OpenCV tutorials (docs.opencv.org)

